



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第六讲 微处理器设计

第1部分 单周期处理器

李学清

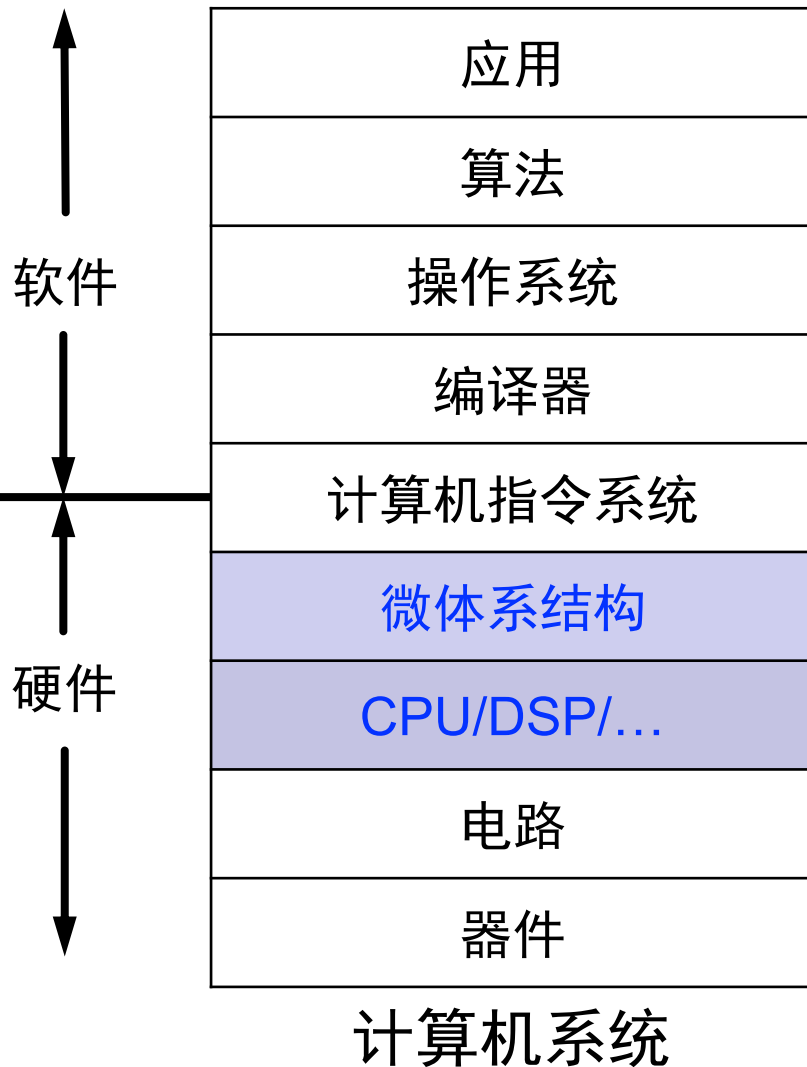
助教：陈仲豪*/唐文骏/曾令安

#腾讯会议：453-1488-6548

*本节内容负责答疑助教

2024年11月11日

微体系结构



周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

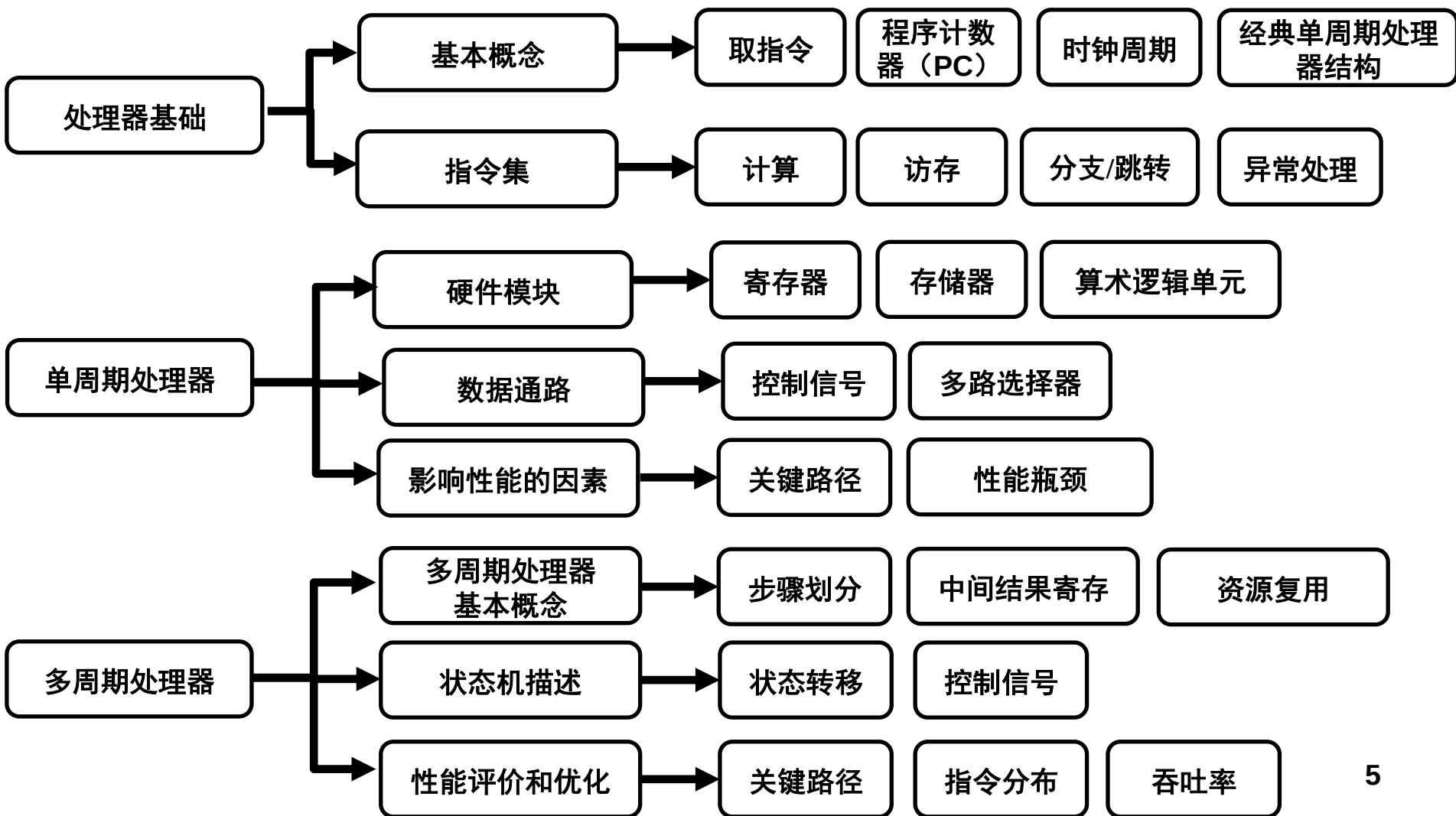
本节课学习目标

- **知识点：概念、原理和指标**
 - 高级语言 → 汇编语言
 - 硬件上的指令执行
- **能力**
 - 掌握设计和分析单周期处理器的方法
 - 掌握设计折衷和评估方法
- **思想**
 - 同步和结构化的计算
 - 折衷

本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

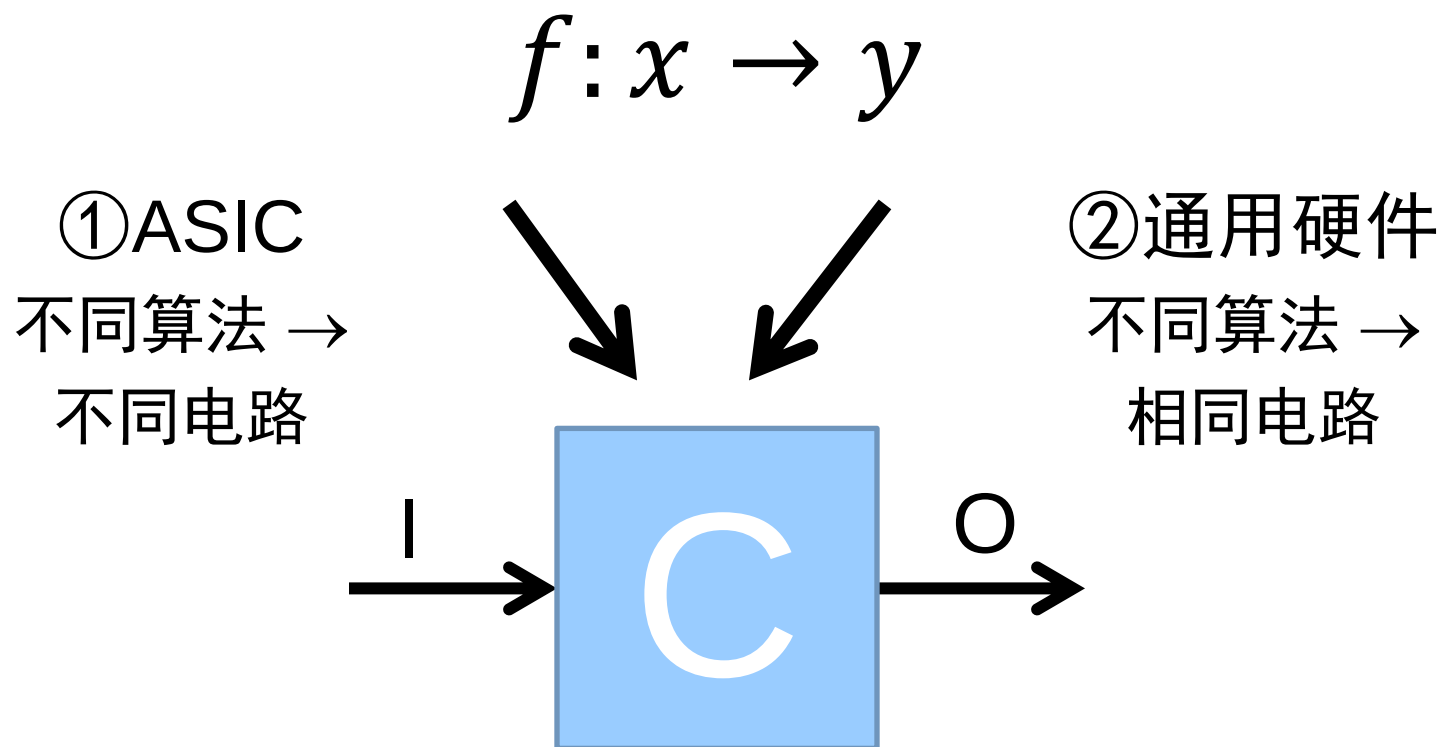
知识点框图



回顾：两种实现方法

① 定制硬件：一种算法，一种电路 (ASIC)

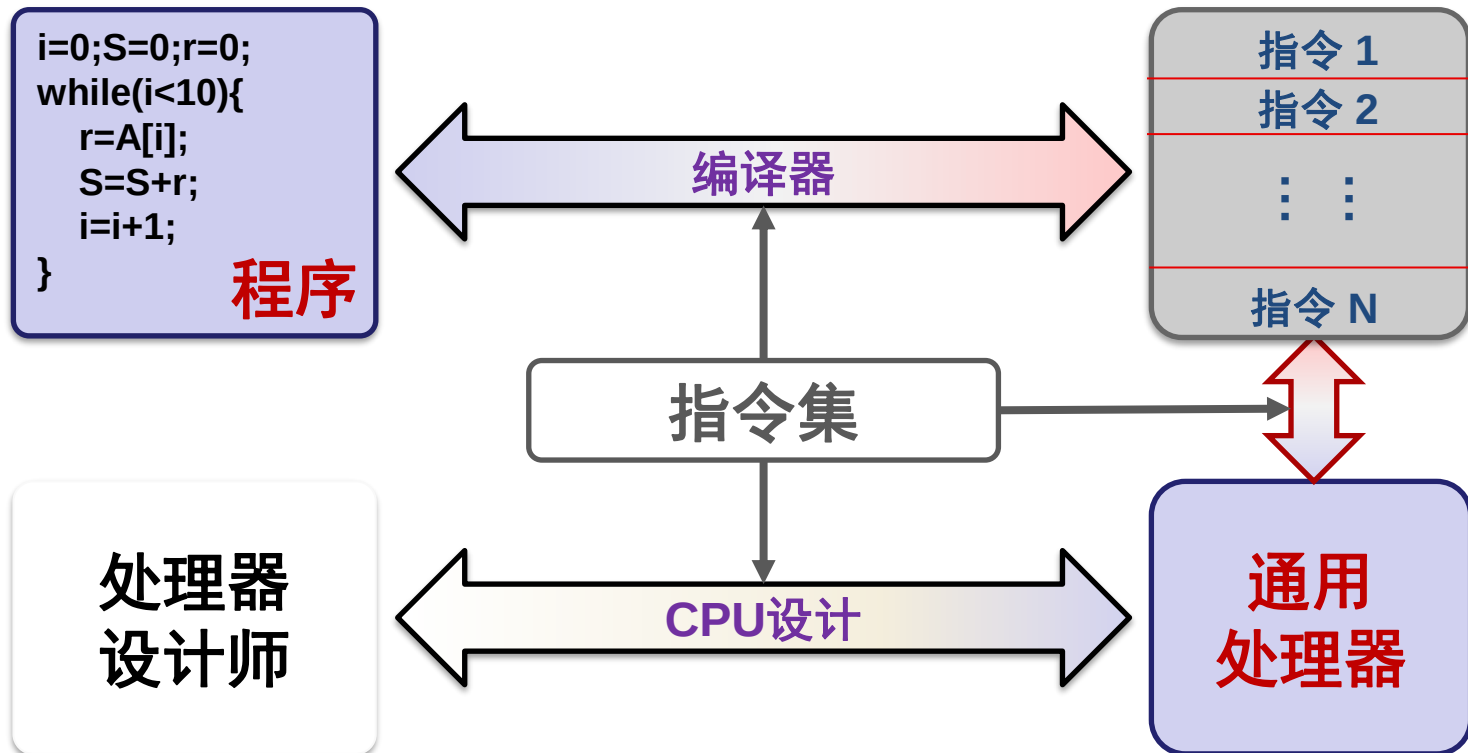
② 通用硬件：多种算法，一种电路



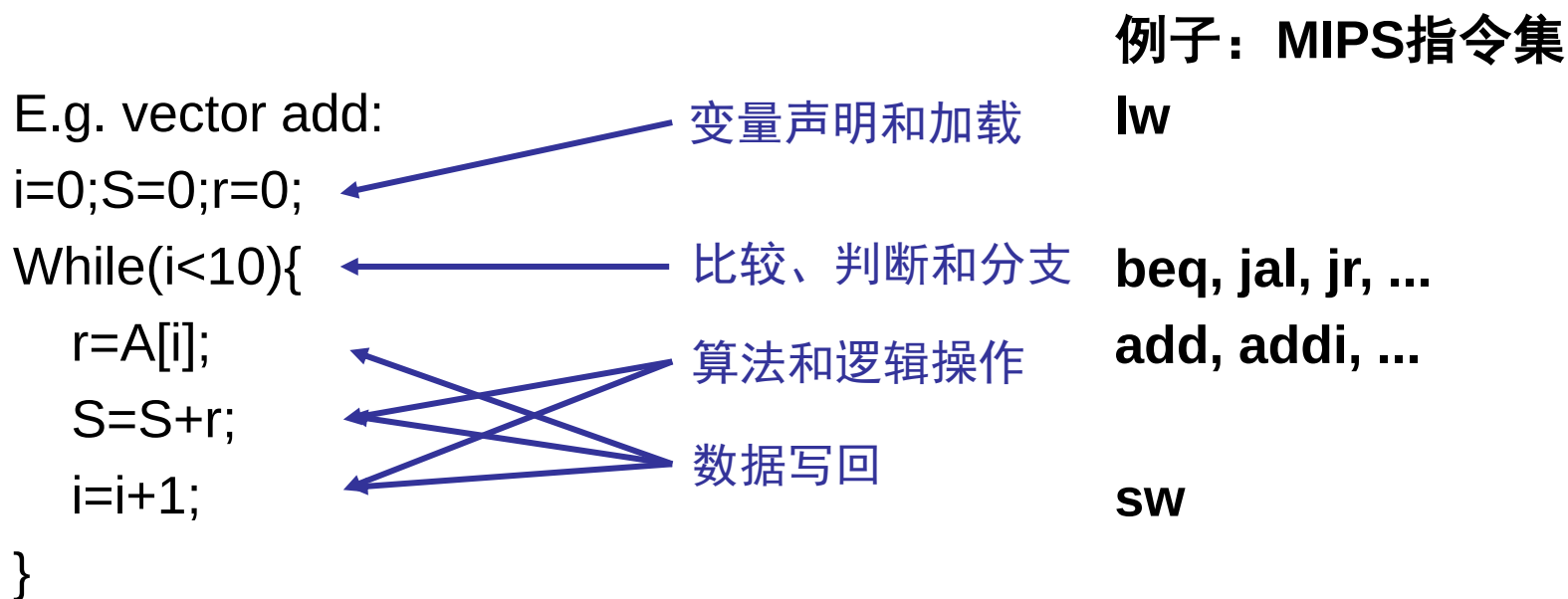
回顾：如何支持通用算法？

- 指令集的角色

- 软件和通用处理器之间的桥梁



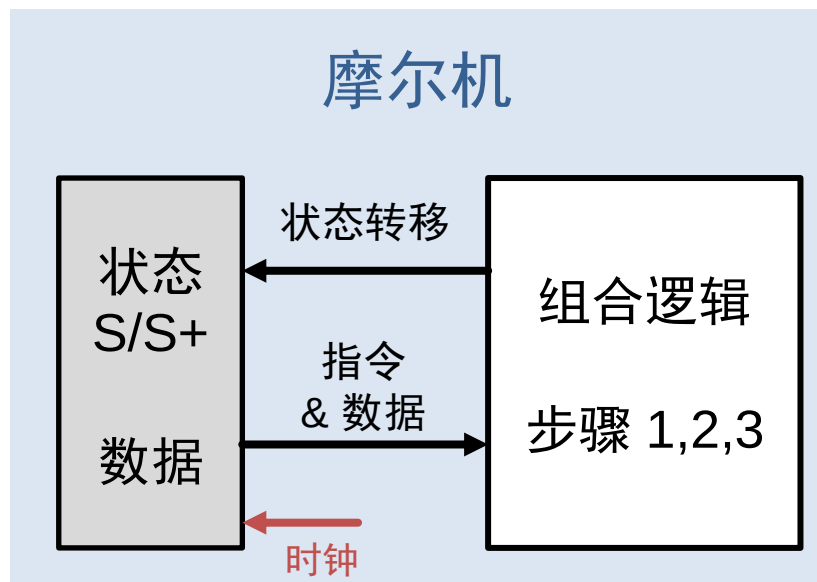
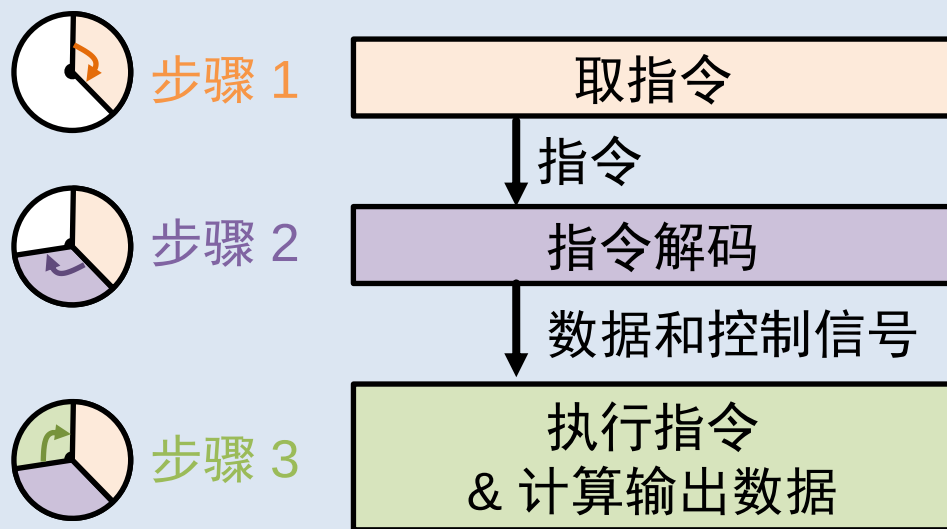
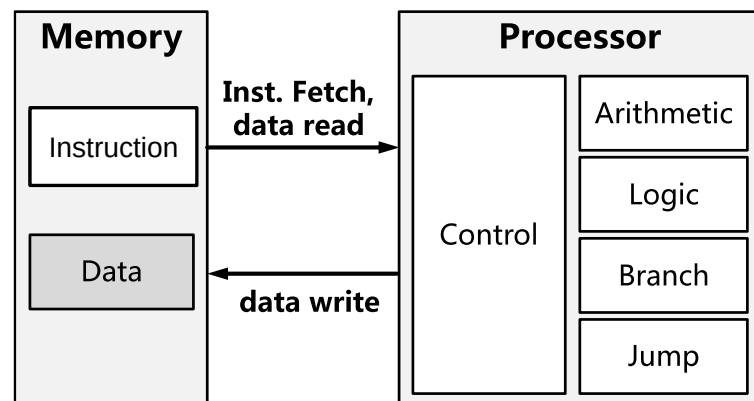
从指令集到处理器



处理器应当具有哪些功能？

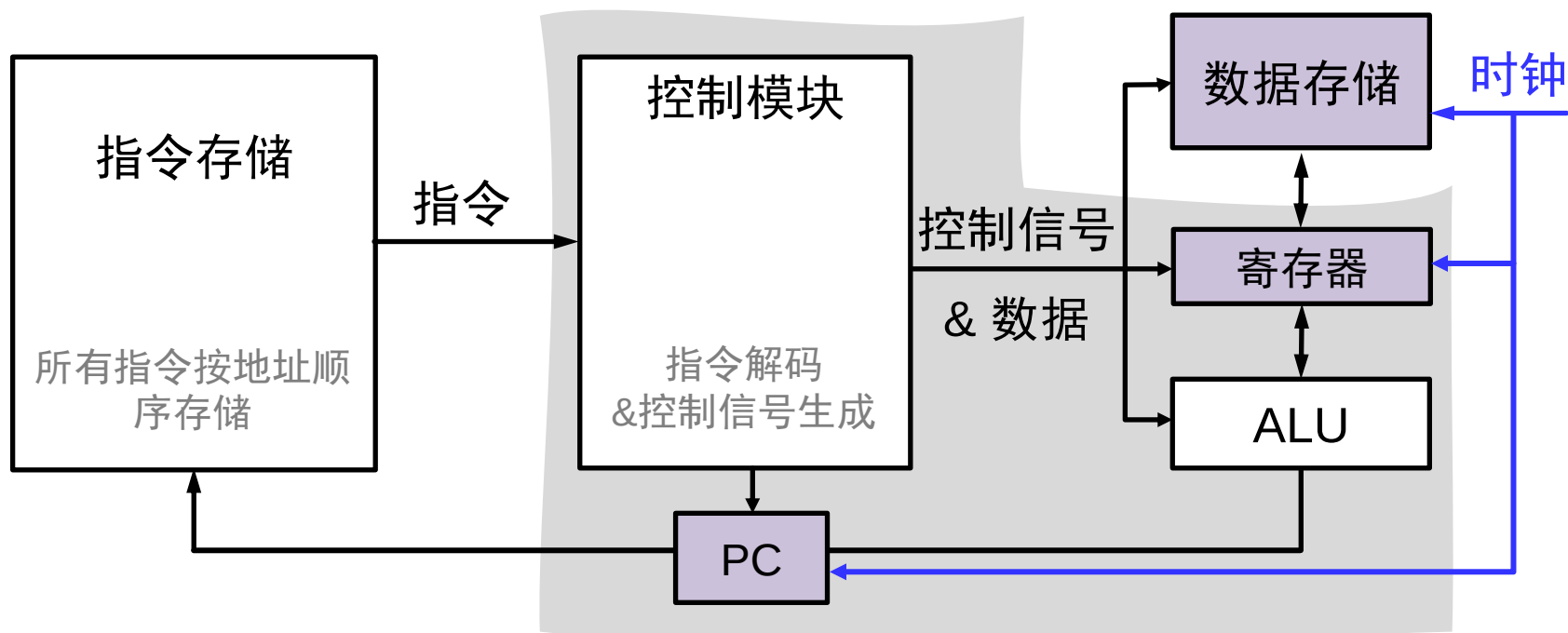
回顾：从专用芯片到处理器

- 如何让CPU支持指令集？
 - 明确指令操作步骤
 - 构造摩尔机



回顾：处理器硬件

- 通用架构

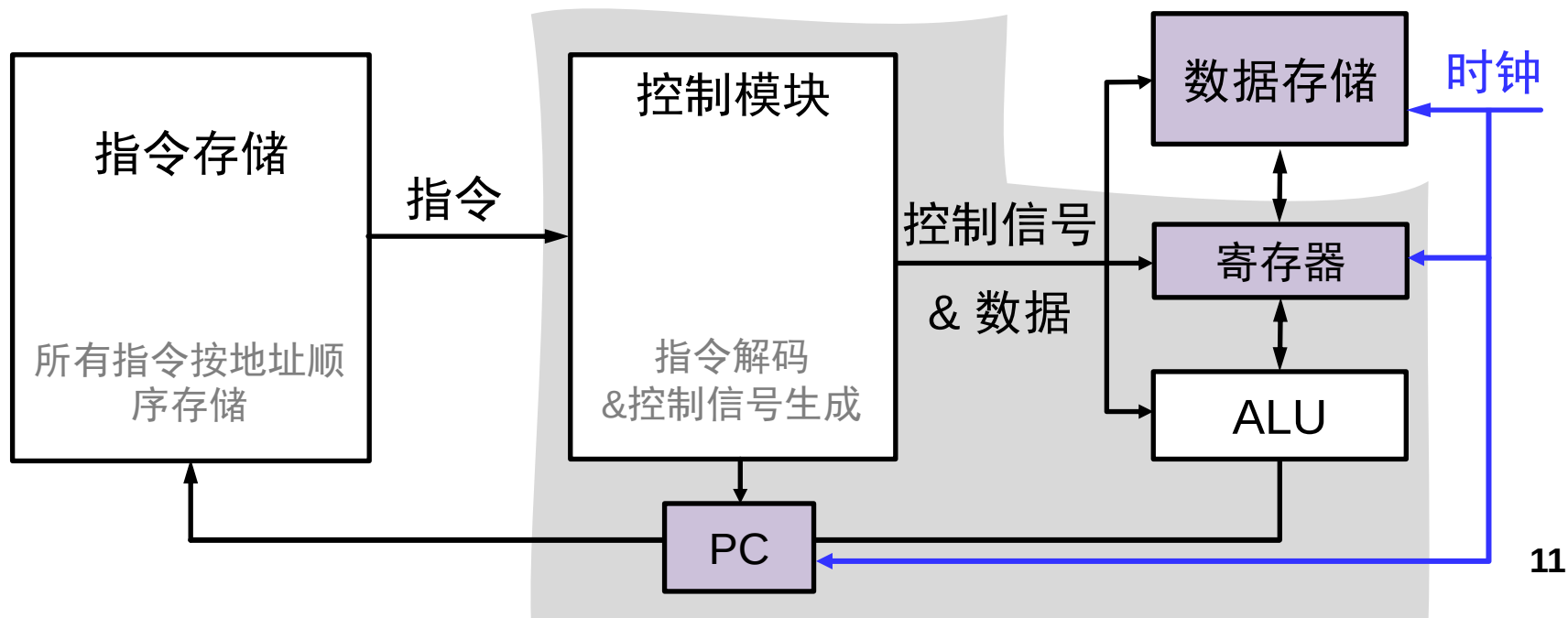
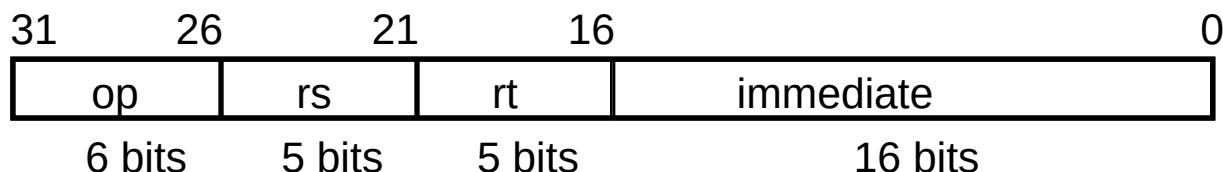


回顾：处理器硬件

• Load 指令

– $R[\underline{rt}] \leftarrow \text{Mem}[R[\underline{rs}] + \text{SignExt}[\text{imm16}]]$

e.g. $B = A[25]$

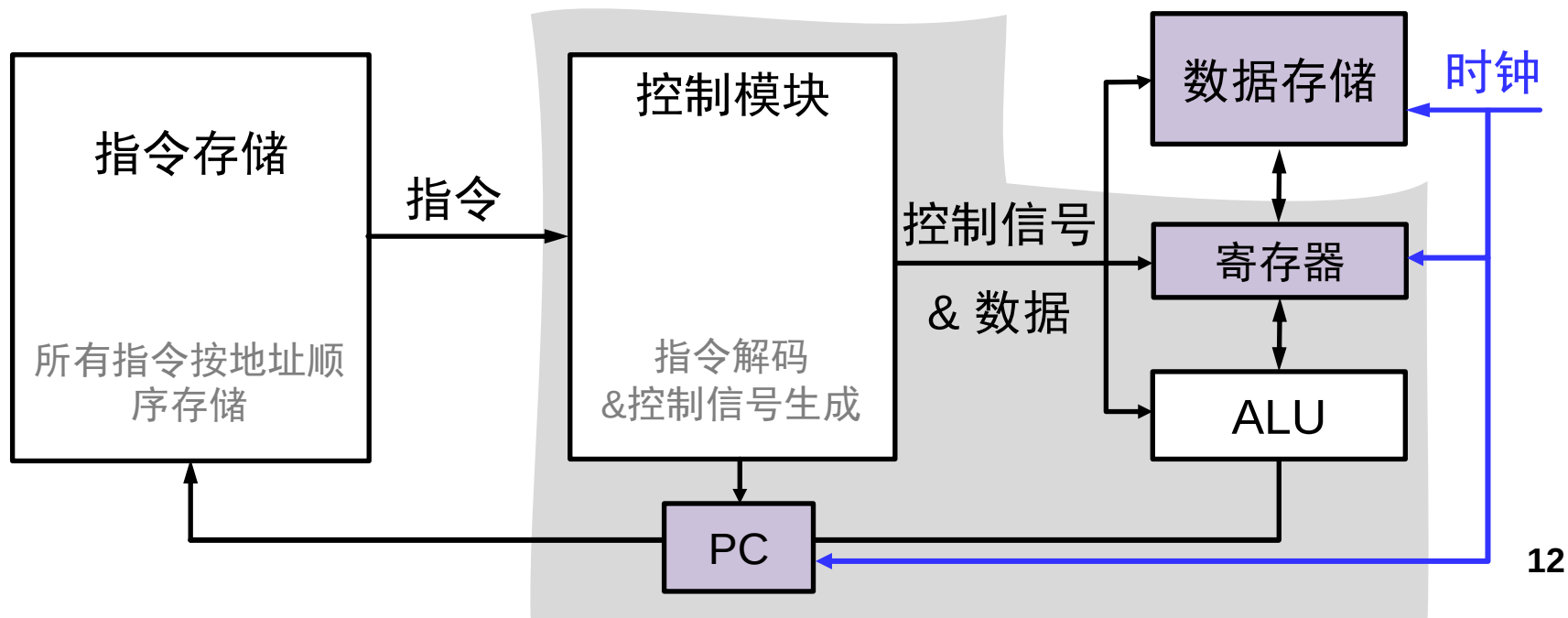
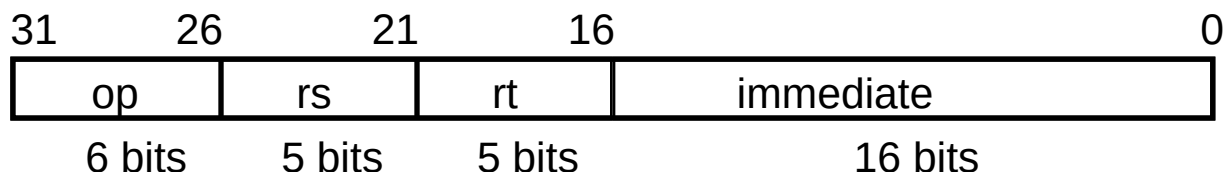


回顾：处理器硬件

• Store 指令

– $\text{Mem}[R[\text{rs}] + \text{SignExt}[\text{imm16}]] \leftarrow R[\text{rt}]$

e.g. $A[25] = B$

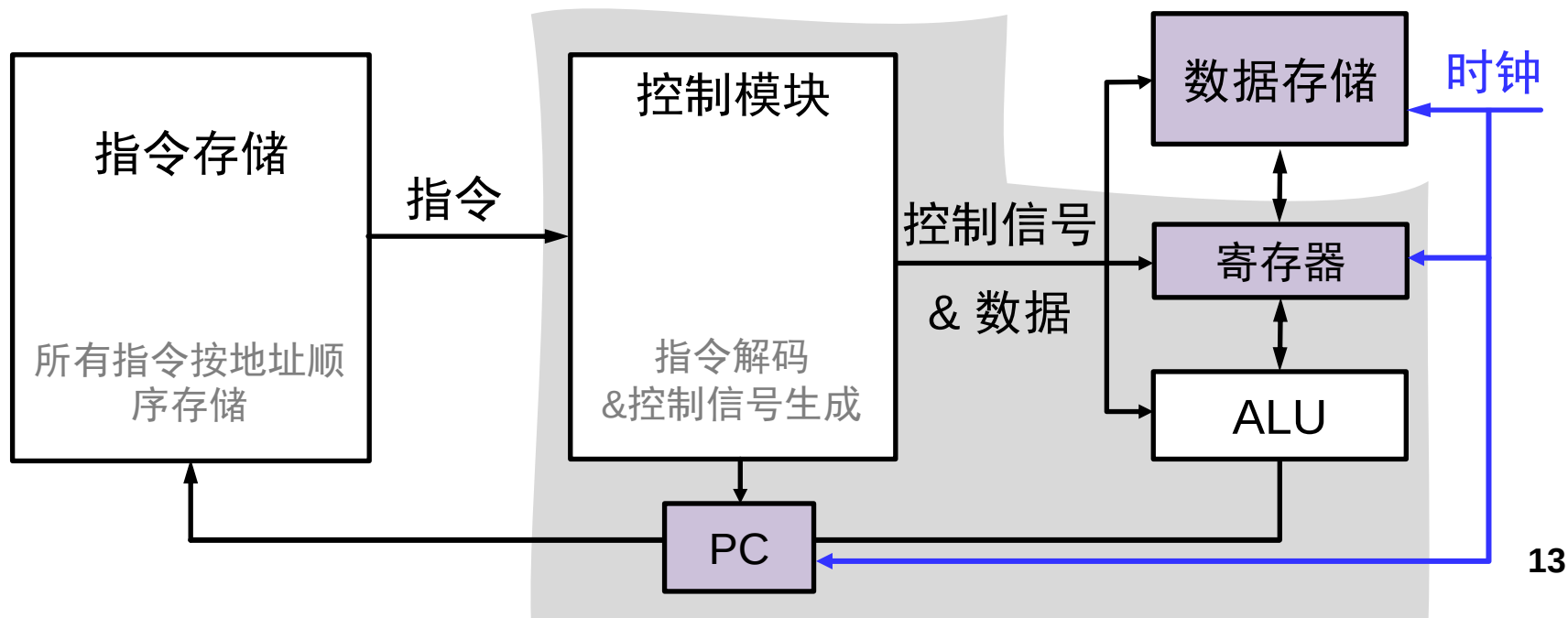
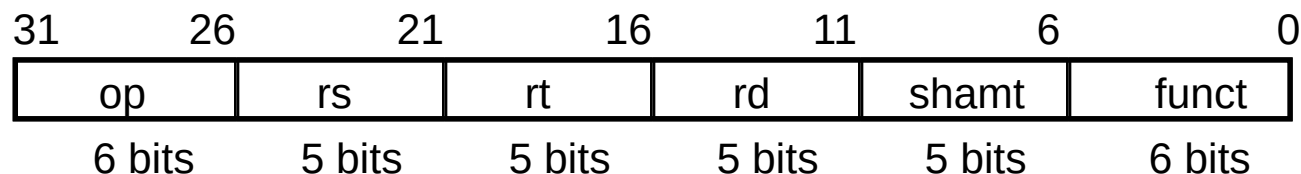


回顾：处理器硬件

• 算术&逻辑运算

– $R[rd] \leftarrow R[rs] \text{ op } R[rt]$

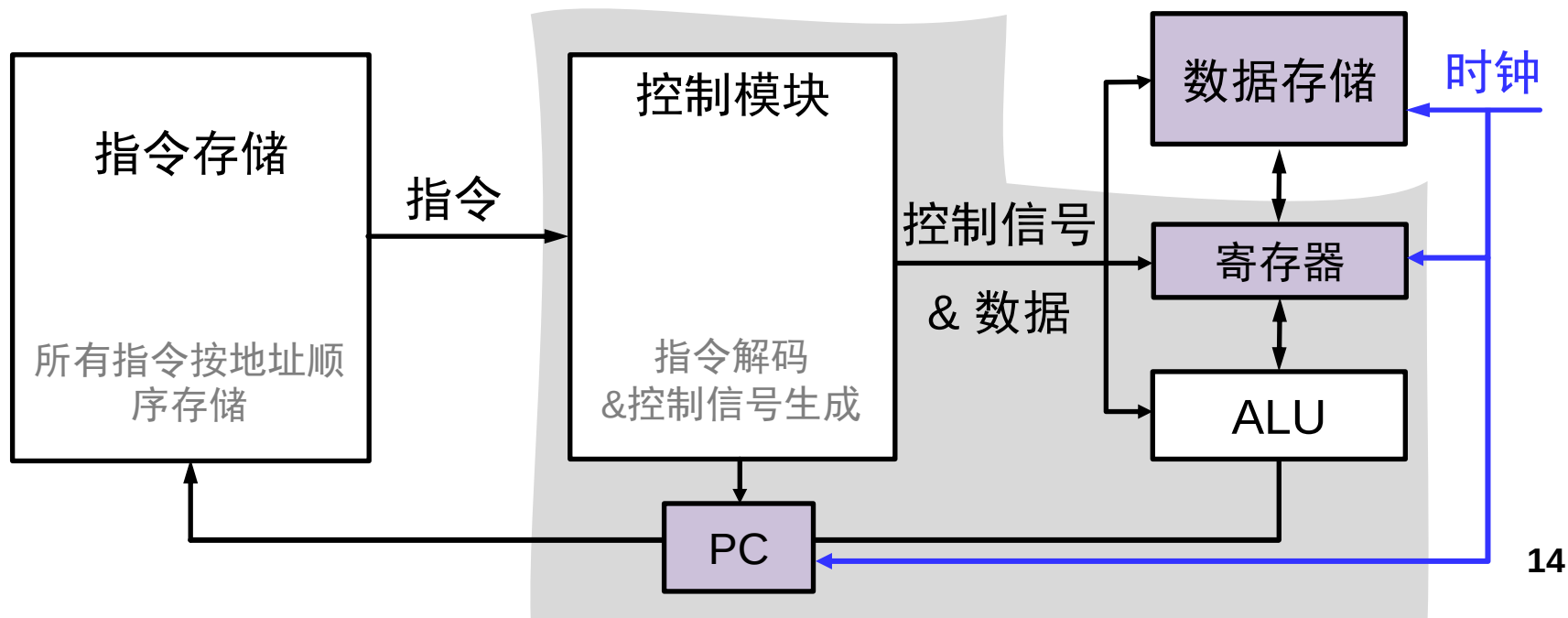
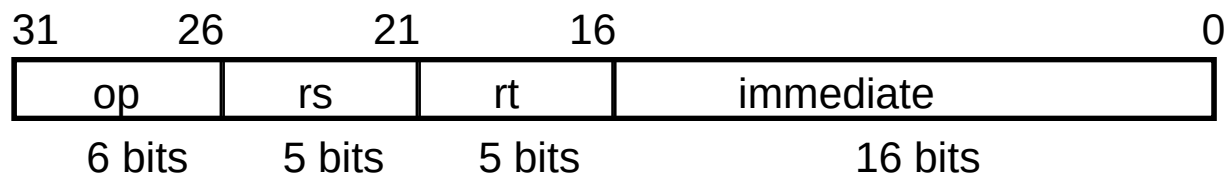
e.g. $A=B+C$



回顾：处理器硬件

- **Branch 分支: beq rs, rt, imm16**

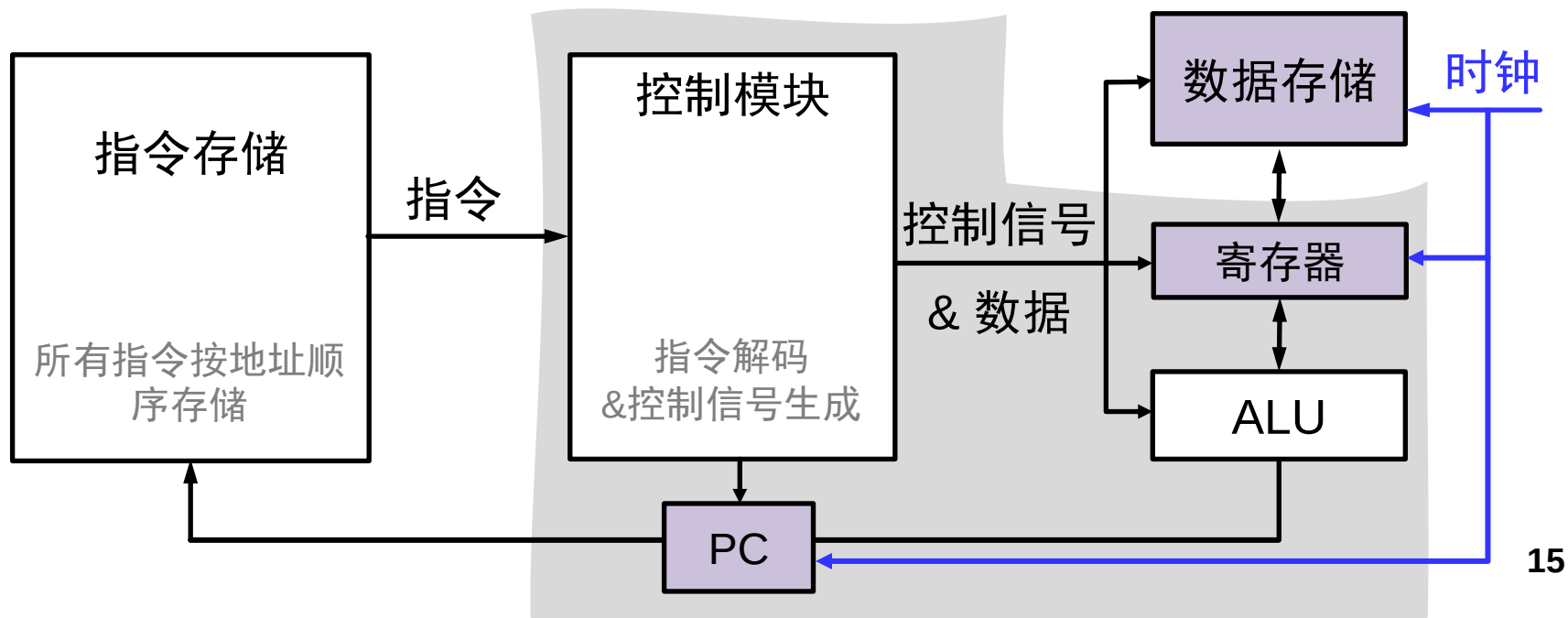
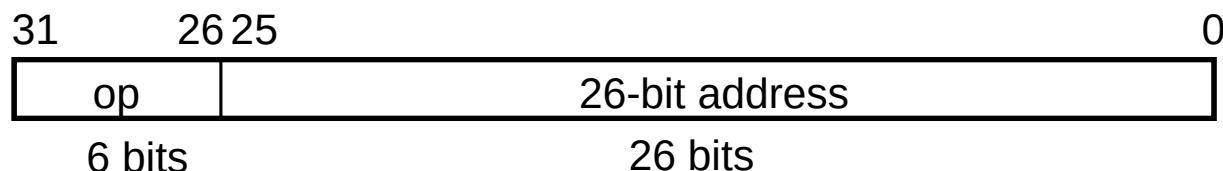
- $PC = PC + 4$ ($rs \neq rt$) or $PC + 4 + \{SignExtImm \ll 2\}$ ($rs == rt$)



回顾：处理器硬件

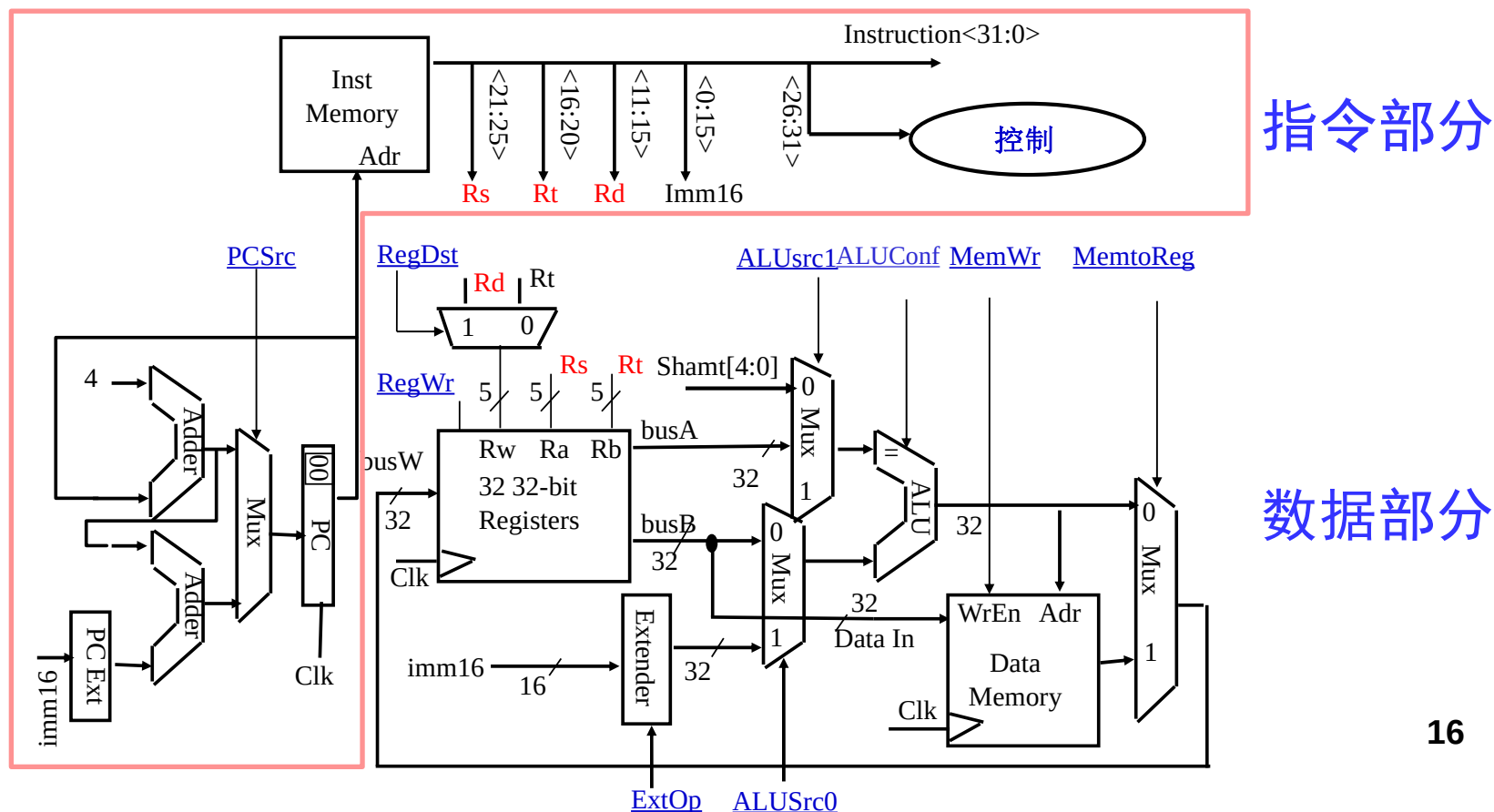
• Jump 跳转

- 跳转地址: $PC = \{ (PC+4)[31:28], \text{target}, 00 \}$



MIPS单周期处理器架构

- 根据上述数据通路的分析，可以设计出具体MIPS单周期处理器（本节课讲授重点）



本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

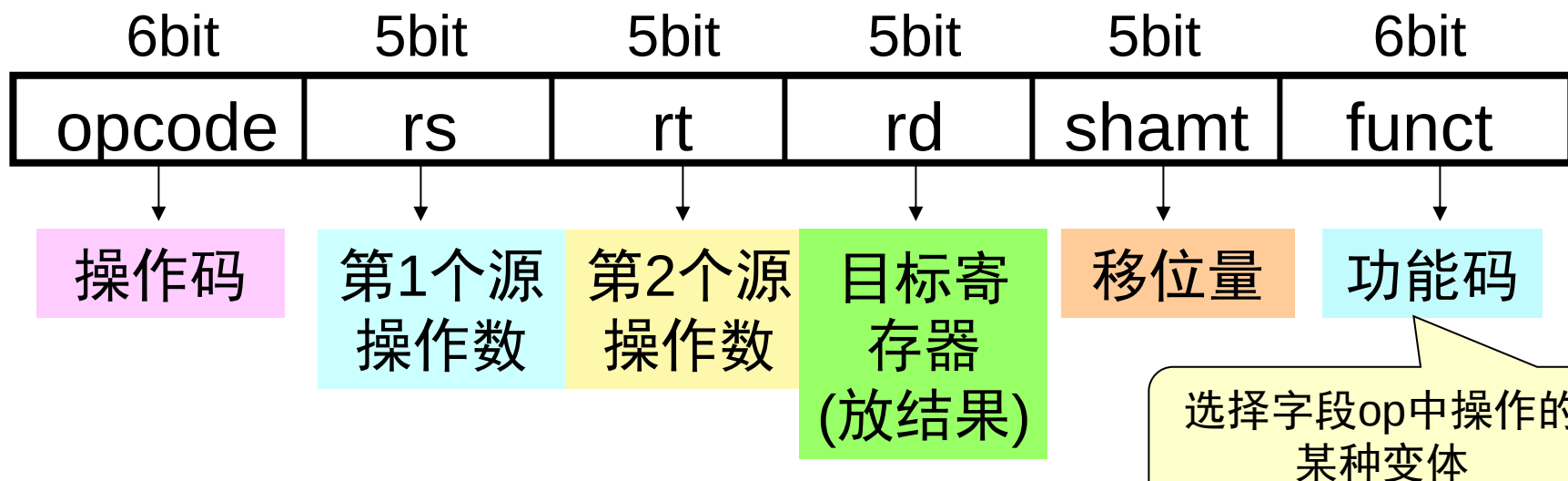
回顾：MIPS指令集架构

- 完备指令集设计应支持计算、访存与决策三类功能
- 在MIPS架构中
 - 计算与访存相关指令根据操作数类型分为两类
 - 操作数以寄存器存储的数据为主：R型指令
 - 操作数涉及到指令字段表示的立即数：I型指令
 - 分支循环等与跳转相关的指令：J型指令

回顾：MIPS指令集架构

- R型指令

- rs: source, rt: target, rd: destination
- opcode与funct共同确定该R型指令的具体功能



- 例子：加法指令

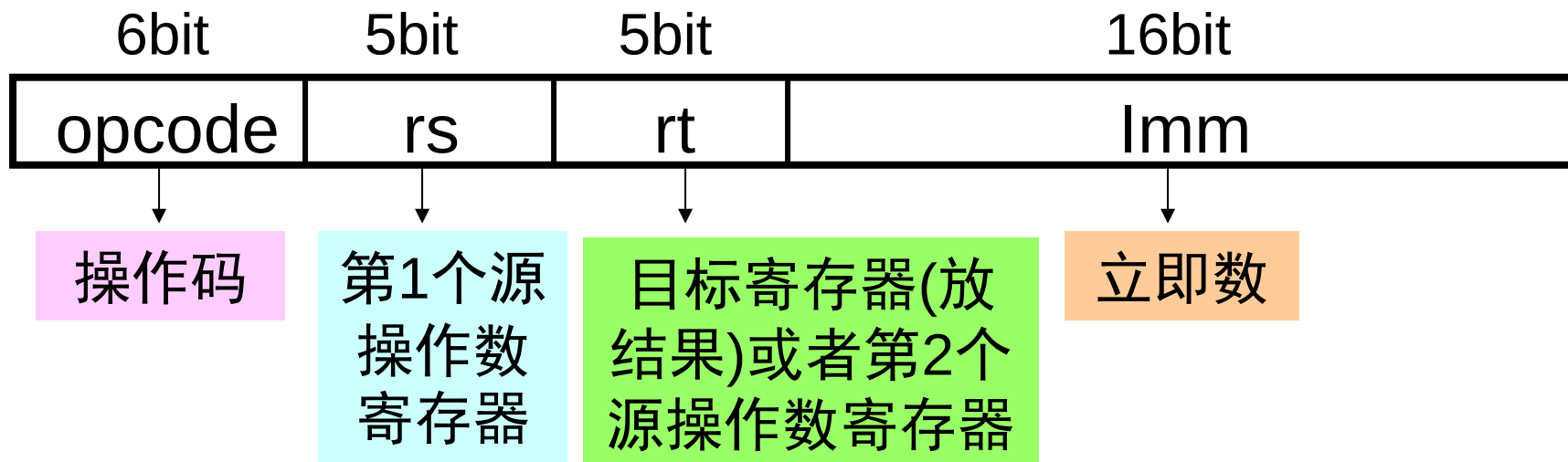
add rd, rs, rt

如add \$8, \$9, \$10

回顾：MIPS指令集架构

• I型指令

- rt同时包含源操作数和目标寄存器的含义（根据具体指令而定）
- 立即数字段可表示**16bit常数**，也可表示地址偏移量

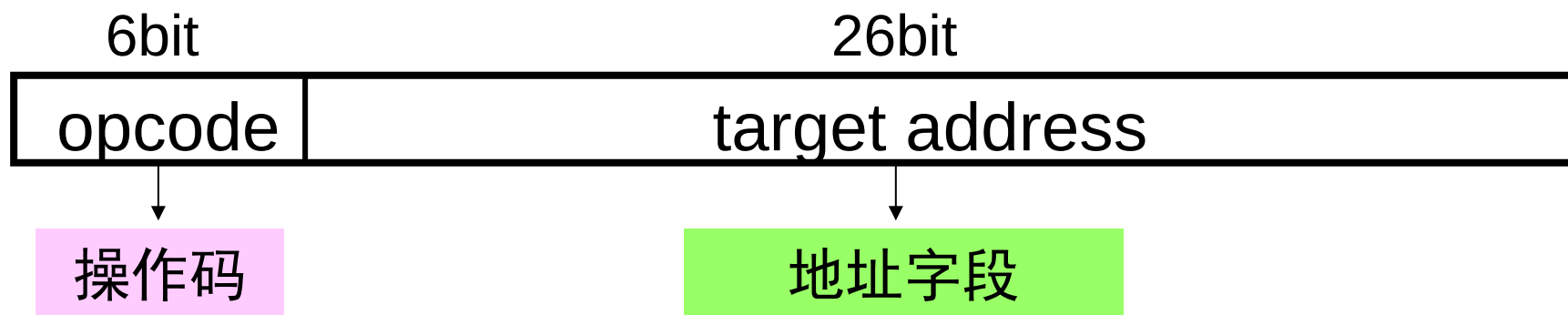


- 例子：装入/存储指令: `sw/lw rt, immediate(rs)`
- 与立即数做计算: `addi rt, rs, 50`

回顾：MIPS指令集架构

- J型指令

- opcode与R/I型指令类似
- 其他字段：给出跳转的目的地址，可以实现很大的跳转范围



例子：跳转指令，j 10000

回顾:MIPS指令格式总结

- **I型：用于有立即数的指令：lw sw beq bne**
 - lw/sw: load word; store word
 - beq/bne: branch on equal; branch on not equal
 - opcode: 除000000, 00001x, 0100xx外
- **J型：用于跳转，j, jal**
 - j: jump; jal: jump and link;
 - opcode: 000010, 000011
- **R型：所有其他指令**
 - opcode: 000000, jr: jump register

opcode: 区分不同
指令的关键

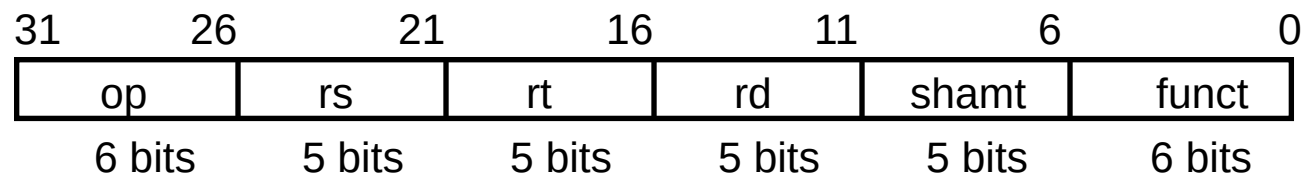
知识回顾

- **MIPS指令功能**

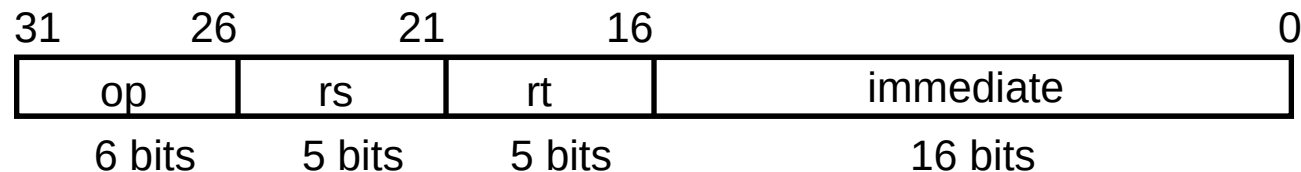
- 数据运算指令： add, sub, and, not, ...
- 数据传送指令： lw, sw, lui, ...
- 分支与跳转指令： beq, bne, slt, j, jr, jal

回顾：MIPS指令集架构

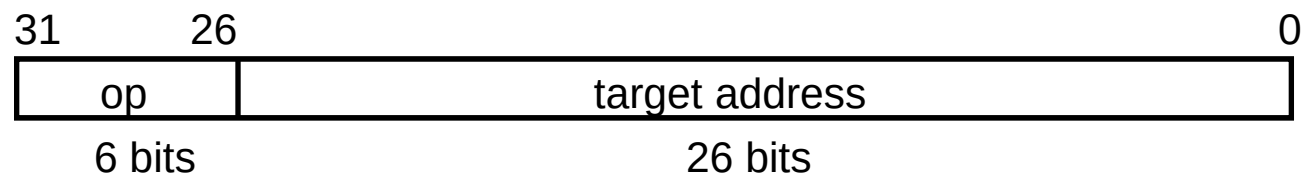
– R-type



– I-type



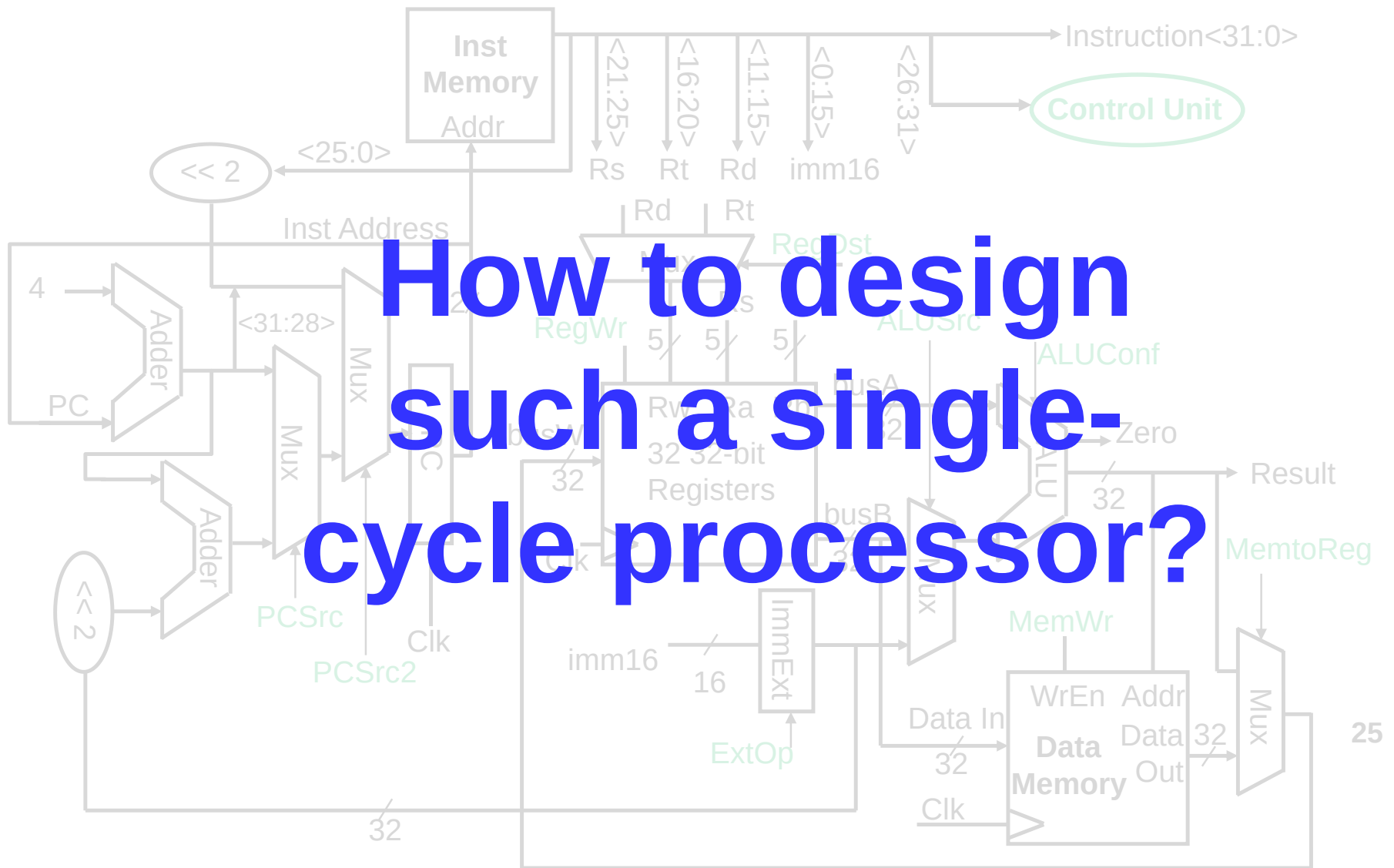
– J-type



各个域分别是

op	指令操作标识
rs, rt, rd	源,目的寄存器地址
shamt	移位量
funct	对op类操作，确定详细的操作类型
immediate	立即数或者地址
target address	目标地址

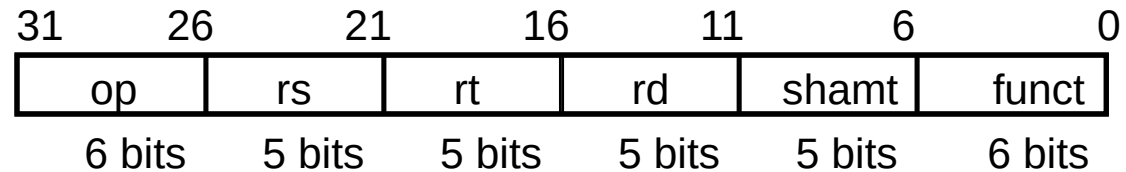
单周期处理器构成



设计目标：MIPS指令子集

- ADD, SUB, AND, SLT

- add rd, rs, rt
- sub rd, rs, rt
- and rd, rs, rt
- slt rd, rs, rt



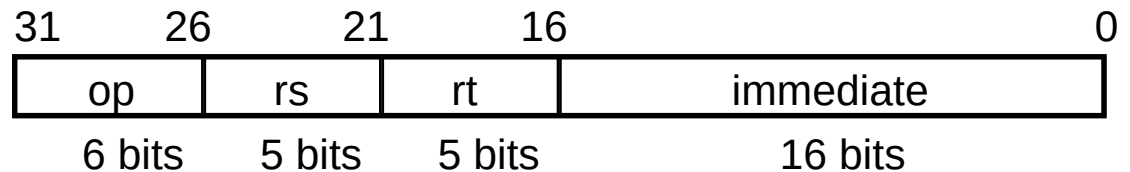
R-type

- OR Immediate:

- ori rt, rs, imm16

- LOAD and STORE Word

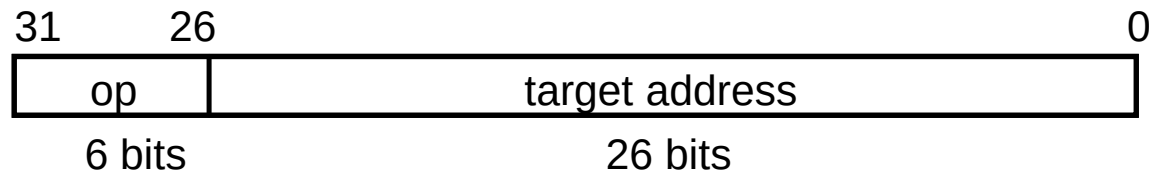
- lw rt, imm16(rs)
- sw rt, imm16(rs)



I-type

- BRANCH and JUMP:

- beq rs, rt, imm16
- j Label



J-type

根据指令集分析数据通路组成

存储单元：输入、输出和内部状态

- 存储器(Memory)
 - 一个指令存储器、一个数据存储器
- 寄存器(32 x 32bit Register File, RF)
 - 可以为 rs 提供数据读取
 - 可以为 rt 提供数据读取
 - 可以为 rt 或者 rd 提供数据写入
- 程序计数器(Program Counter, PC)

- ADD , SUB, AND, SLT
 - add rd, rs, rt
- OR Immediate:
 - ori rt, rs, imm16
- LOAD and STORE Word
 - lw rt, imm16(rs)
- BRANCH, JUMP
 - beq rs, rt, imm16
 - j Label

根据指令集分析数据通路组成

计算单元（组合逻辑部分）

- 算术/逻辑运算单元（ALU）

- 两个寄存器之间或者是一个寄存器和一个经过扩展后的立即数之间

- 扩展电路

- 为立即数提供扩展支持

- PC更新电路

- 可以加4或者加上一个立即数扩展
- 或者j跳转更新

- ADD , SUB, AND, SLT
 - **add** rd, rs, rt
- OR Immediate:
 - **ori** rt, rs, imm16
- LOAD and STORE Word
 - **lw** rt, imm16(rs)
- BRANCH, JUMP
 - **beq** rs, rt, imm16
 - **j** Label

本节课目录

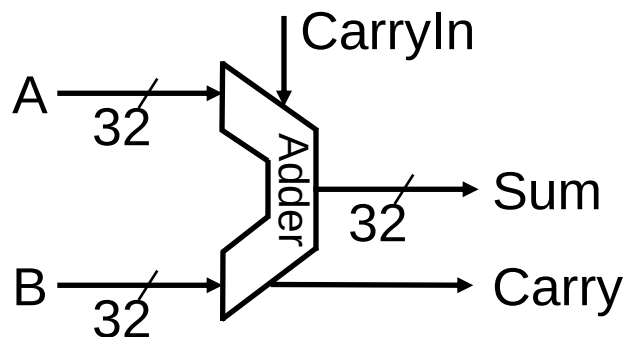
- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

确定数据通路的组成模块

- 组合逻辑电路部分
 - “计算”为主，不同于单纯的“逻辑”
- 存储单元
 - “数据”为主，不同于单纯的“状态”

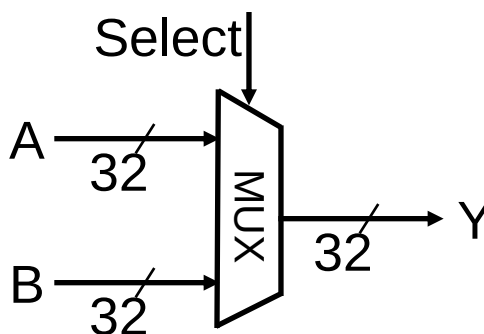
组合逻辑部分

- 加法器



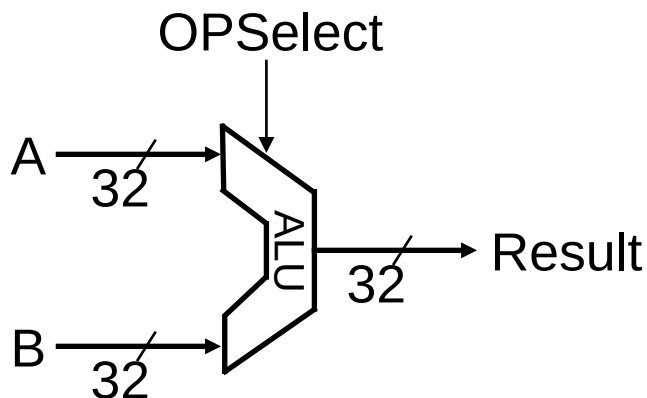
提供32位
加法运算

- 多路选择器



提供多路
输入选择

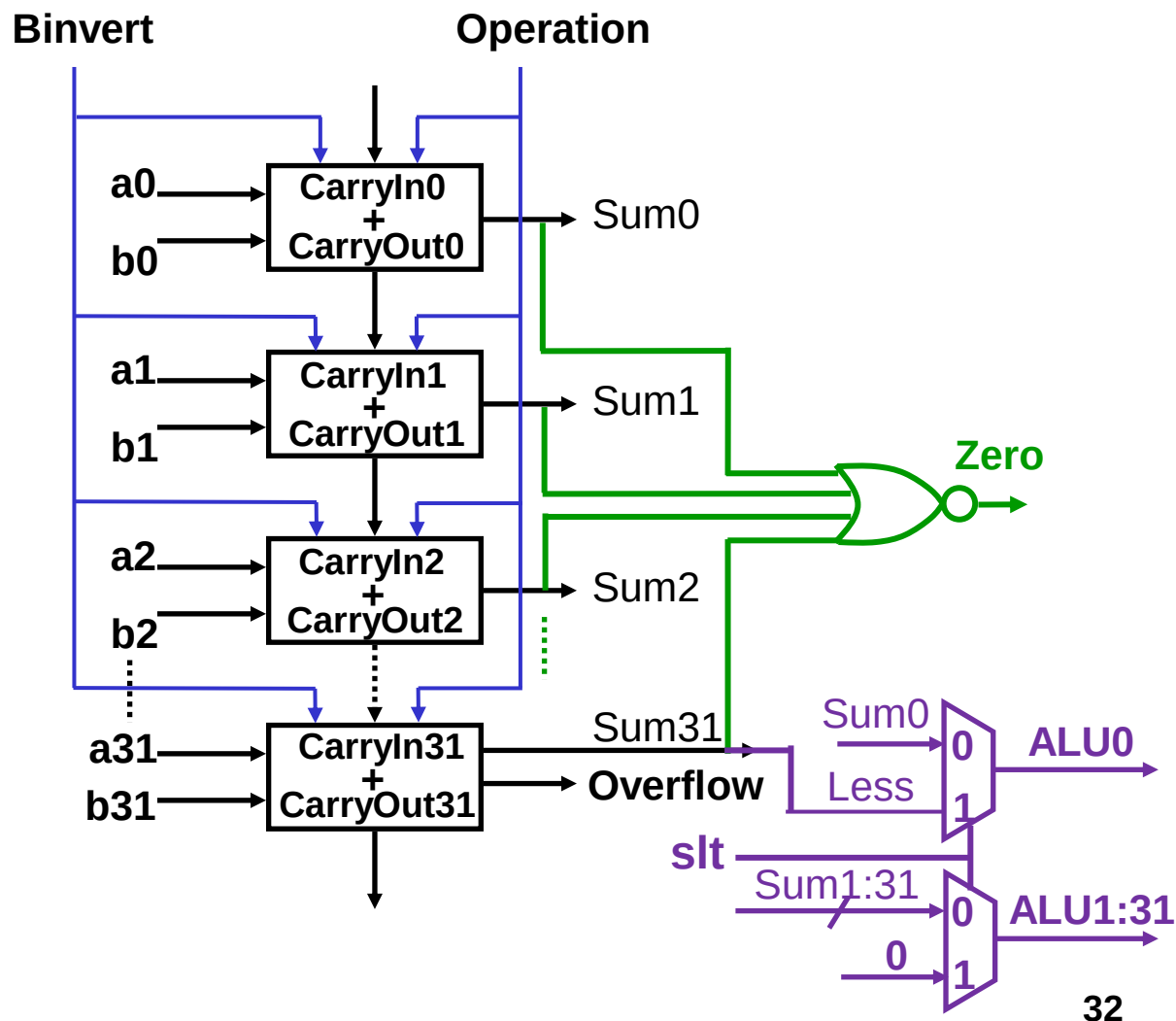
- 算术逻辑单元



实现算术
逻辑运算

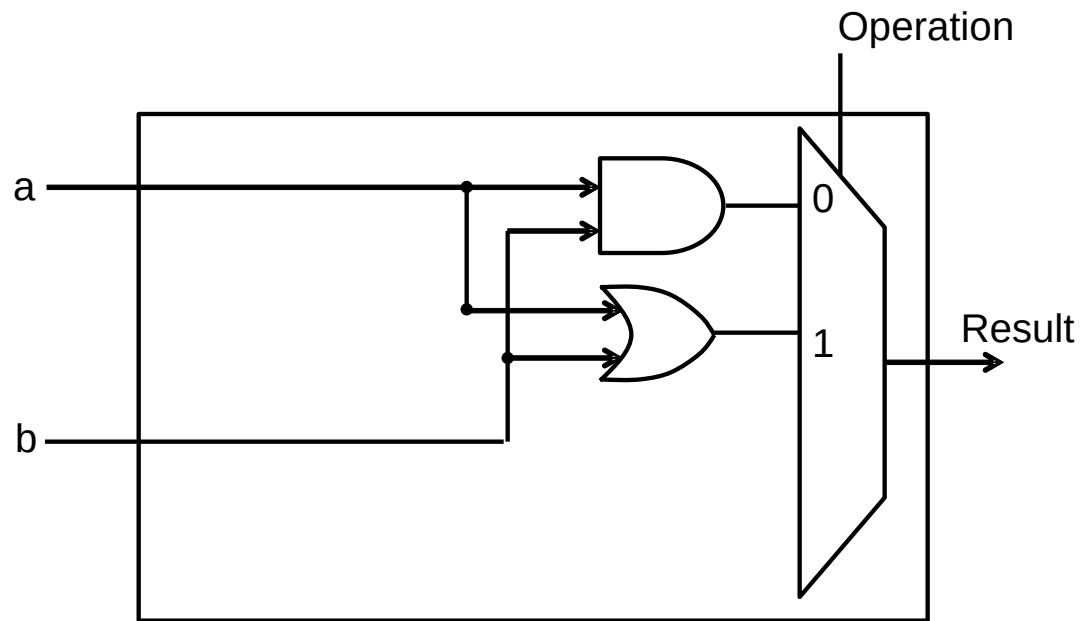
32位ALU设计

- 应具备的功能
 - add/sub/and/or
 - 支持溢出检测
 - 支持beq
 - slt



1位ALU电路

按位AND/OR



AND/OR Operation

1位ALU电路

按位AND/OR

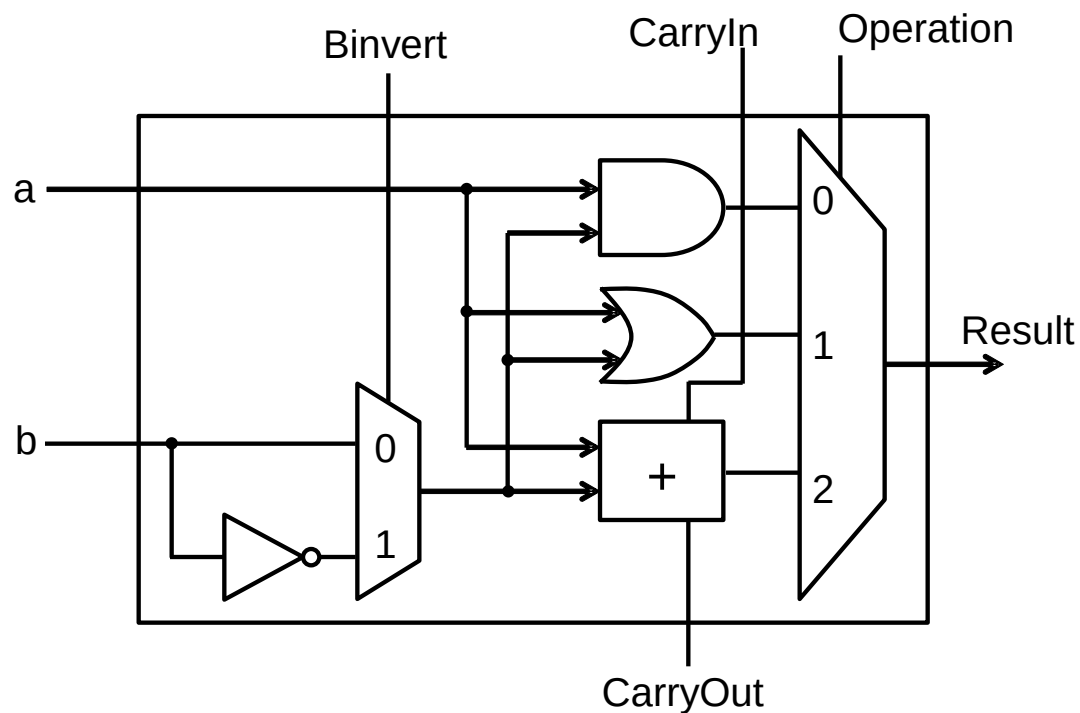
做加法：

Binvert=CarryIn=0

做减法：

Binvert=? CarryIn=?

回顾补码的概念！



可实现与、或、加法

1位ALU电路

按位AND/OR

做加法：

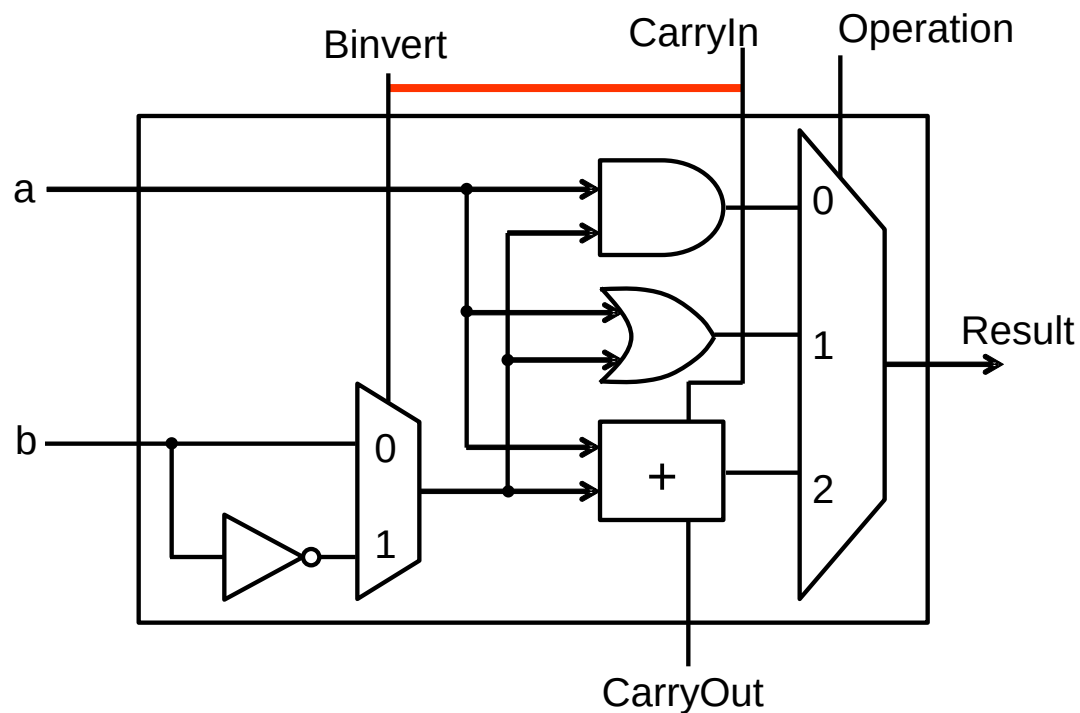
$\text{Binvert} = \text{CarryIn} = 0$

做减法：

$\text{Binvert} = ? \quad \text{CarryIn} = ?$

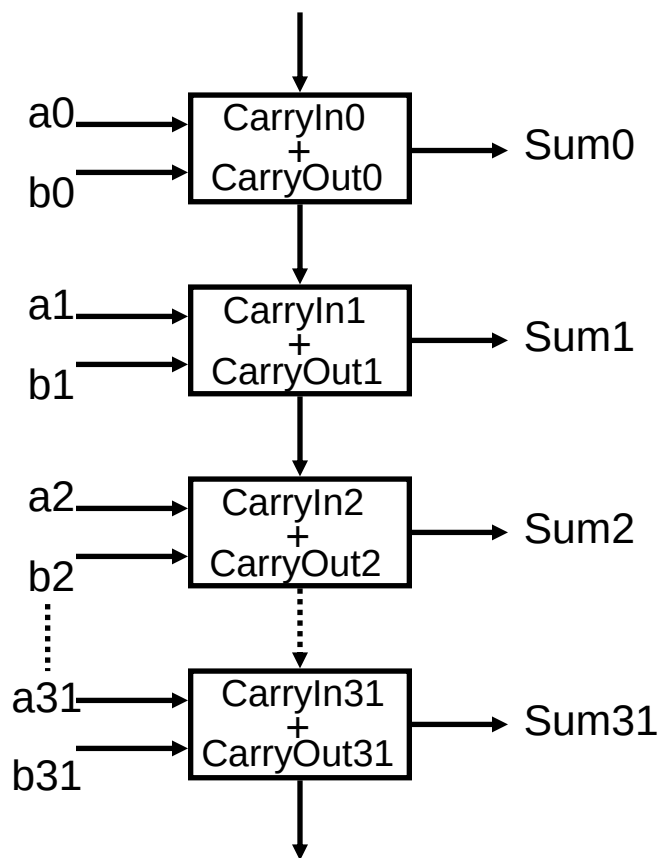
$\text{Binvert} = \text{CarryIn} = 1$

Bnegate：控制简化



可实现与、或、加/减法

N位加法器



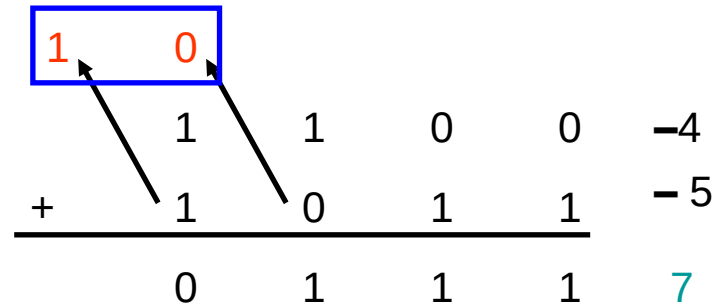
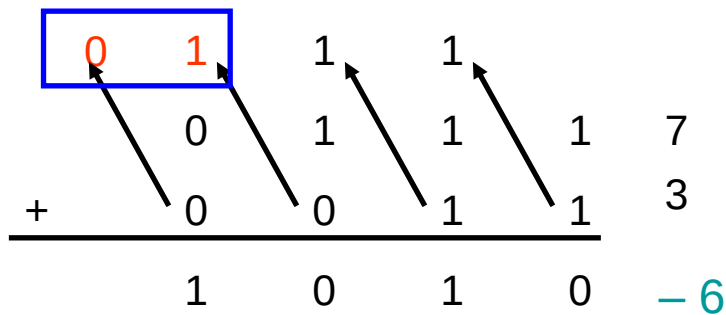
行波进位加法器
(ripple carry adder)

- 大多数MIPS指令都可以用这个ALU实现

32-bit ALU: 溢出检测

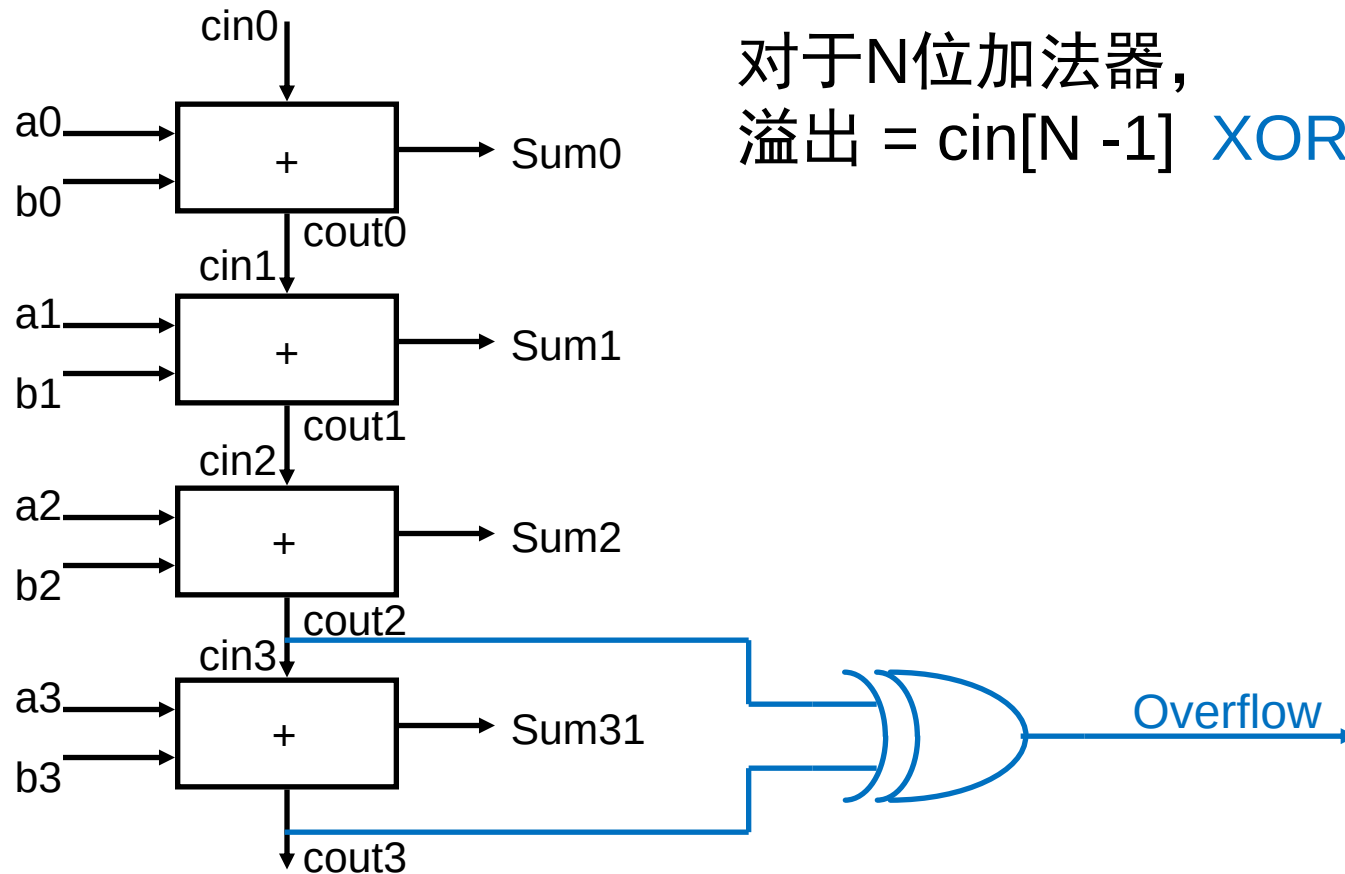
- Add, Sub 应支持溢出检测
- 溢出可以用如下方法检测:

输入到最高位的进位 != 从最高位输出的进位
(Carry into MSB != Carry out of MSB)

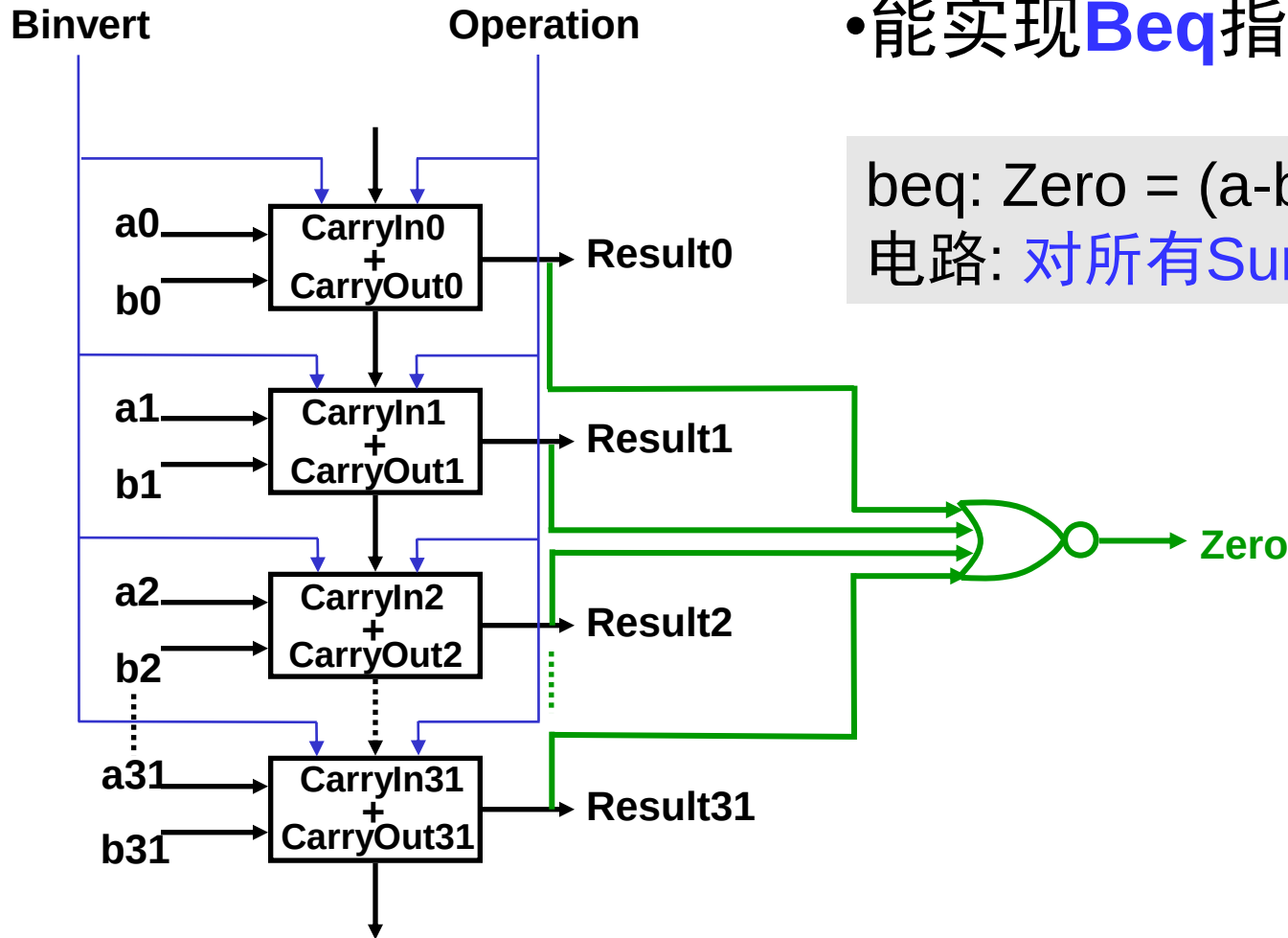


32-bit ALU: 溢出检测

溢出检测逻辑



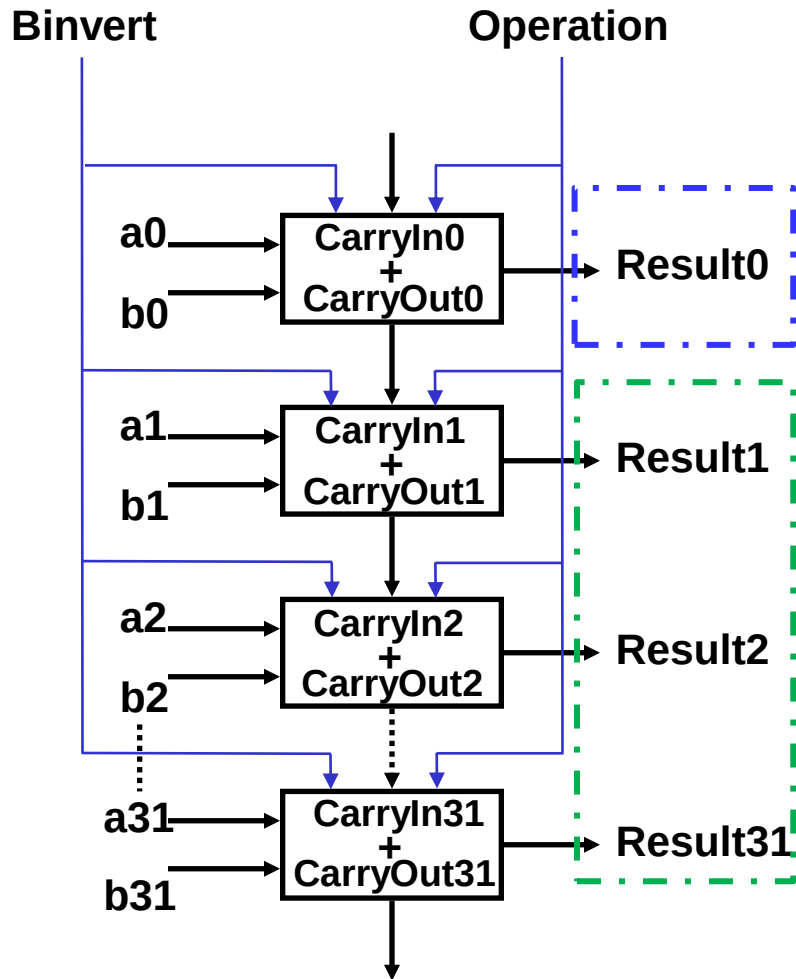
32-bit ALU: Beq



• 能实现 **Beq** 指令吗？

beq: Zero = (a-b == 0) ? 1 : 0;
电路: 对所有Sum结果求NOR

32-bit ALU: slt



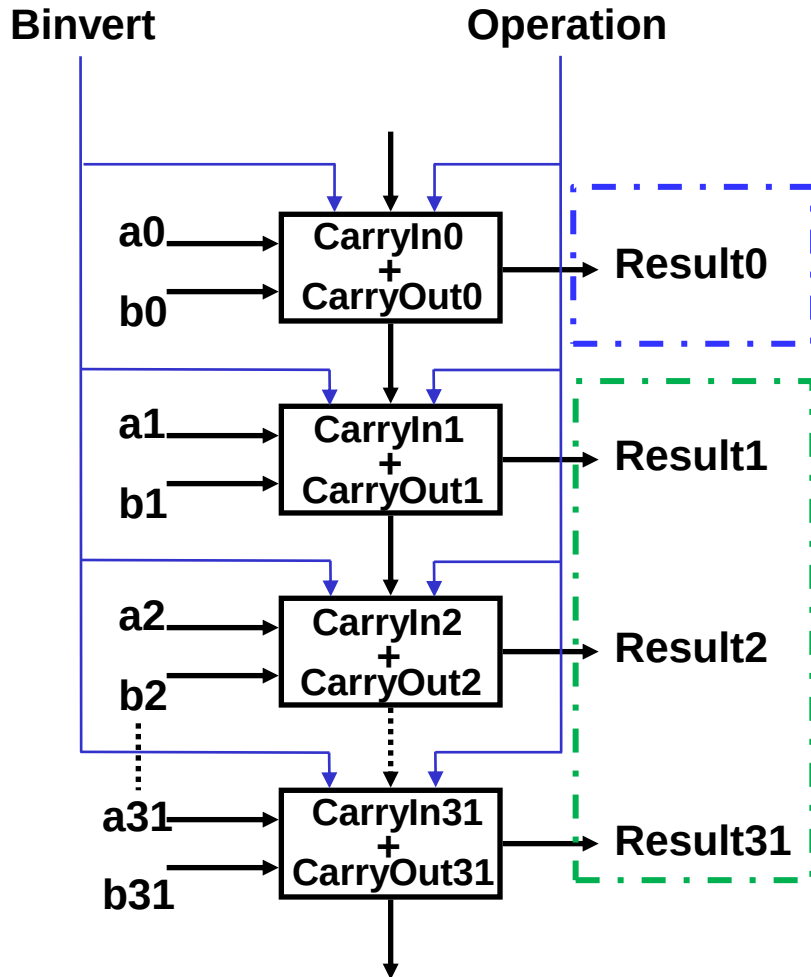
$\text{Slt: } rd = rs < rt ? 1 : 0;$

取 $a=rs$, $b=rt$,
若 $a-b < 0$, 则 $a-b$ 结果的符号位为1;
若 $a-b \geq 0$, 则符号位为 0

Result0置为 $a-b$ 的符号位
Result1-31置为0

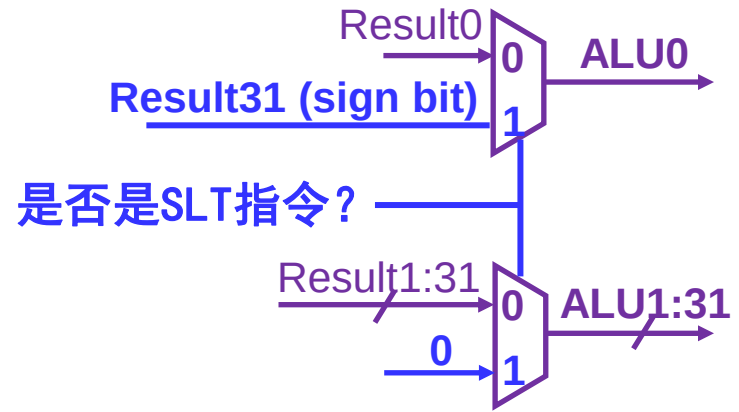
Q: $a-b$ 的符号位是哪一个输出?

32-bit ALU: slt

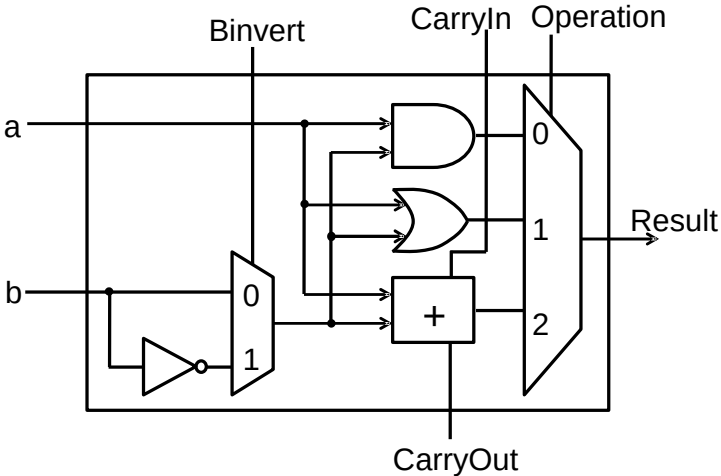


Result0置为的符号位
Result1-31置为0

后续添加第二级:

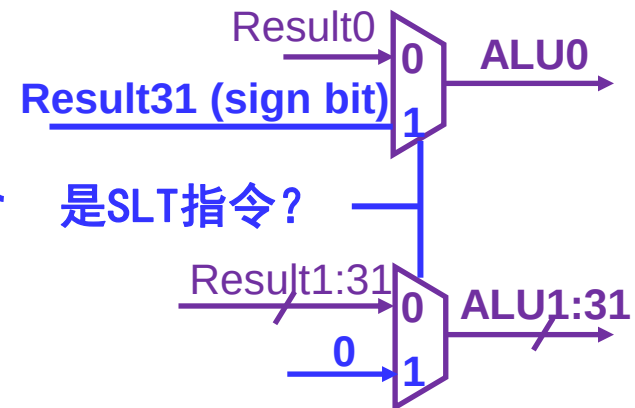
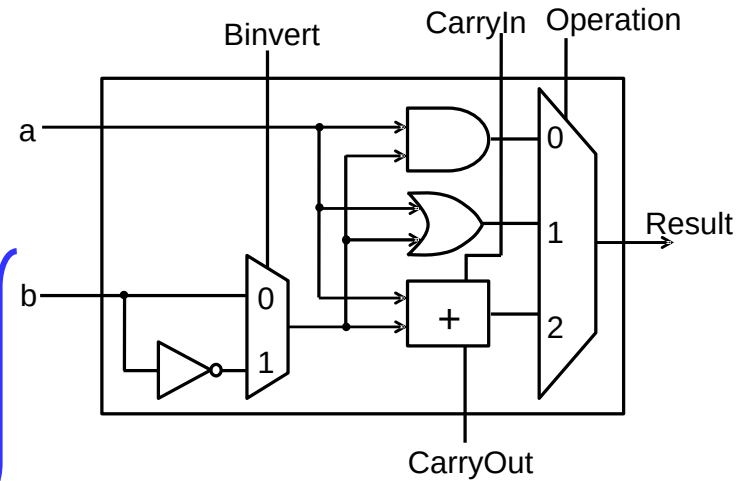
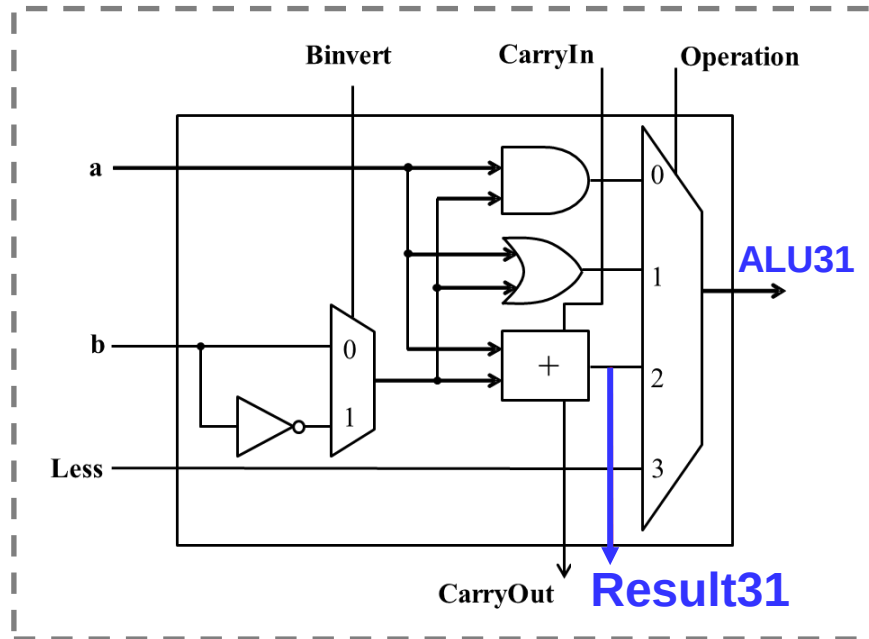


Response	Percentage
Yes	45%
No	55%

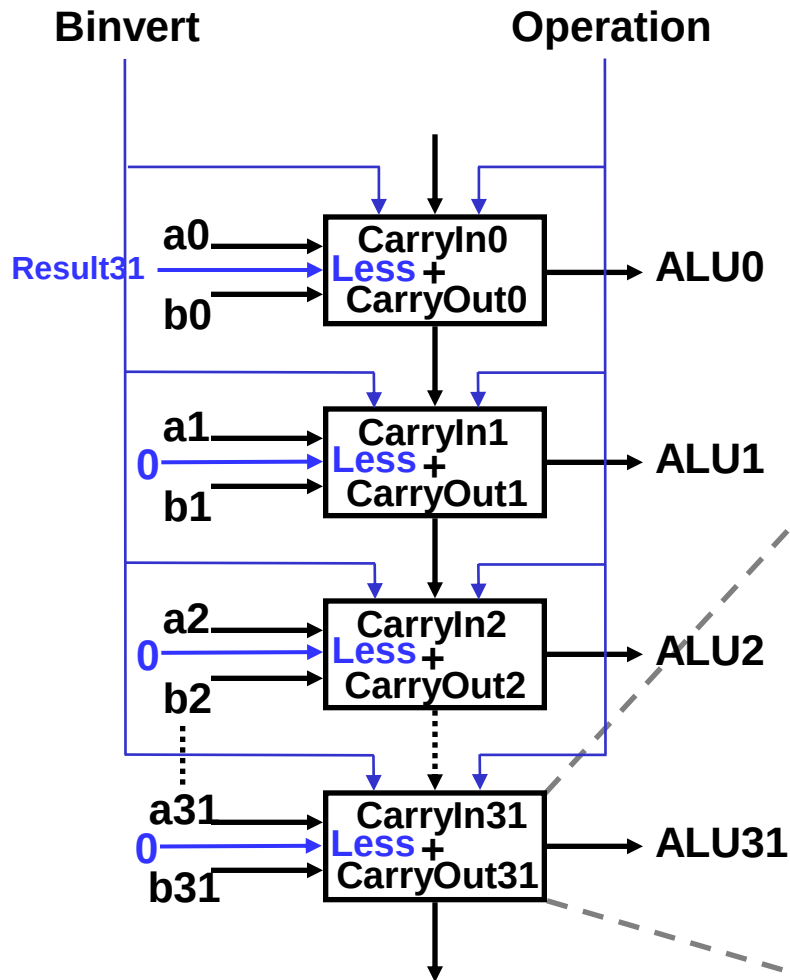


32-bit ALU: slt

前后两级MUX可合并：

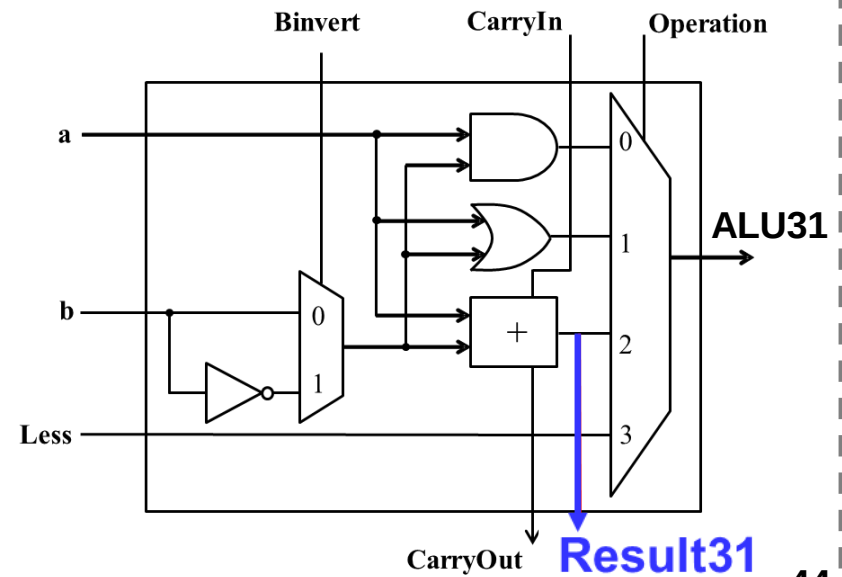


32-bit ALU: slt



After merging the MUXes:

MSB 1-bit ALU

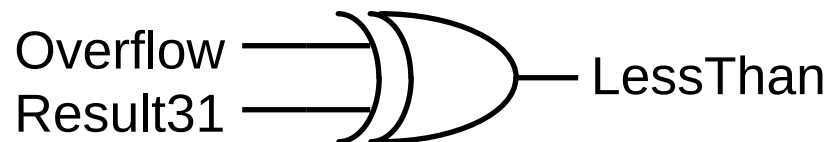


ALU:考虑溢出的Slt计算电路

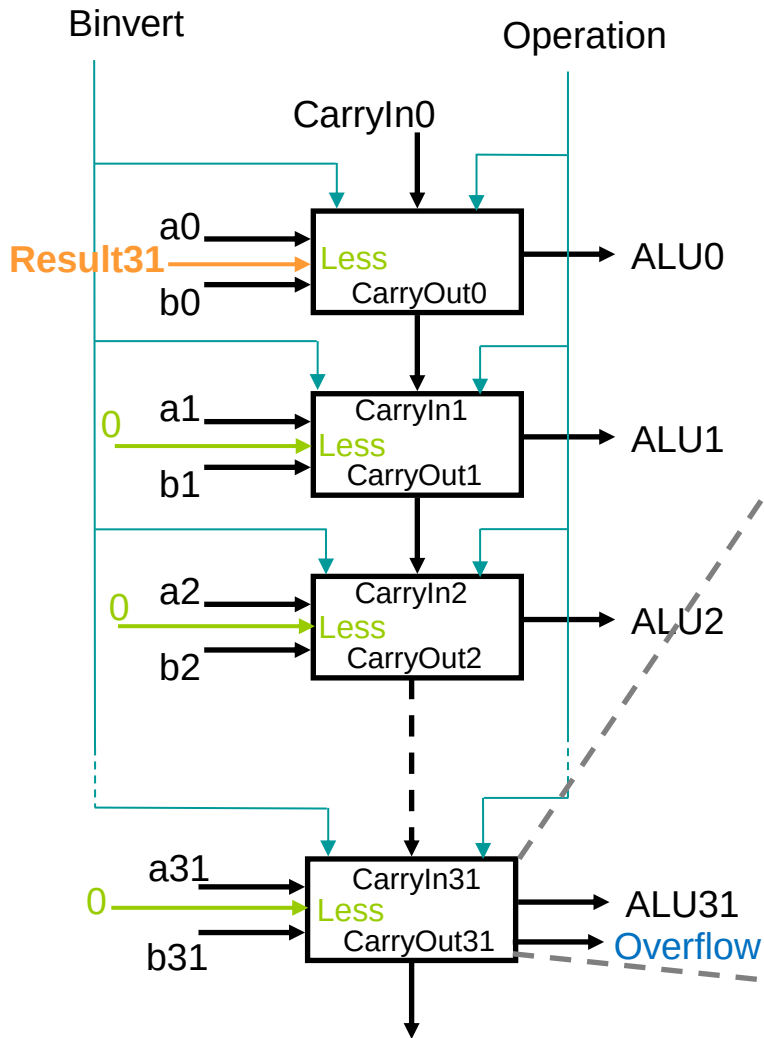
The impact of Overflow on Slt

Overflow	Result31	LessThan
0	0	0
0	1	1
1	0	1
1	1	0

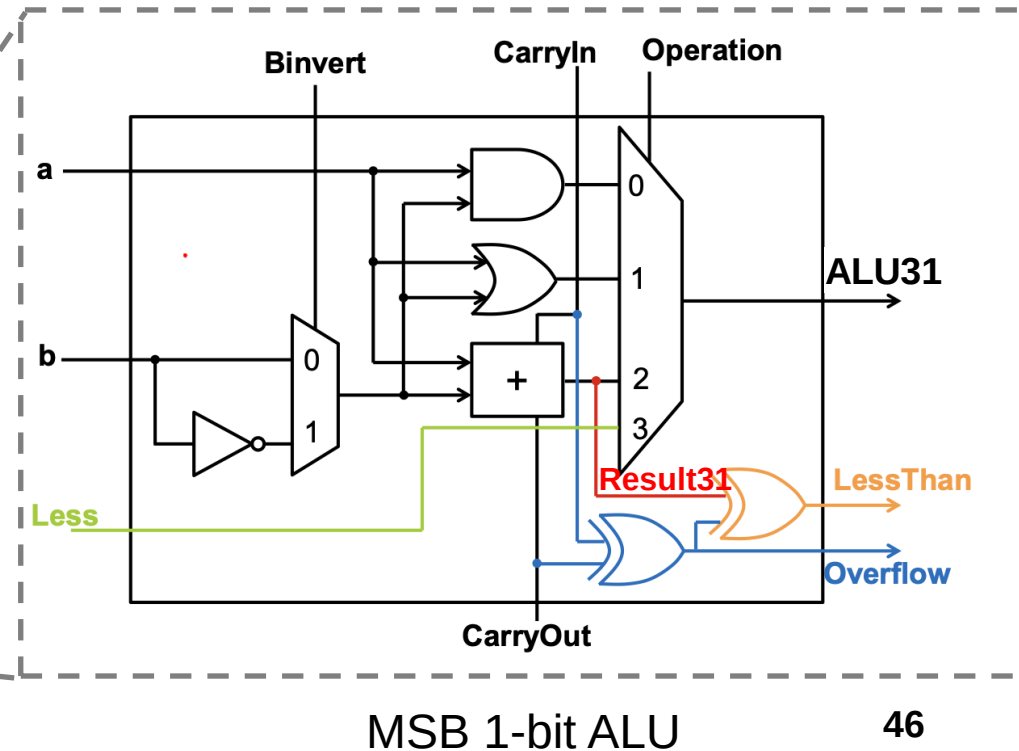
$$\text{LessThan} = \text{Overflow} \oplus \text{Result31}$$



ALU:考虑溢出的Slt计算电路



合并MUX、考虑溢出的Slt计算电路：



32位RISC ALU设计

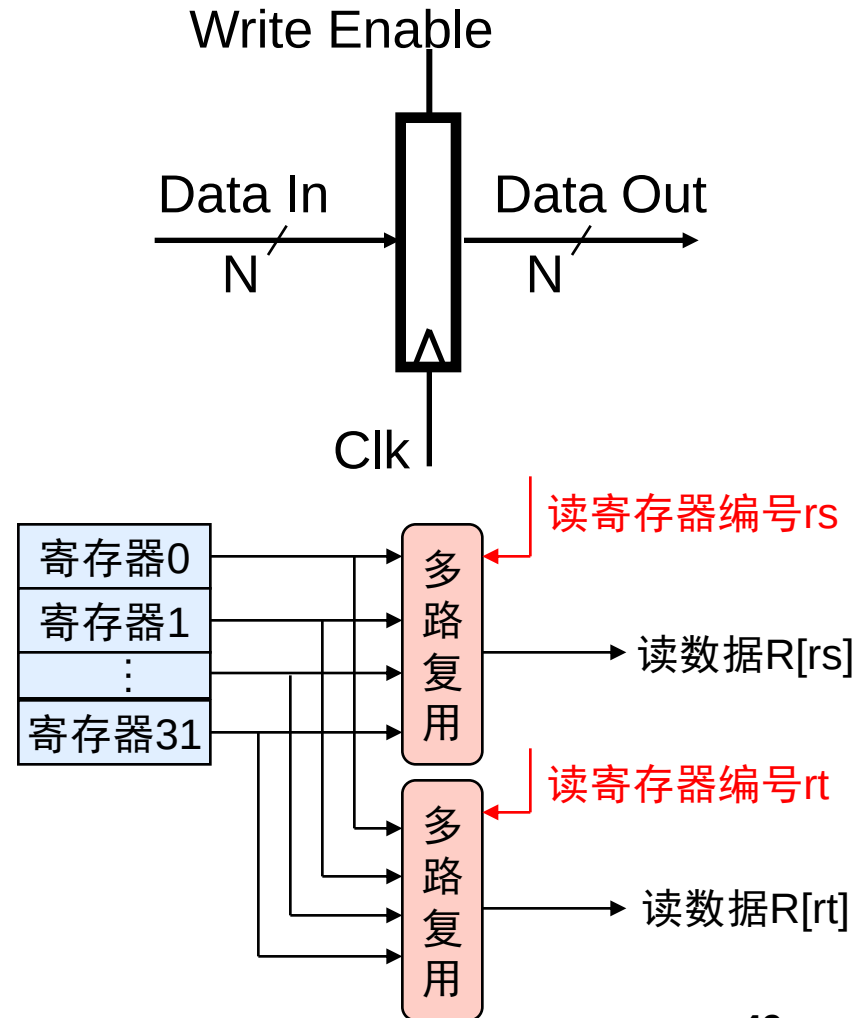
- **ALU还有什么？**
 - 移位运算...
- **更高效的ALU设计**
 - 超前进位加法器...

确定数据通路的组成模块

- 组合逻辑电路部分
 - “计算”为主，不同于单纯的“逻辑”
- 存储单元
 - “数据”为主，不同于单纯的“状态”

存储单元——寄存器

- 寄存器(register)
 - N个D触发器组成
- 32个寄存器构成寄存器堆
 - Register File, RF
 - 可以同时从两个口读取
 - 一个写端口，有写入使能
- 写入使能：
 - 禁止时(0): 寄存器内容不变
 - 使能时(1): 改变



存储单元——寄存器堆

- 寄存器堆包含 32 registers:

- 两个 32-bit 输出总线:
busA and busB
- 一个32-bit 输入总线: busW

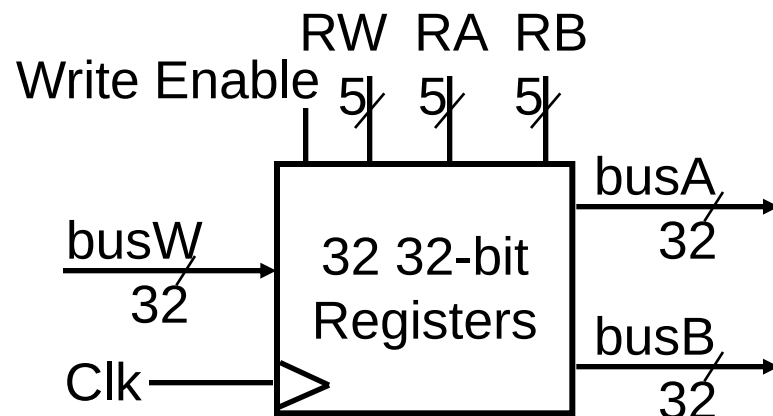
- 寄存器堆中的寄存器选择:

- RA (number) 选出哪个32位寄存器的数据放在busA (data)
- RB (number)选出哪个32位寄存器的数据放在busB (data)
- RW (number)选出哪个32位寄存器的数据将通过busW被写入 (当WE=1时)

- 时钟 (CLK)

- CLK 只在写入时发挥作用
- 读取电路总能读到值

- RA or RB valid => busA or busB valid after “access time.”



存储单元——存储器

- 存储器(memory): 容量大

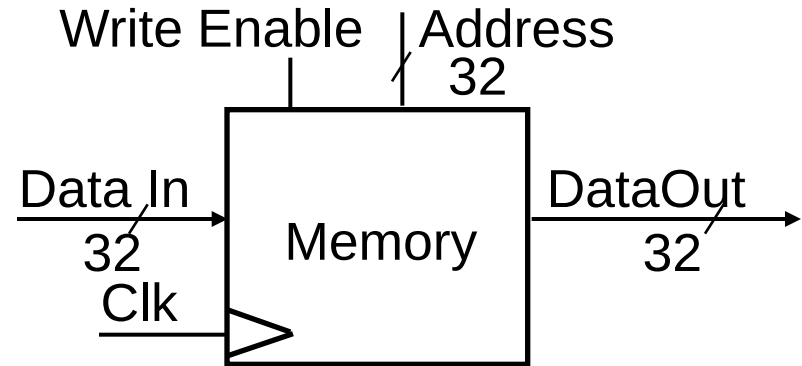
- 一个输入总线: 数据输入
- 一个输出总线: 数据输出

- 存储器选择:

- 通过地址线Address 选择哪个32位字被放在数据输出总线上
- Write Enable = 1: 通过address选择哪个地址的32位字写为输入总线的数据

- Clock input (CLK)

- CLK input 只在写入有效
- 在读出时, 类似于组合逻辑:
 - Address valid => Data Out valid after “access time.”

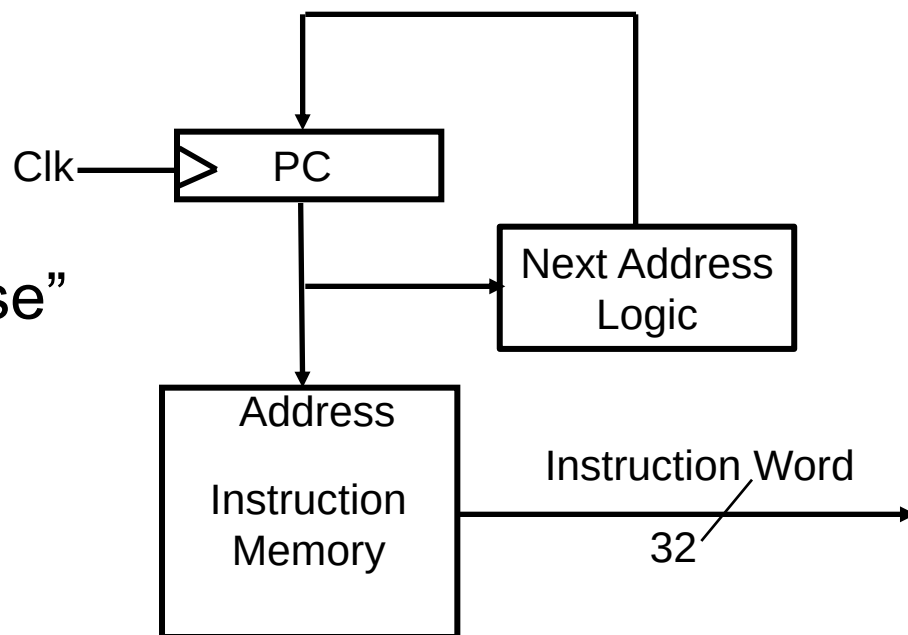


本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

取指令单元IF

- 所有指令都公用的单元
 - Fetch the Instruction: $\text{Inst_Mem}[\text{PC}]$
 - 更新PC: Next Address Logic
 - 顺序指令时:
 $\text{PC} \leq \text{PC} + 4$
 - 分支跳转时:
 $\text{PC} \leq \text{"something else"}$

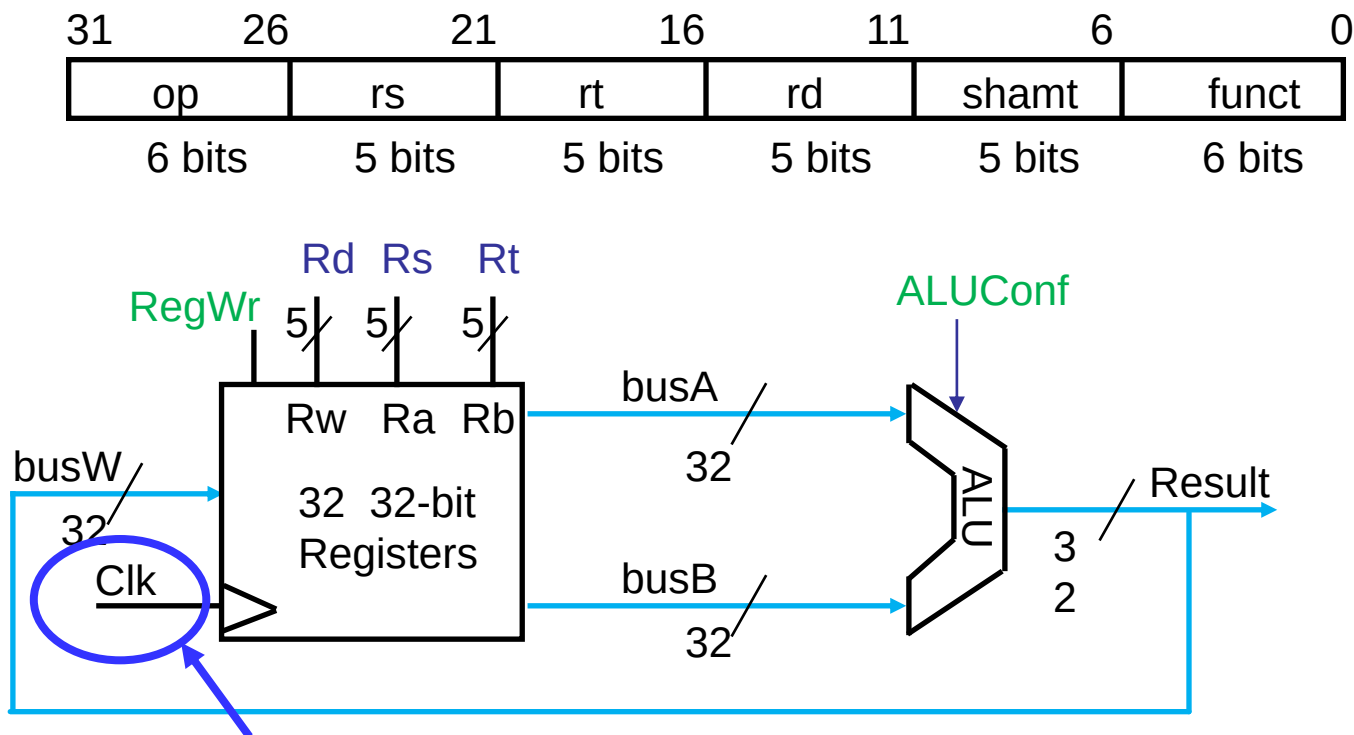


加减法等运算

- $R[rd] \leq R[rs] \text{ op } R[rt]$

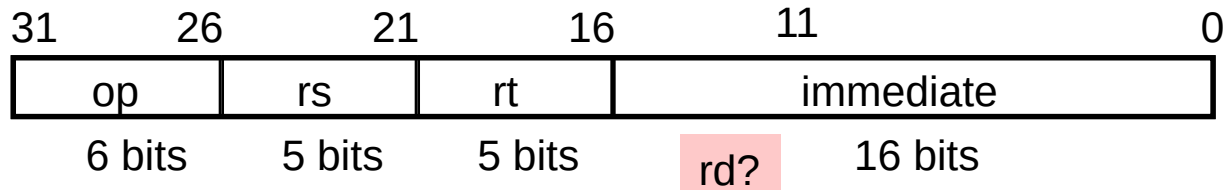
例子: `addu rd, rs, rt`

- Ra , Rb , and Rw 来自指令自中的 rs , rt , 和 rd
- $ALUConf$ and $RegWr$: 根据指令译码确定的控制信号

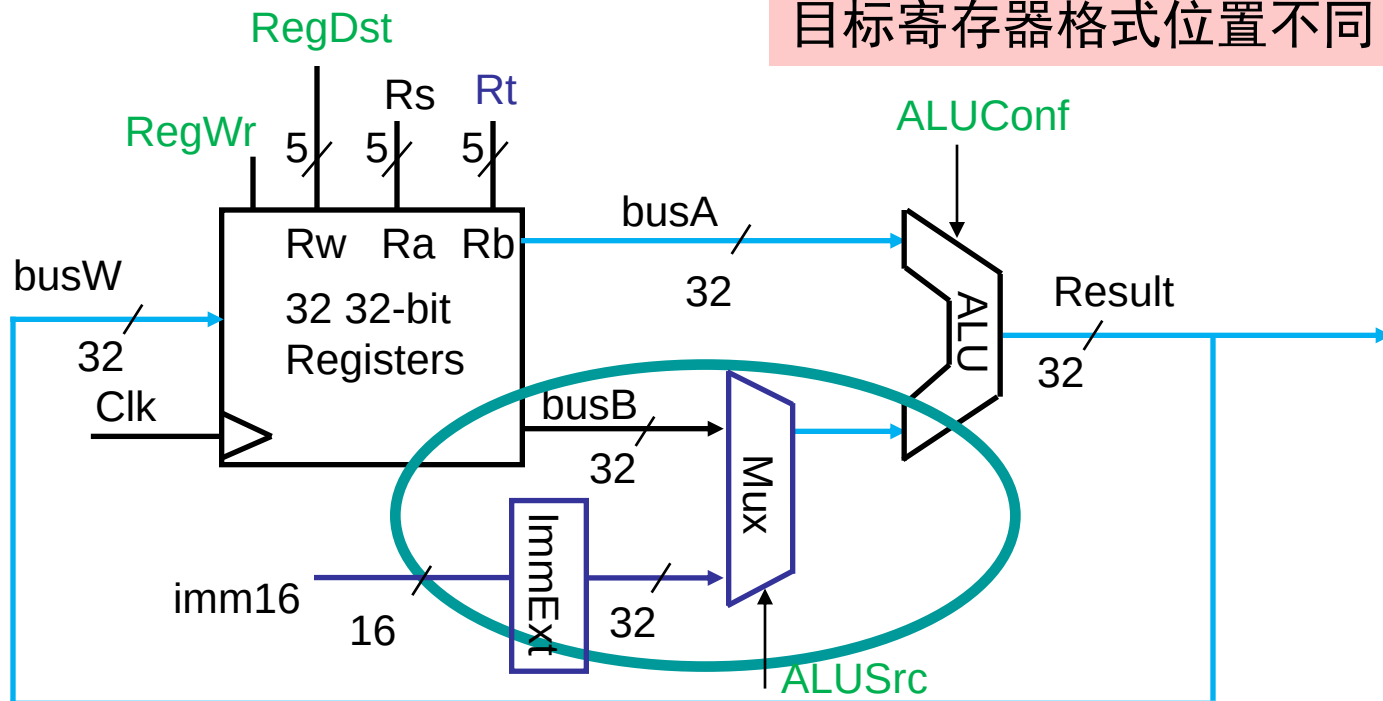


立即数操作

- $R[\underline{rt}] \leq R[rs] \text{ op Ext}[imm16]$

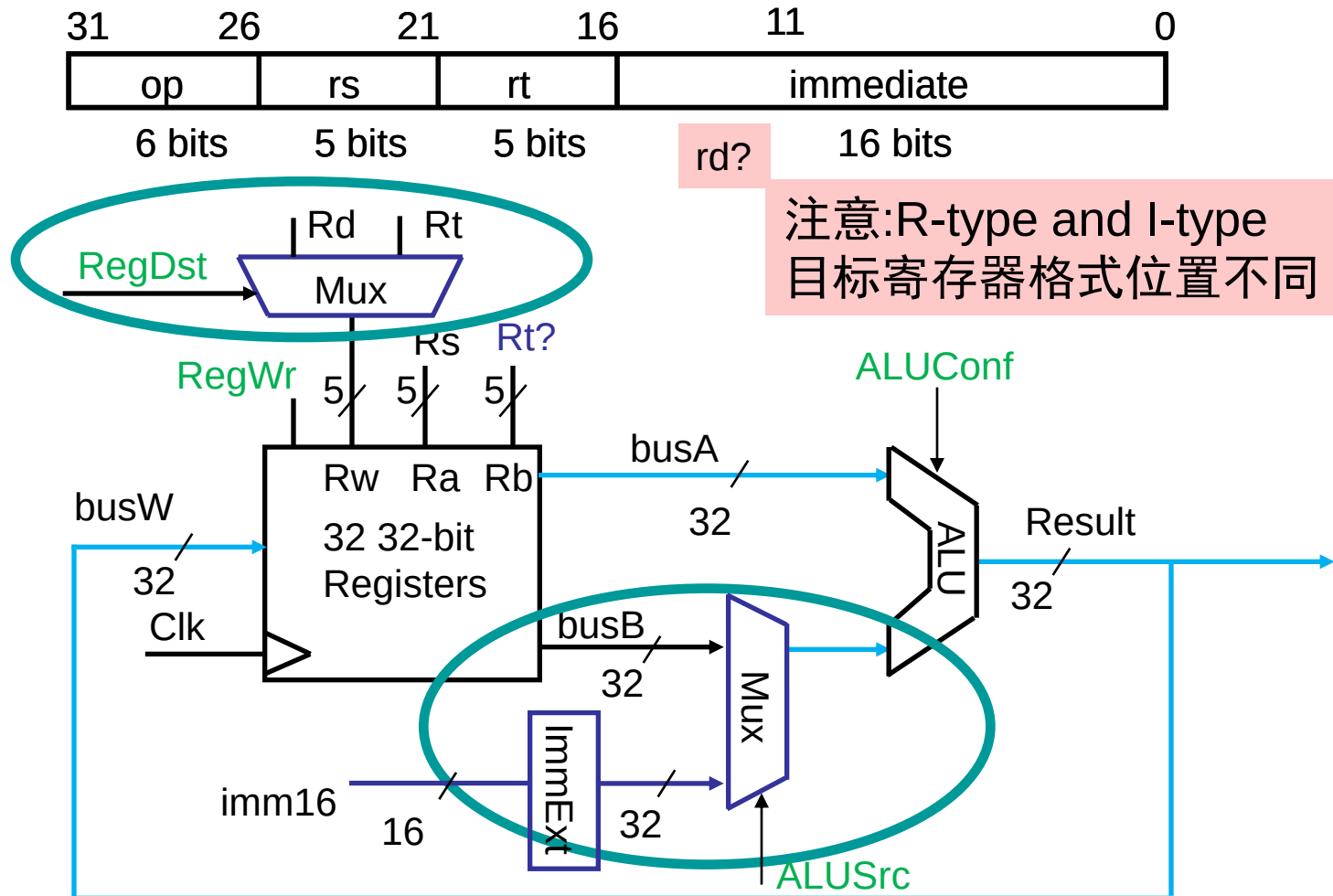


注意: R-type and I-type
目标寄存器格式位置不同



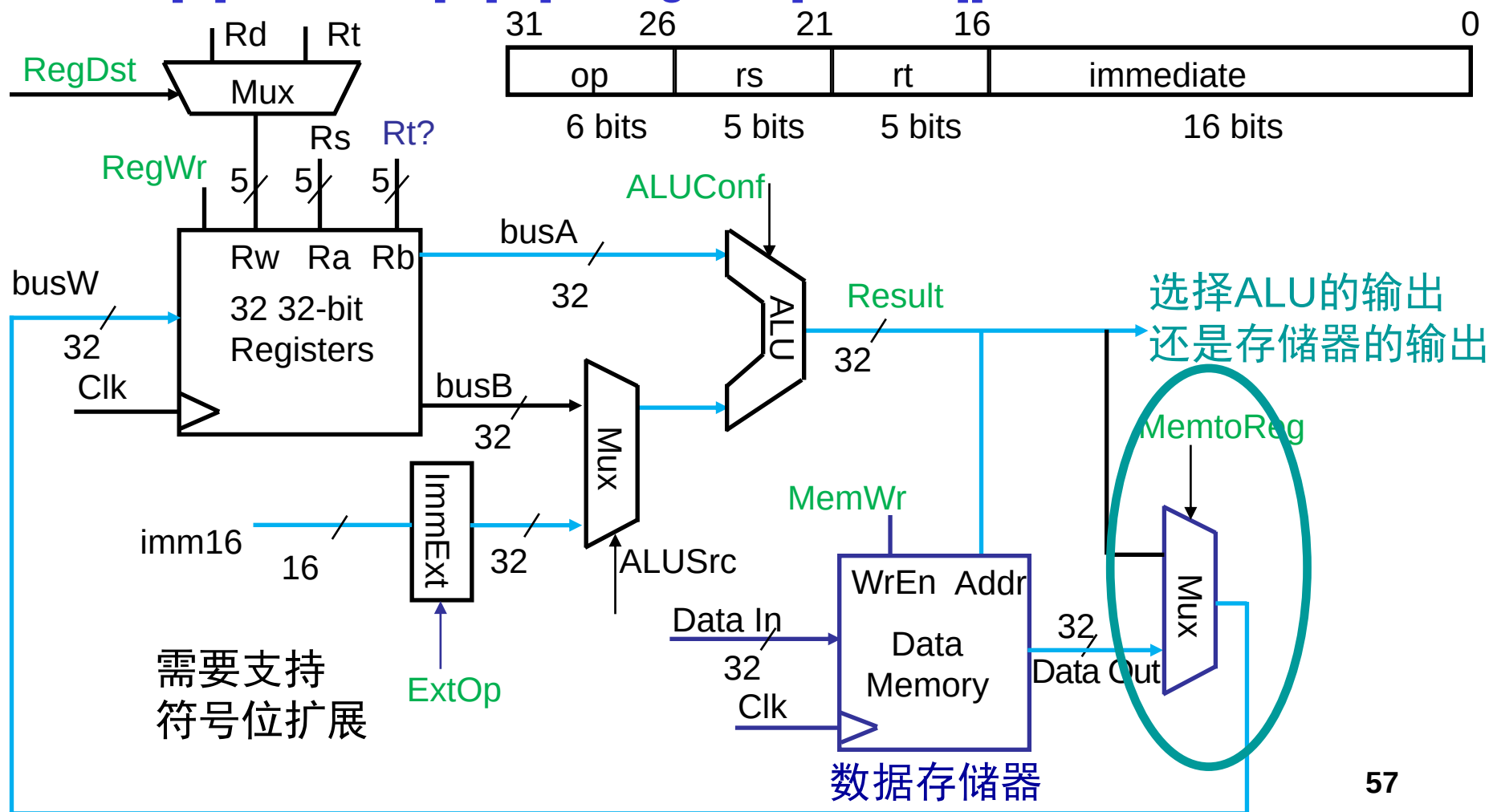
立即数操作

- $R[\underline{rt}] \leq R[rs] \text{ op Ext}[imm16]$



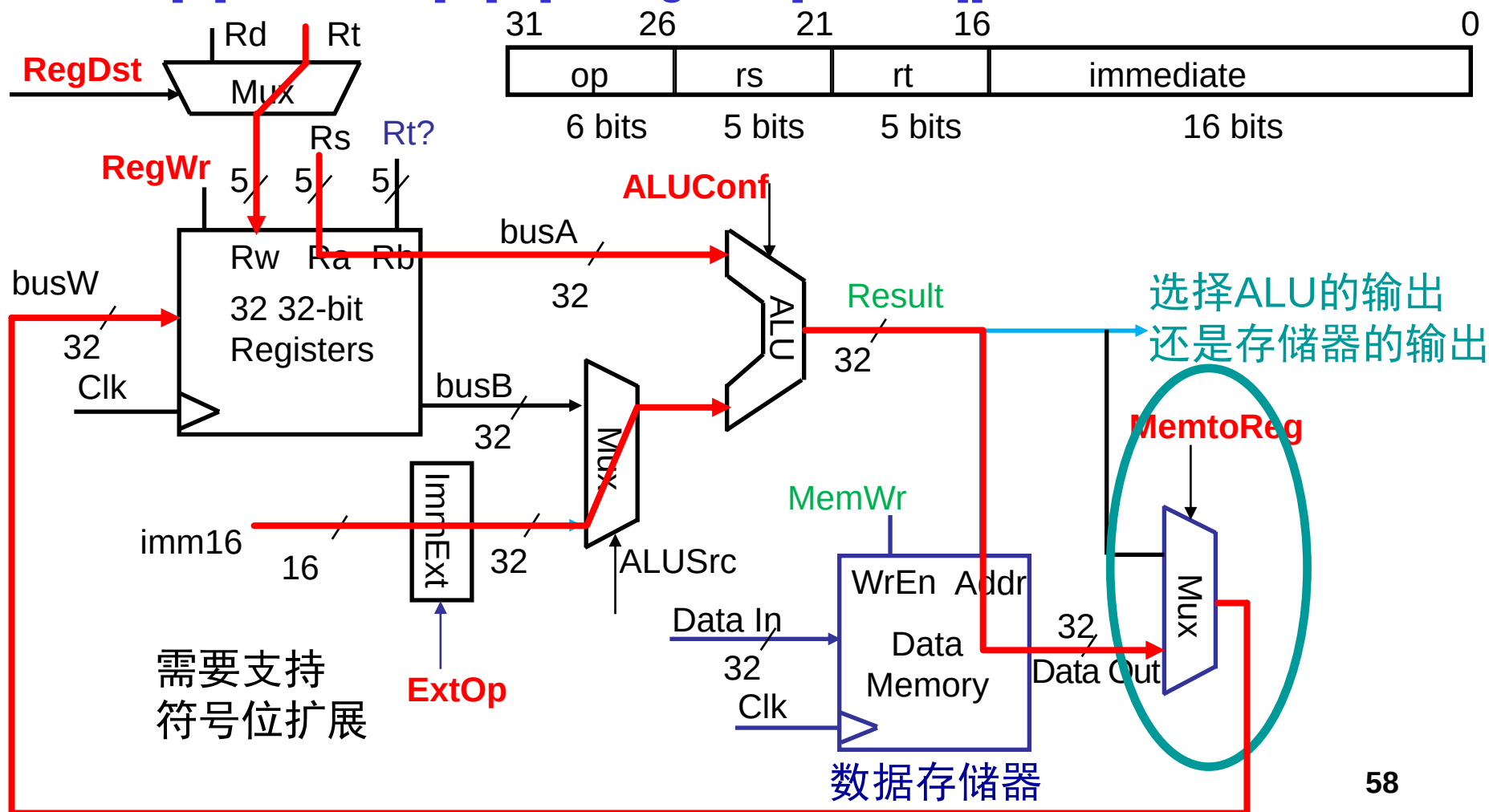
Load Word 操作

- $R[rt] \leq Mem[R[rs] + SignExt[imm16]]$



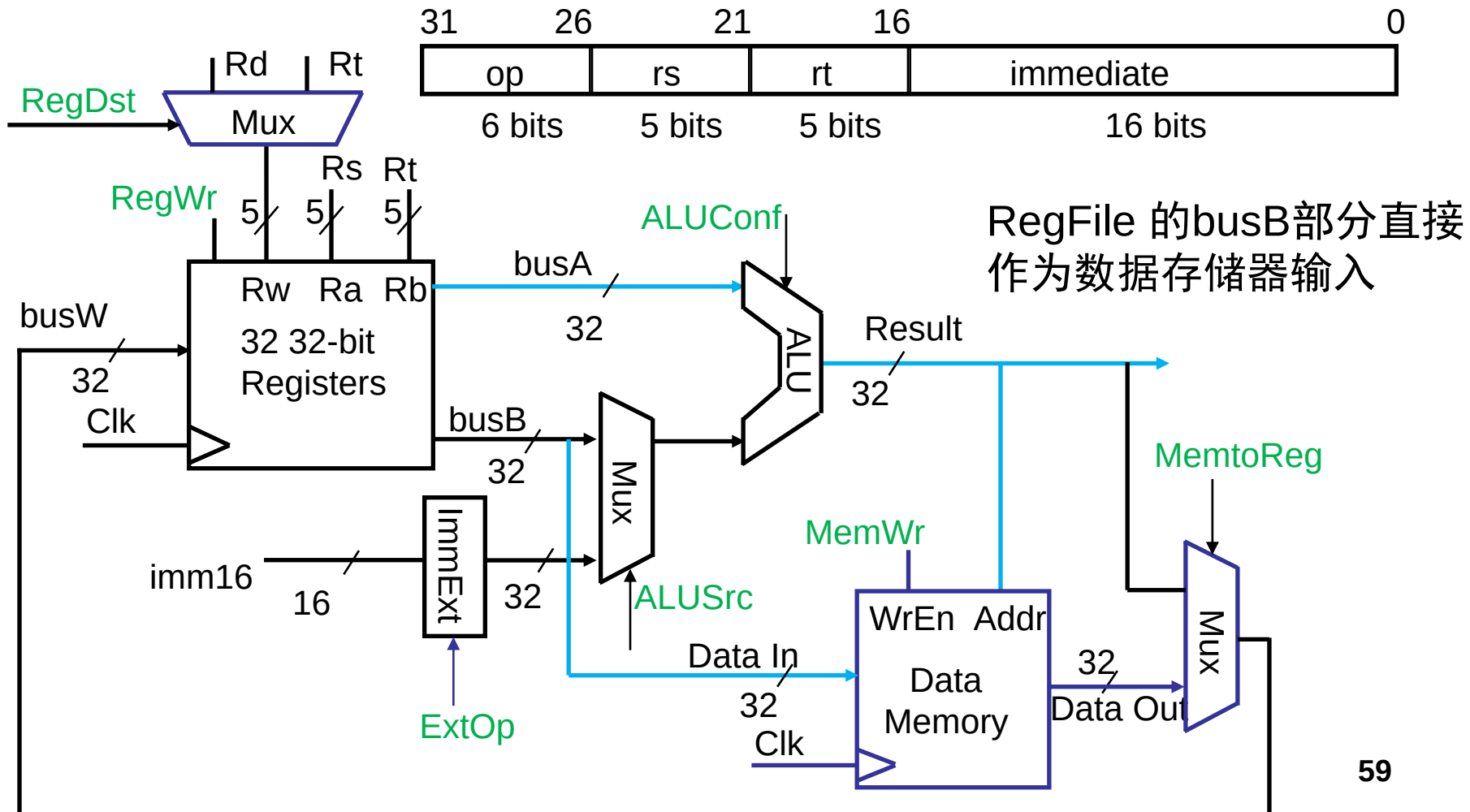
Load Word 操作

- $R[rt] \leq Mem[R[rs] + SignExt[imm16]]$



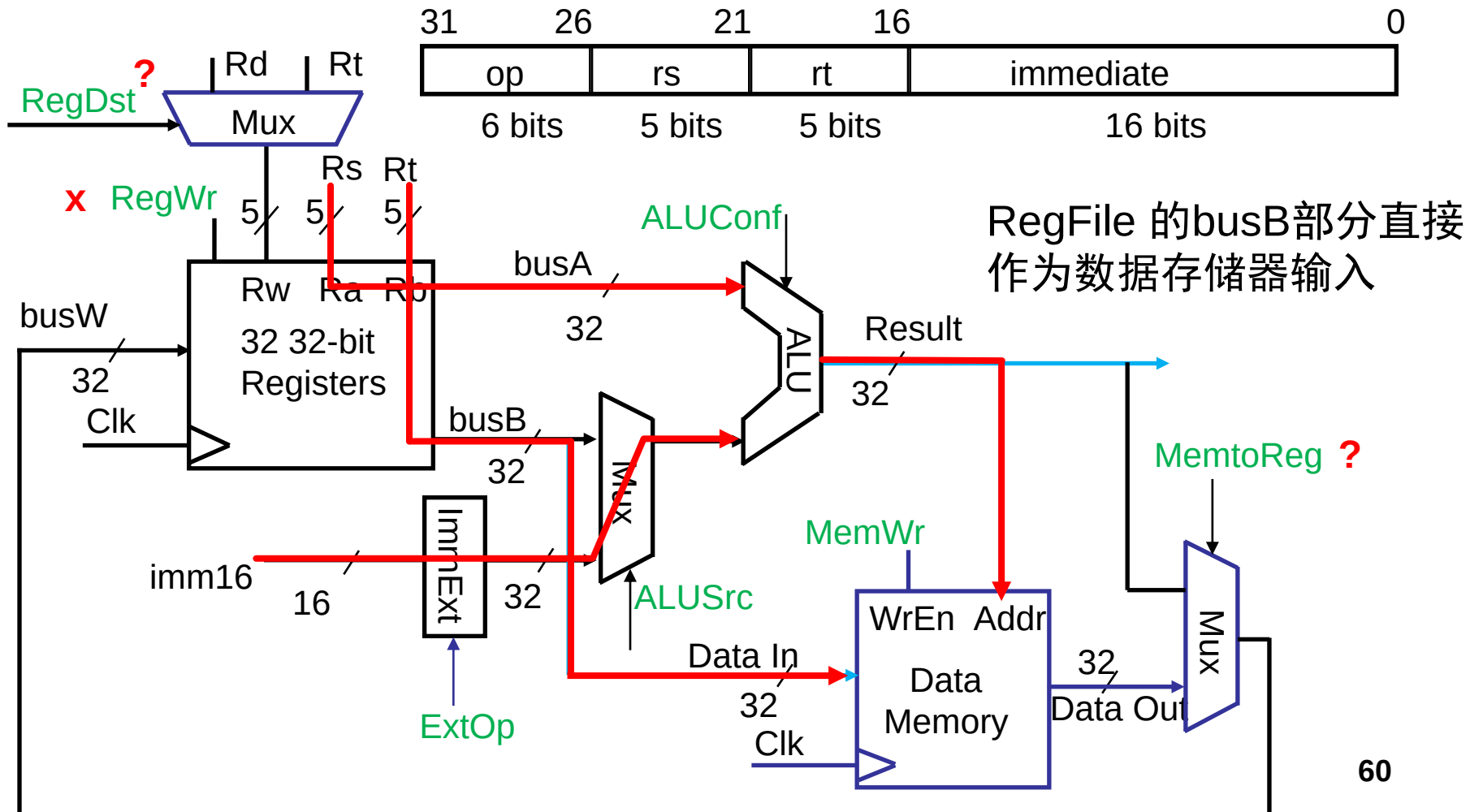
Store Word 操作

- $\text{Mem}[\text{R}[\text{rs}] + \text{SignExt}[\text{imm16}]] \leftarrow \text{R}[\text{rt}]$

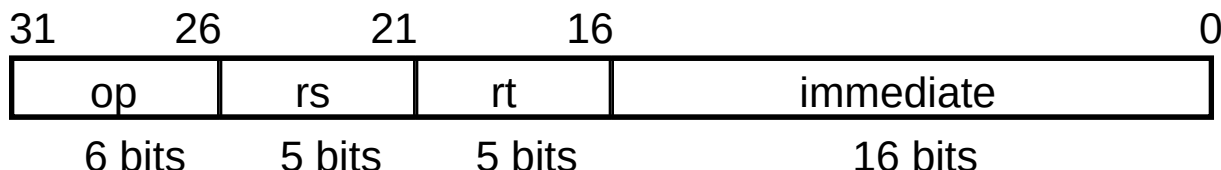


Store Word 操作

- **Mem[R[rs] + SignExt[imm16]] <= R[rt]**



分支指令



- beq rs, rt, imm16

–mem[PC]

从存储器取指令

–Equal $\Leftarrow (R[rs] == R[rt])$

计算分支条件

–if (Equal)

计算下一个指令的地址

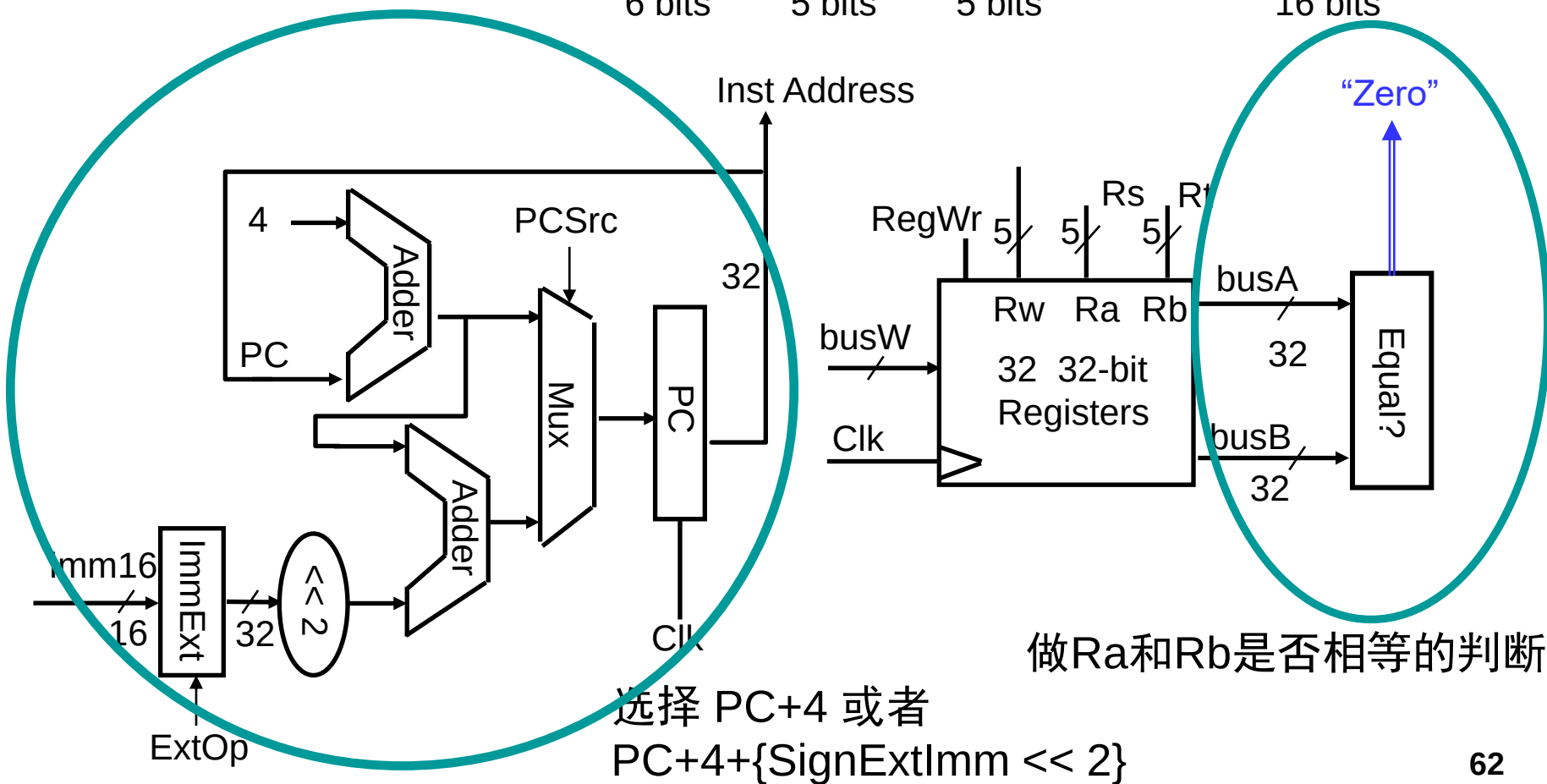
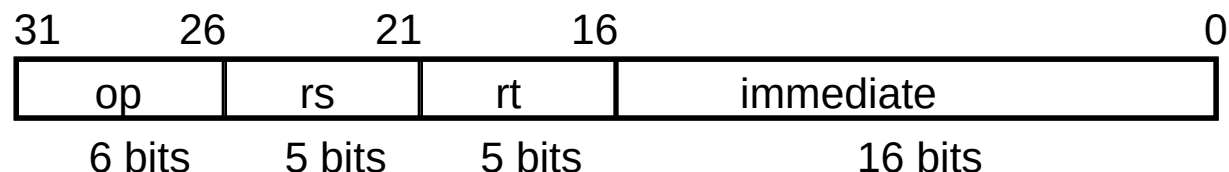
- $PC \Leftarrow PC + 4 + \{ \text{SignExt}(\text{imm16}), 2b00 \}$

– else

- $PC \Leftarrow PC + 4$

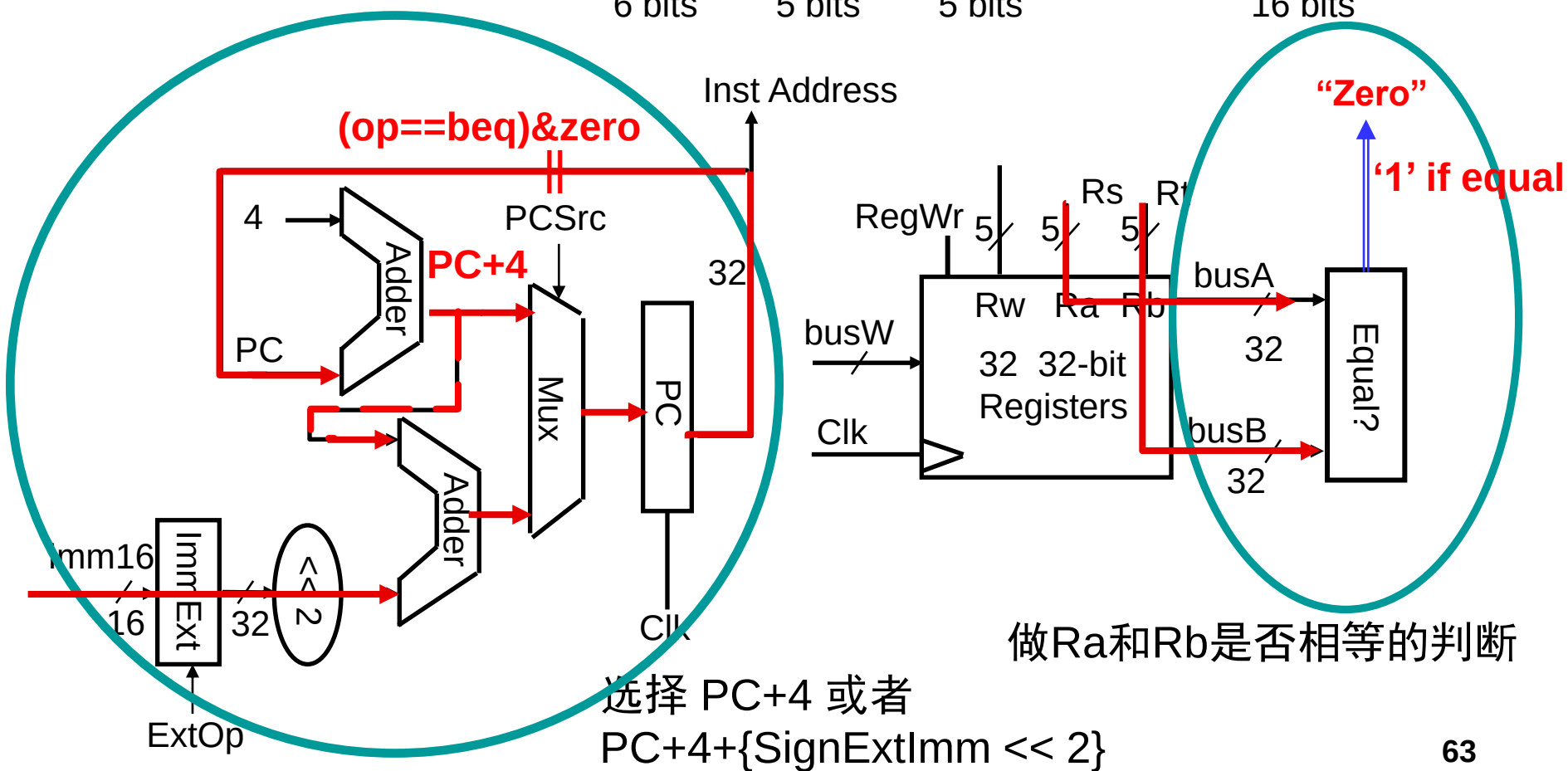
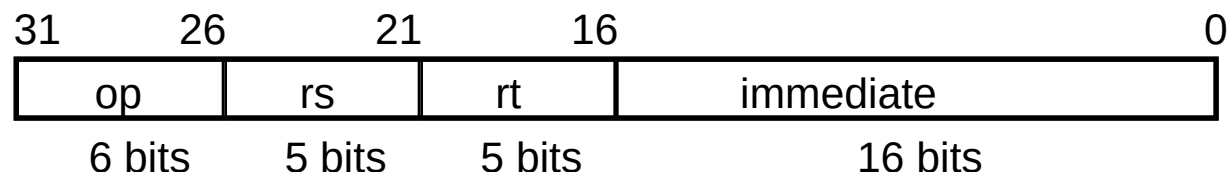
分支指令——取指和条件选择

beq rs, rt, imm16

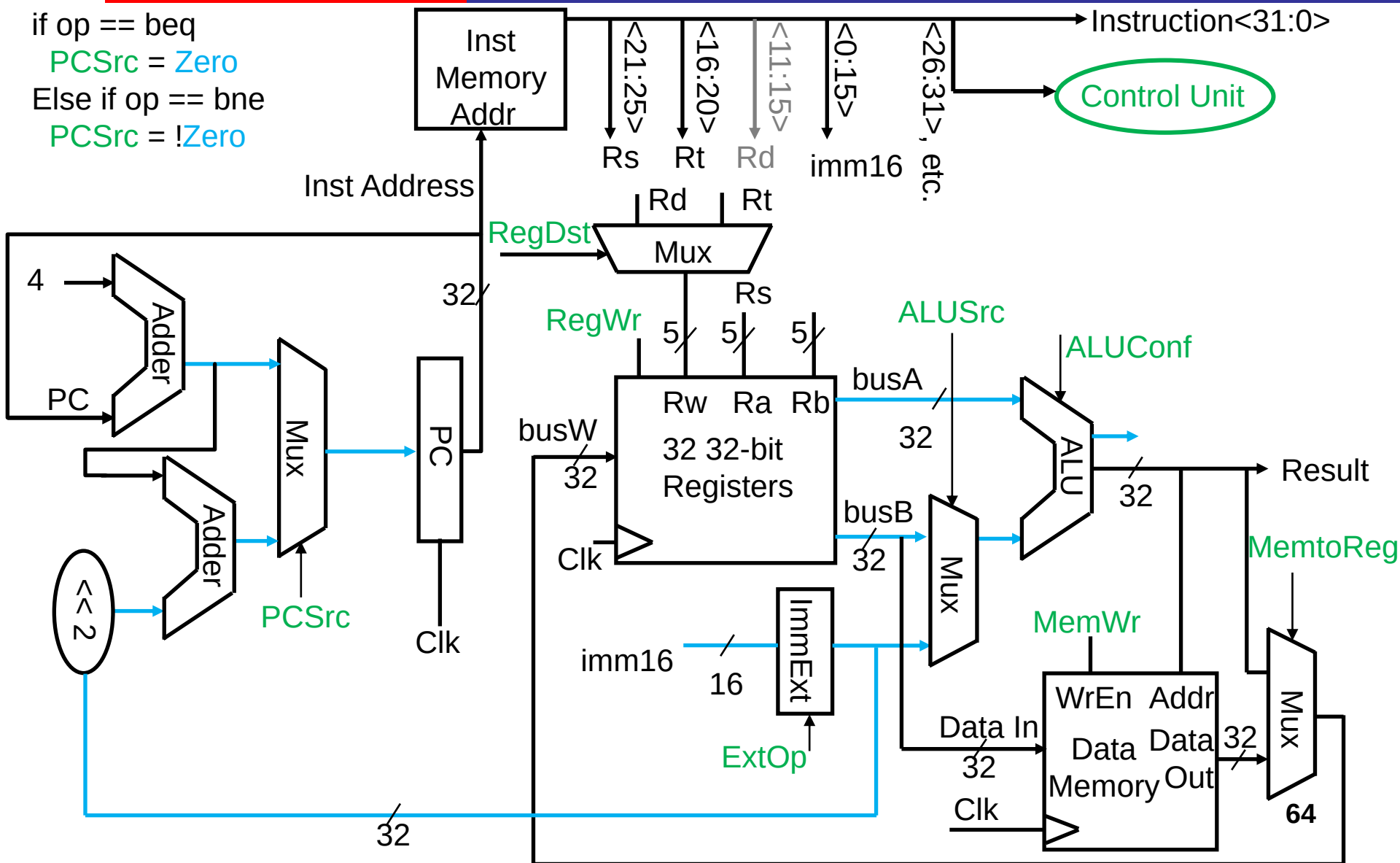


分支指令——取指和条件选择

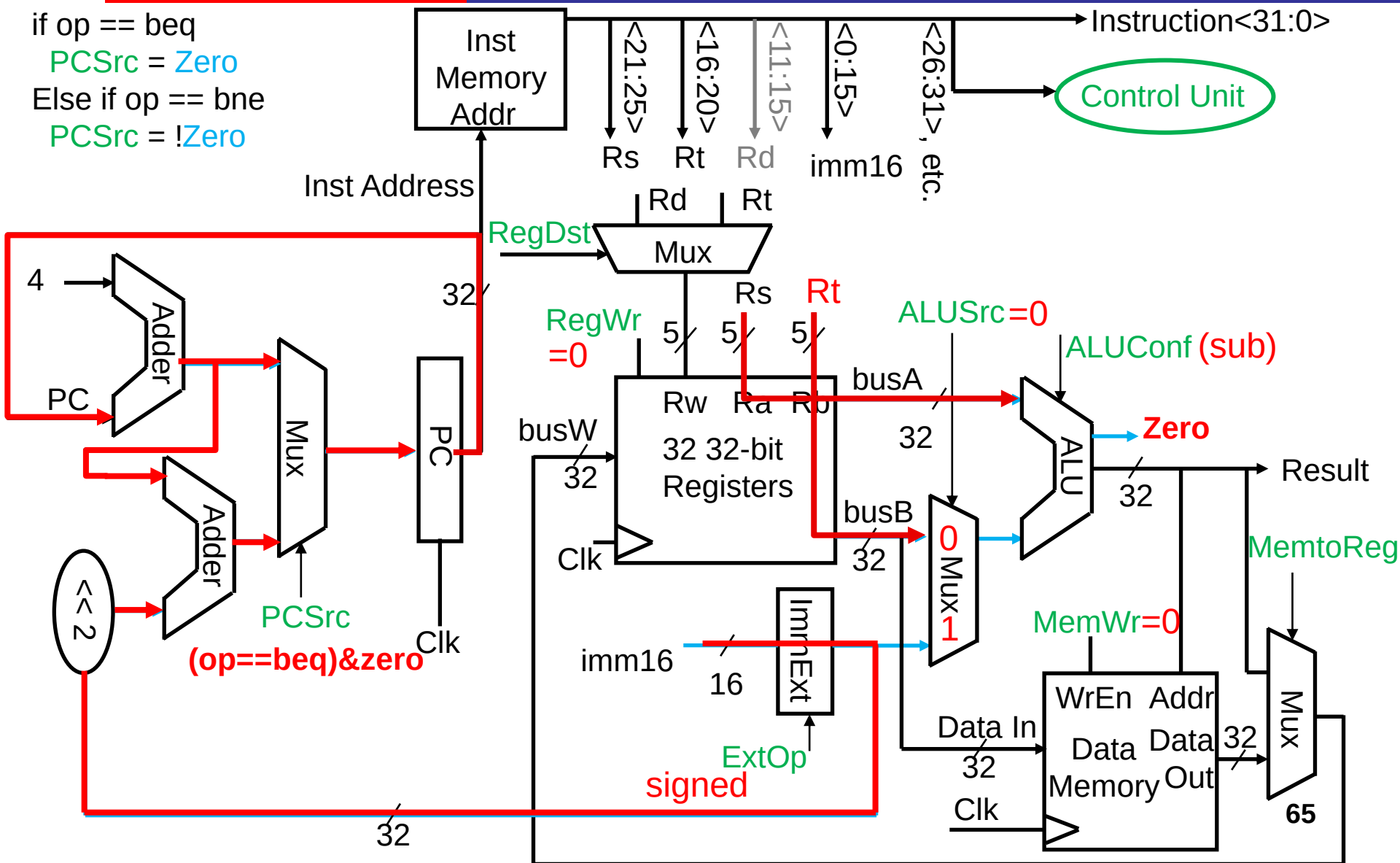
beq rs, rt, imm16



分支指令——完整datapath

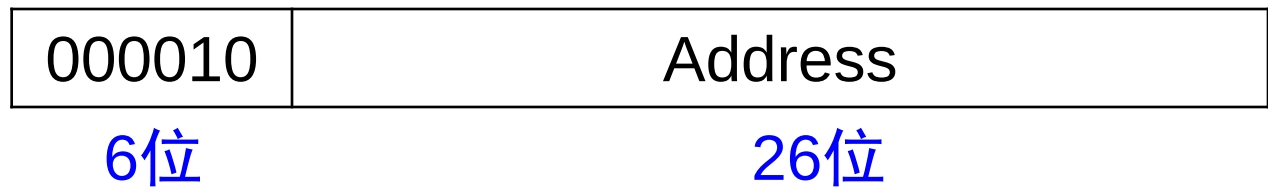


分支指令——完整datapath



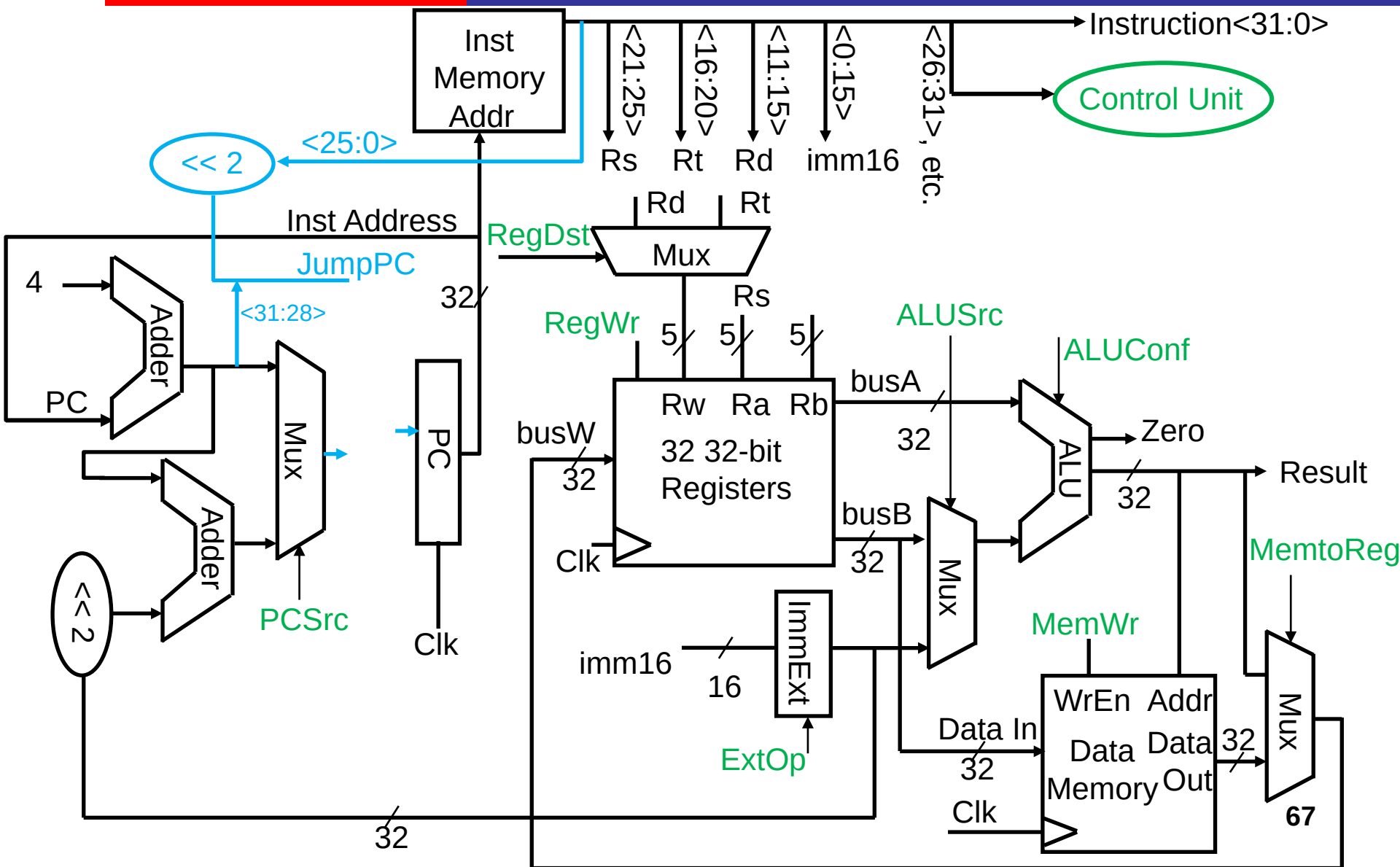
跳转指令

- j Label

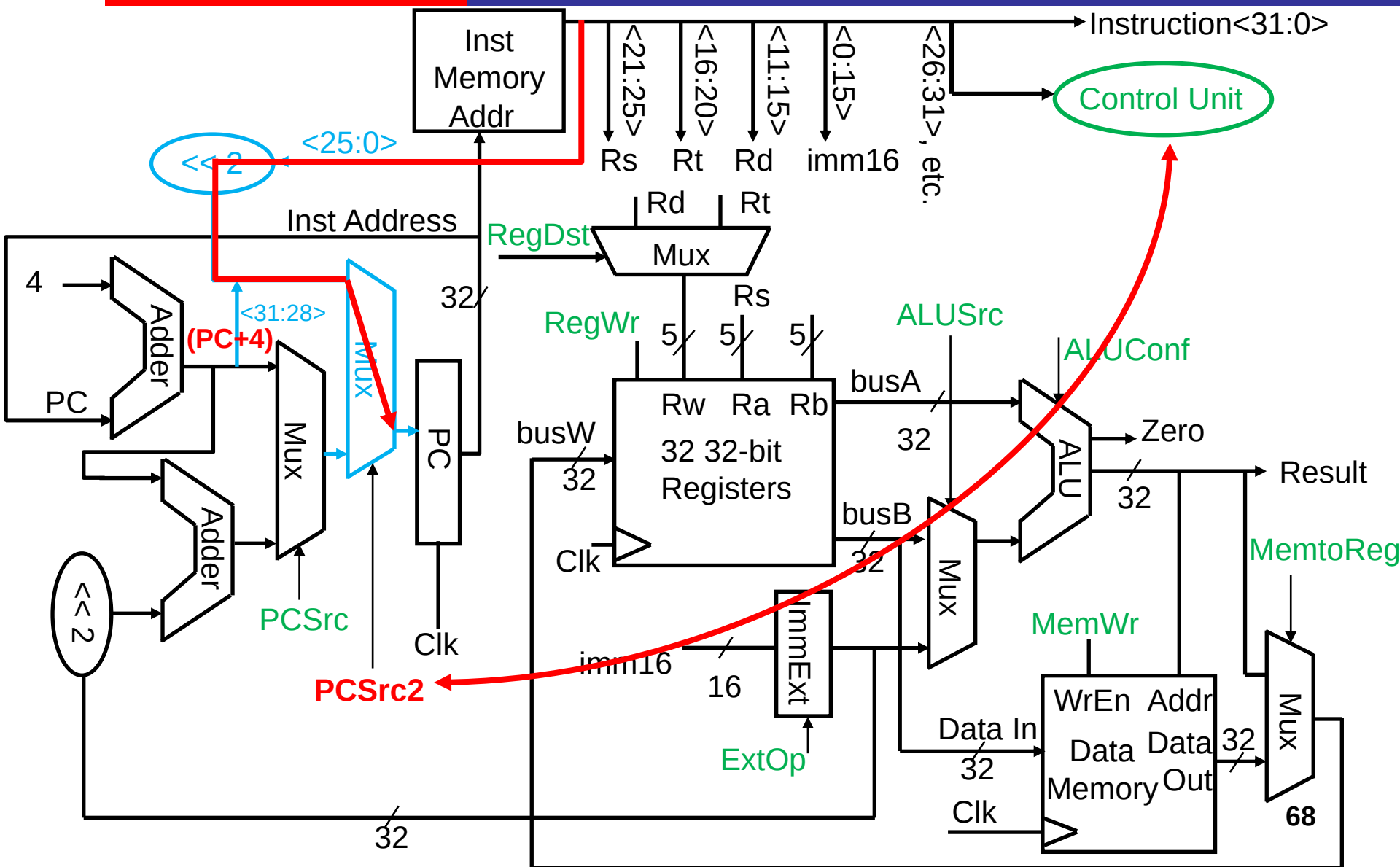


跳转 PC = { (PC+4)[31:28], 目标地址, 00 }

跳转指令



跳转指令



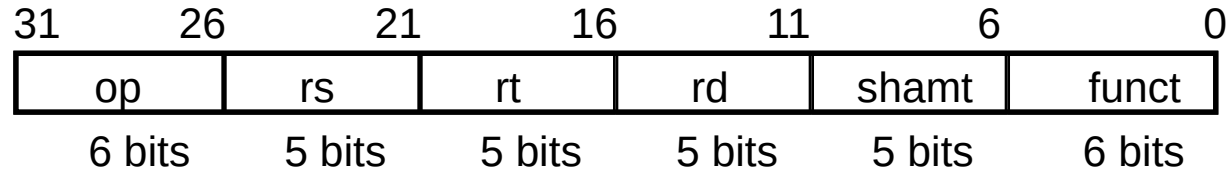
本节课目录

- 复习与回顾
- 单周期数据通路设计流程
 - 指令集和设计需求分析
 - 功能模块设计
 - 数据通路模块组装
 - 控制信号分析与逻辑设计
- 单周期数据通路延时分析
- 多周期数据通路
- 异常与中断

进一步精简的MIPS子集

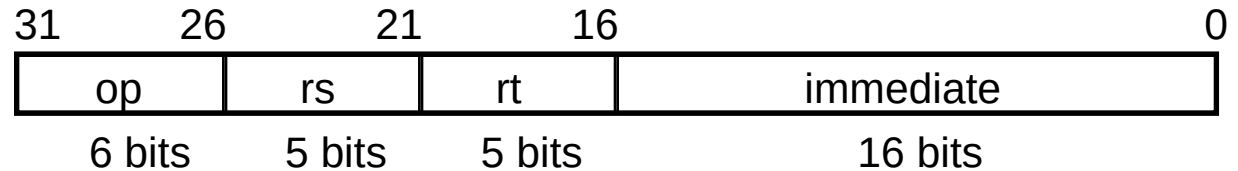
- ADD, SUB, AND, SLT

- add rd, rs, rt
- sub rd, rs, rt
- and rd, rs, rt
- slt rd, rs, rt



- OR Immediate:

- ori rt, rs, imm16

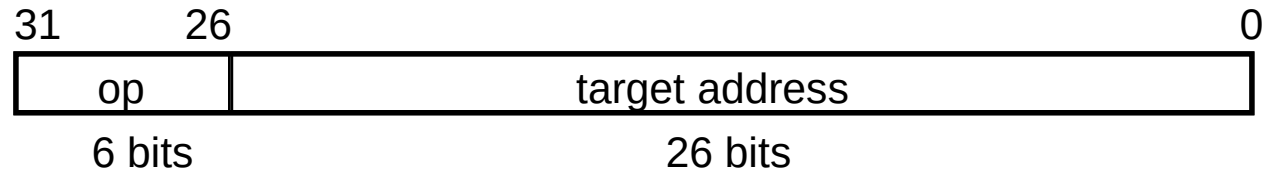


- LOAD and STORE Word

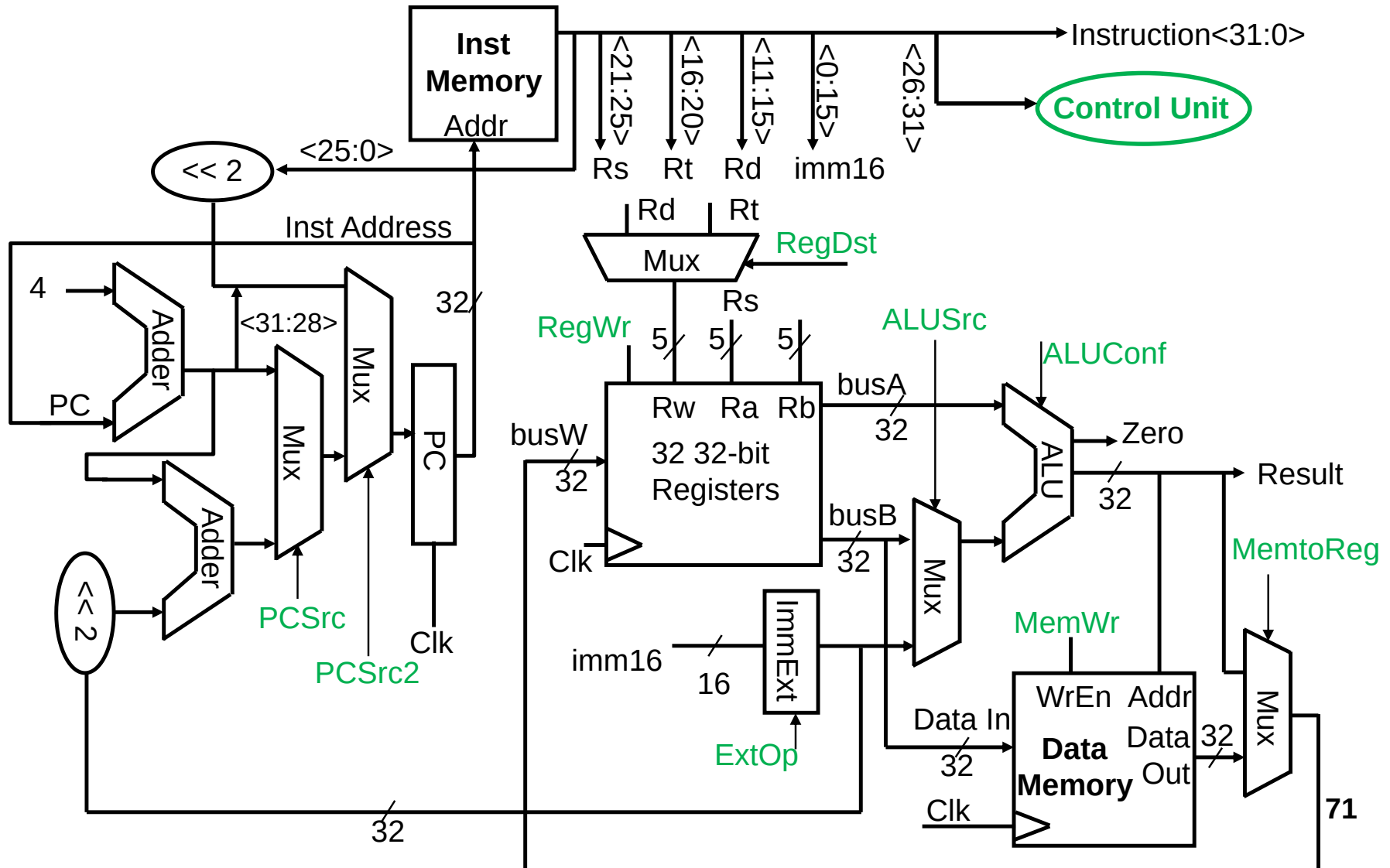
- lw rt, imm16(rs)
- sw rt, imm16(rs)

- BRANCH and JUMP:

- beq rs, rt, imm16
- j Label

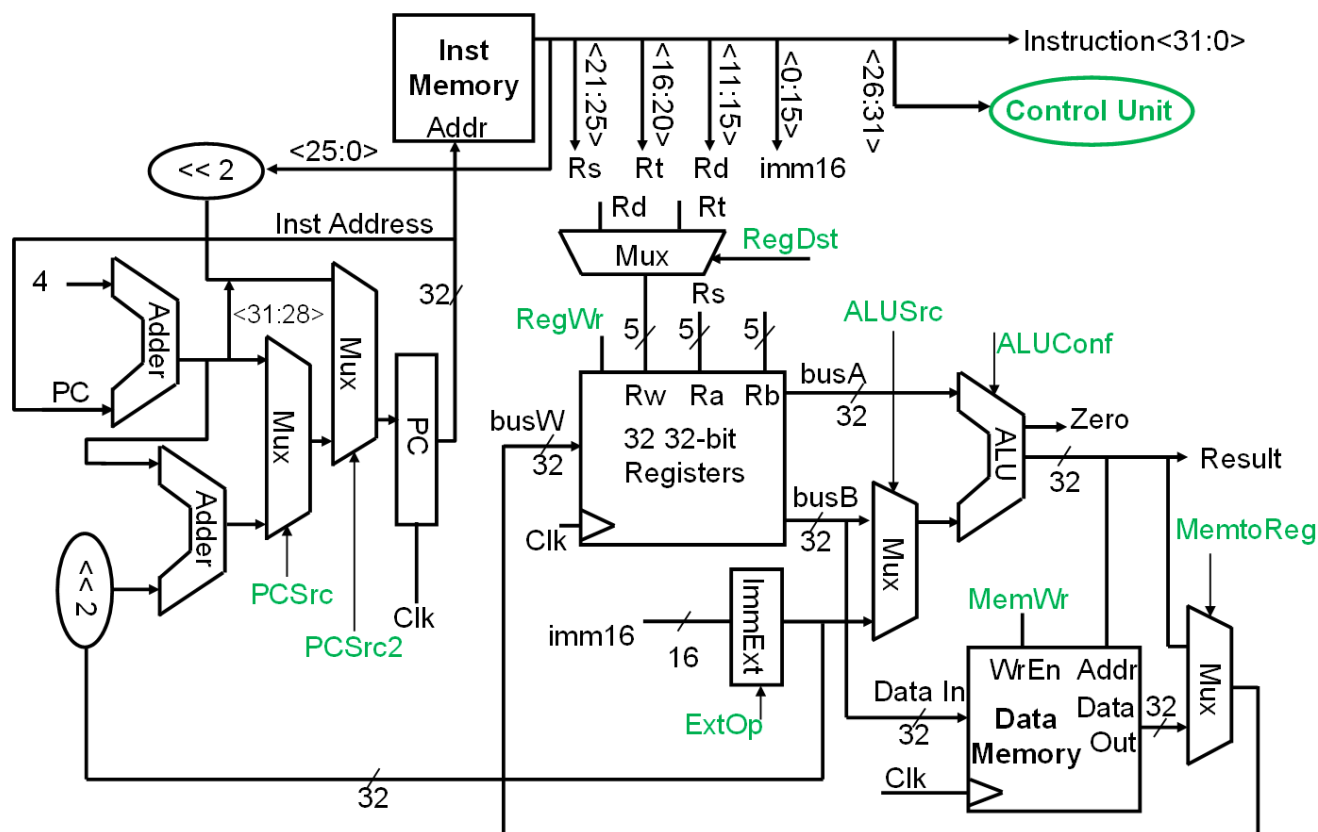


精简MIPS子集的数据通路设计



控制信号分析

- PCSrc: 0-顺序, 1-分支跳转
- ExtOp: “zero”, “sign”
- ALUSrc: 0 $\Rightarrow$ regB; 1 $\Rightarrow$ imm32
- ALUconf: “add”, “sub”, “or”
- MemWr: 1 $\Rightarrow$ write memory
- MemtoReg: 0 $\Rightarrow$ ALU; 1 $\Rightarrow$ Mem
- RegDst: 0 $\Rightarrow$ “Rt”; 1 $\Rightarrow$ “Rd”
- RegWr: 1 $\Rightarrow$ write registers



控制信号分析

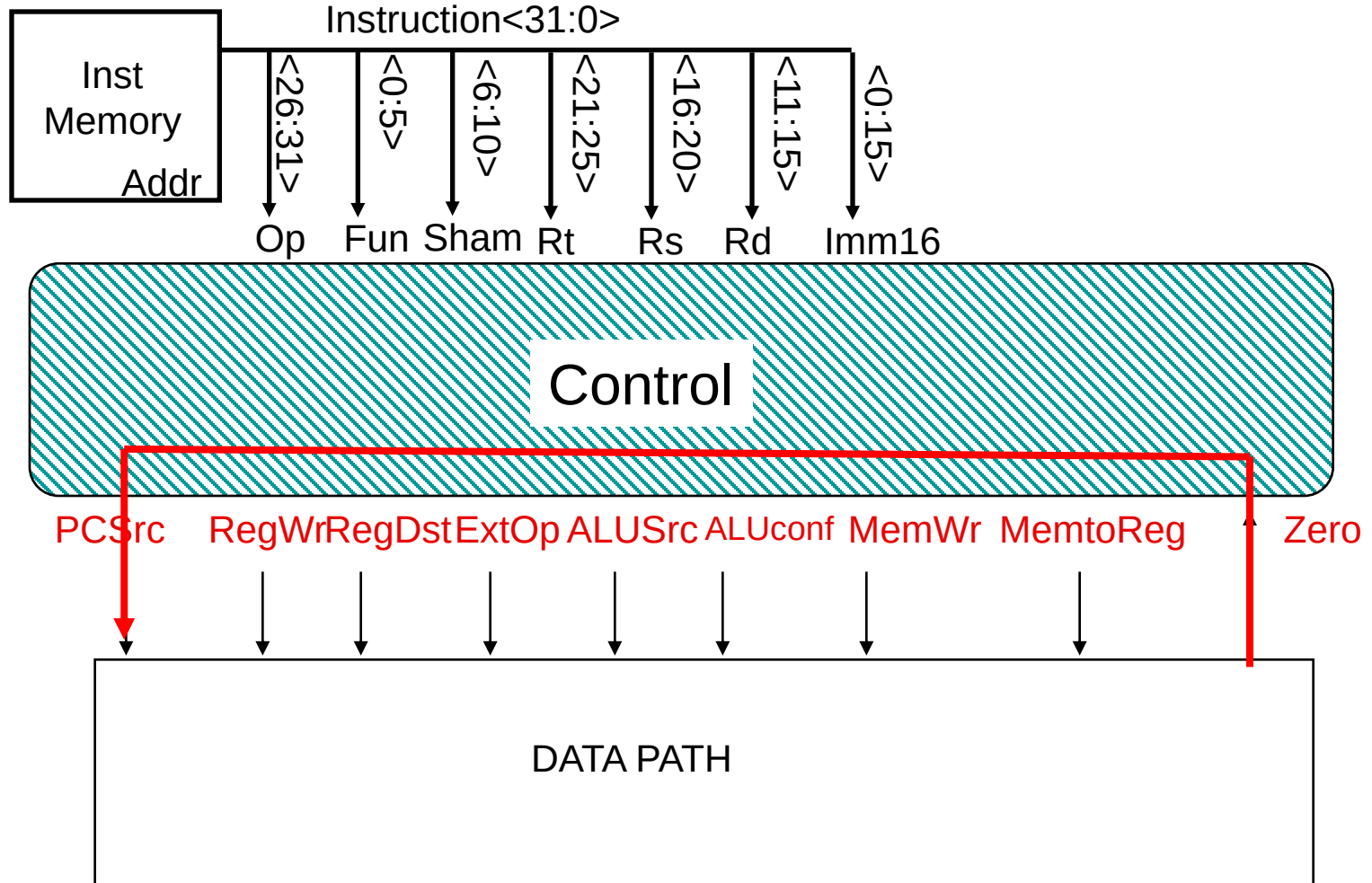
- 选择

- PCSrc(1/2) 程序计数方式控制
- ALUSrc ALU输入选择控制
- ExtOp 立即数扩展方式控制
- ALUconf ALU运算选择

- 写入控制

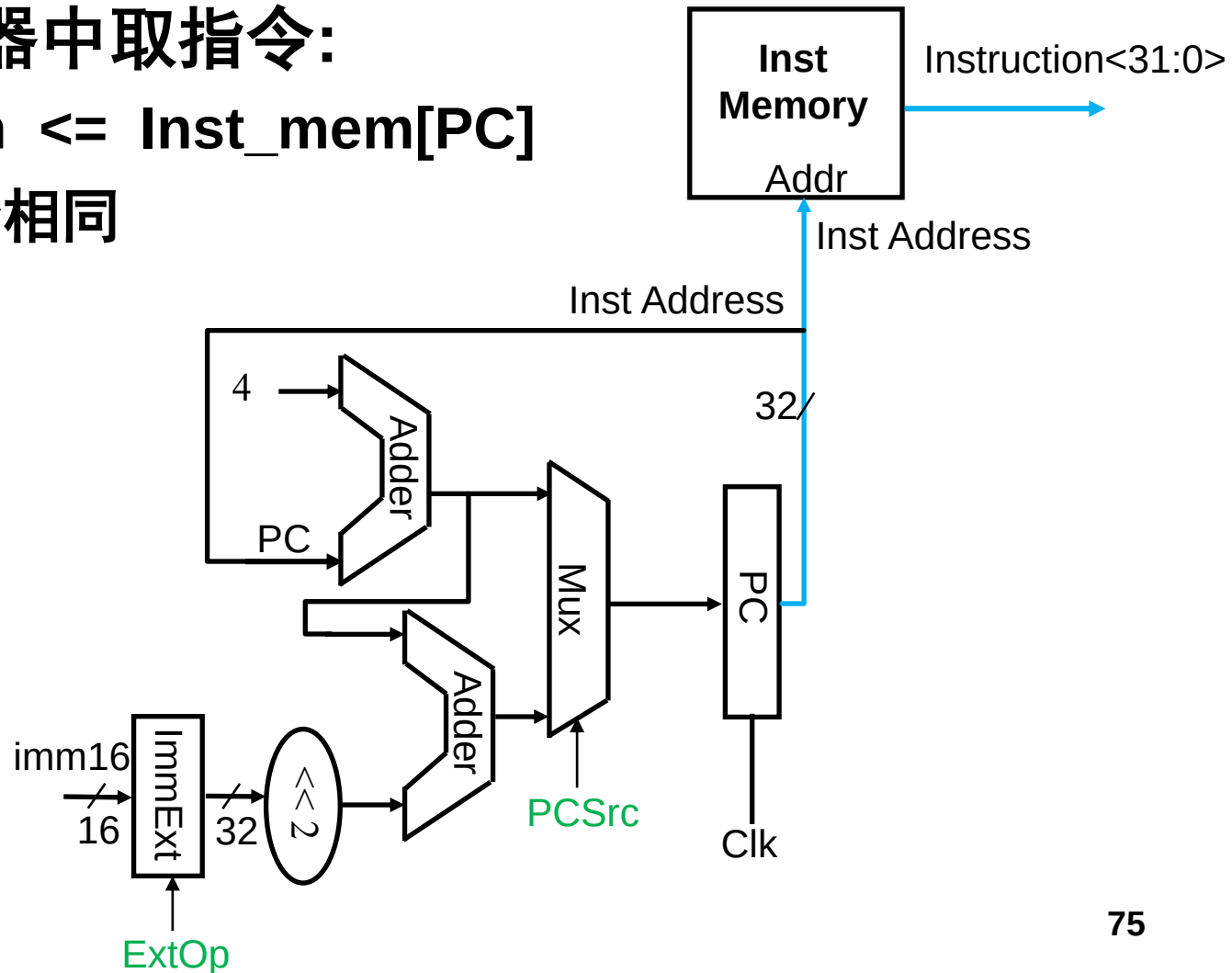
- RegWr 寄存器写入控制
- RegDst 写入寄存器选择
- MemWr 存储器写入控制
- MemtoReg 寄存器数据写入端选择

控制信号和Datapath



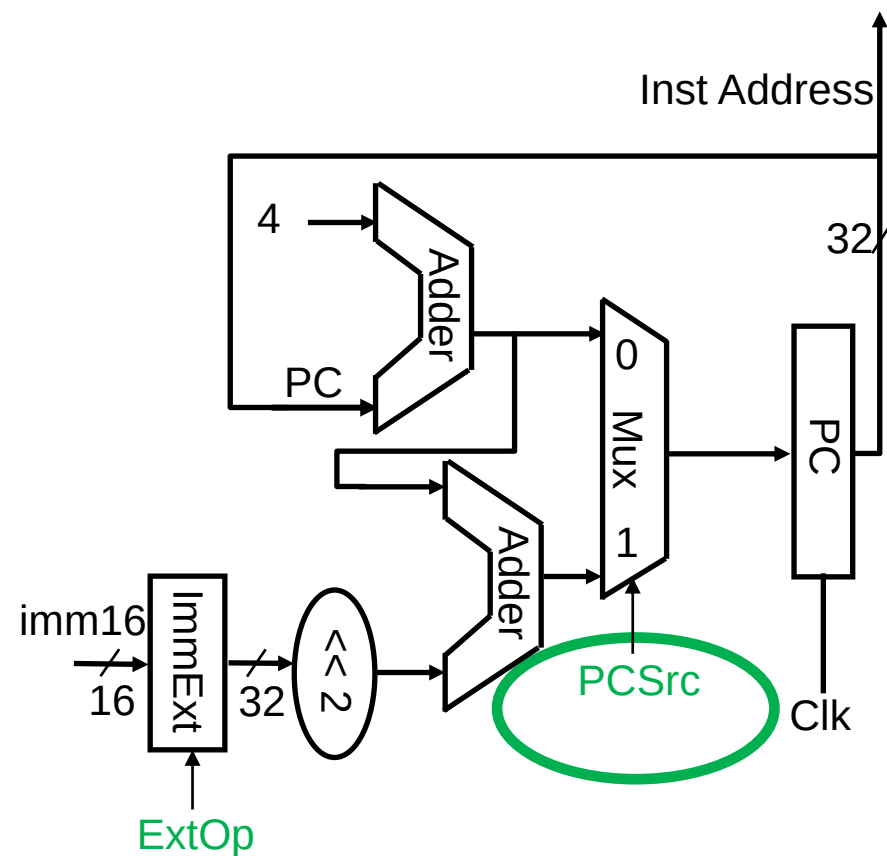
取指令过程

- 从指令存储器中取指令：
 - $\text{Instruction} \leq \text{Inst_mem}[\text{PC}]$
 - 对所有指令相同



PCSrc信号的控制逻辑

- **PCSrc:** $0 \Rightarrow PC \leq PC + 4$
 $1 \Rightarrow PC \leq PC + 4 + \{\text{SignExt}(\text{Im16}), 2b00\}$

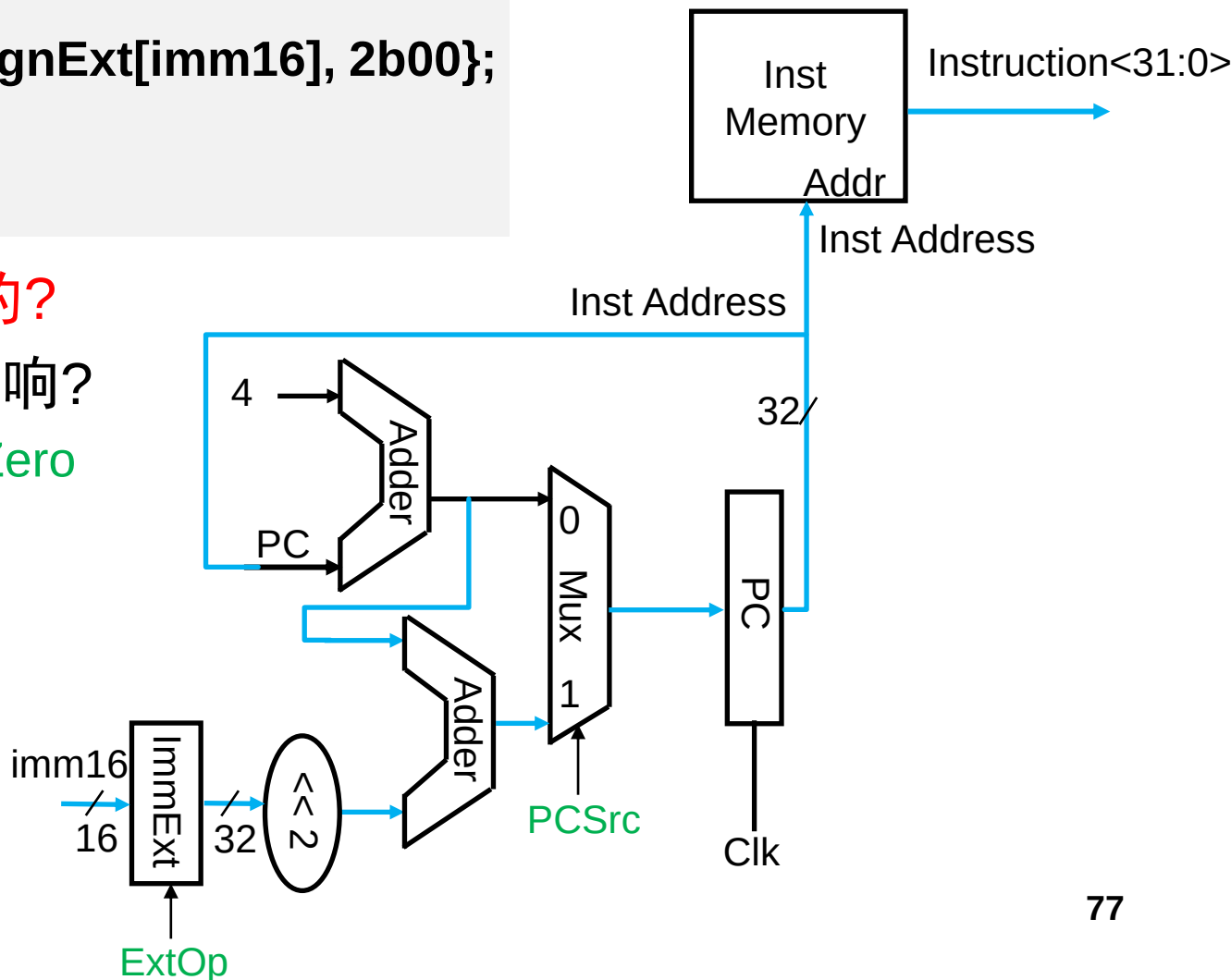


	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	1?

对于Beq的指令地址更新过程

```
if (Zero == 1)
    PC = PC + 4 + {SignExt[imm16], 2b00};
else
    PC = PC + 4;
```

- PCSrc如何实现的?
 - 受哪些因素影响?
- PCSrc = Branch AND Zero



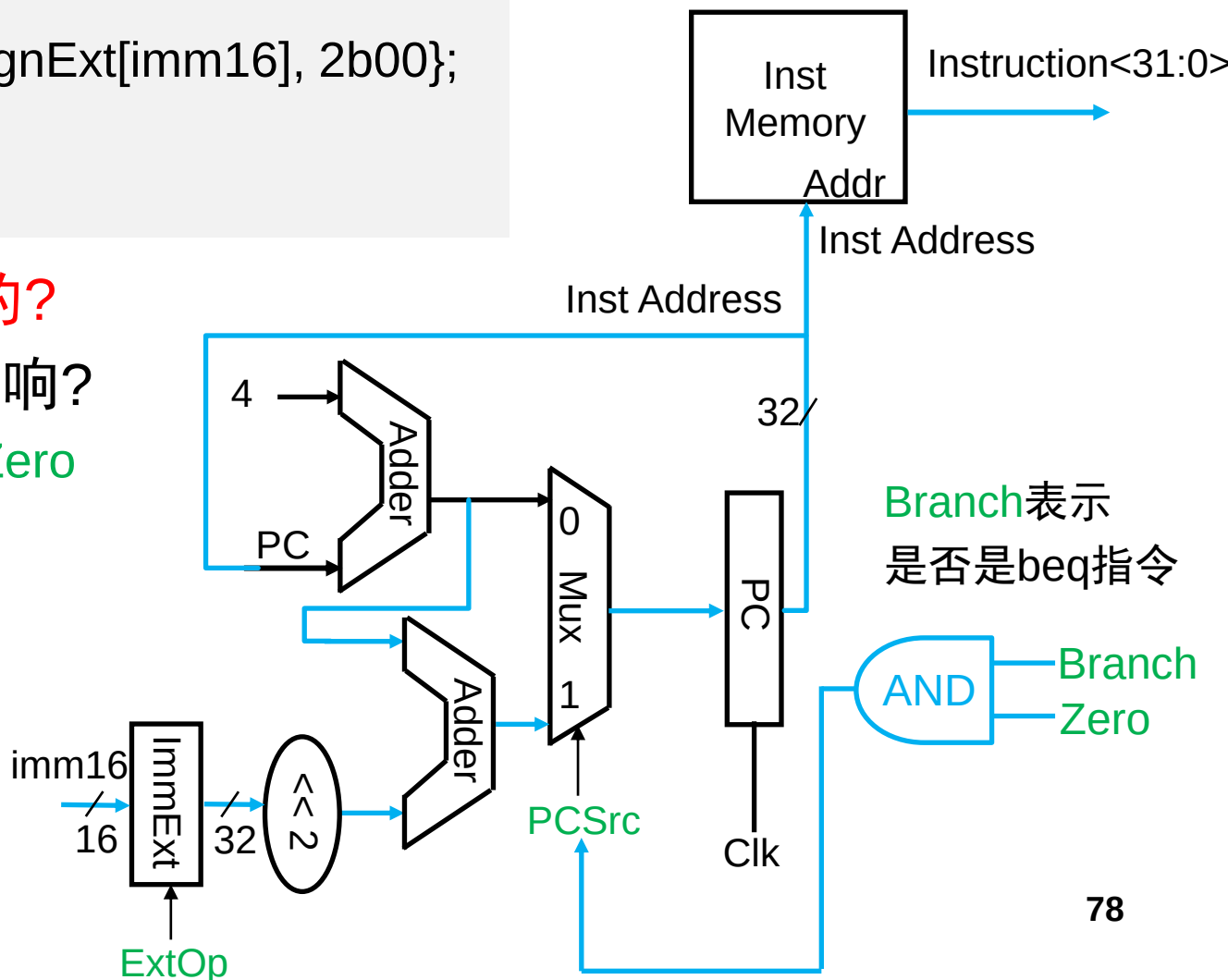
对于Beq的指令地址更新过程

```
if (Zero == 1)
    PC = PC + 4 + {SignExt[imm16], 2b00};
else
    PC = PC + 4;
```

- PCSrc如何实现的?
 - 受哪些因素影响?

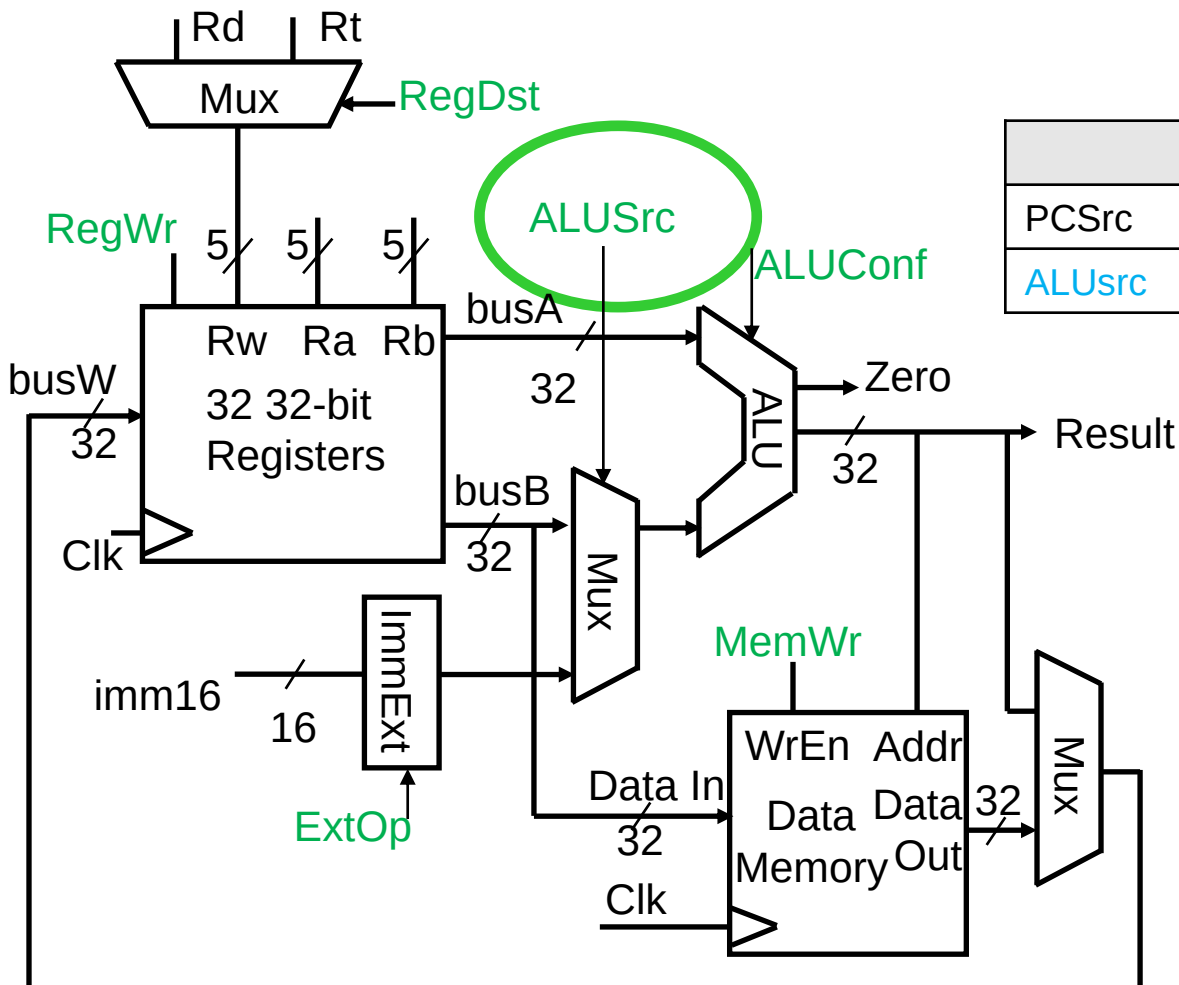
PCSrc = Branch AND Zero

Branch	Zero	MUX
0	X	0
1	0	0
1	1	1



ALUSrc信号的控制逻辑

- ALUSrc**: 0 $\Rightarrow$ reg 为ALU B输入; 1 $\Rightarrow$ 立即数为ALU B 输入



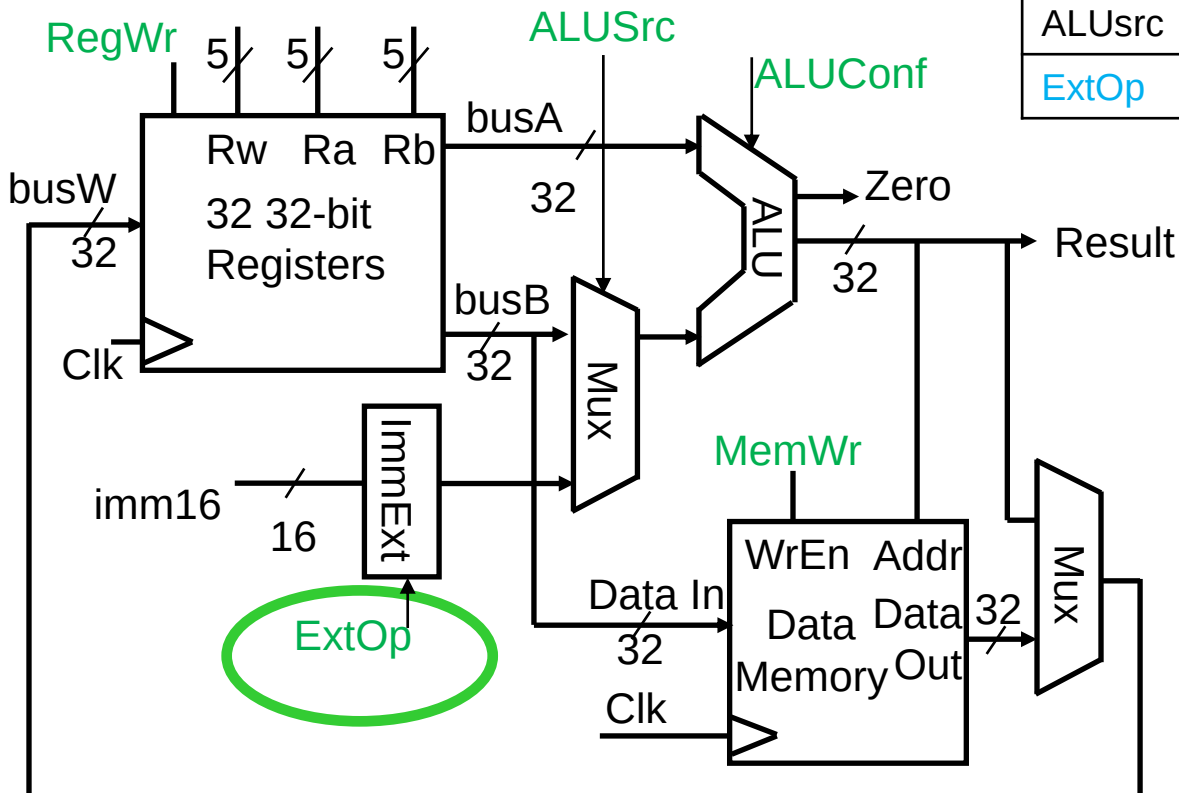
	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUSrc	0	0	1	1	1	0

ExtOp: 立即数扩展方式控制

- ExtOp

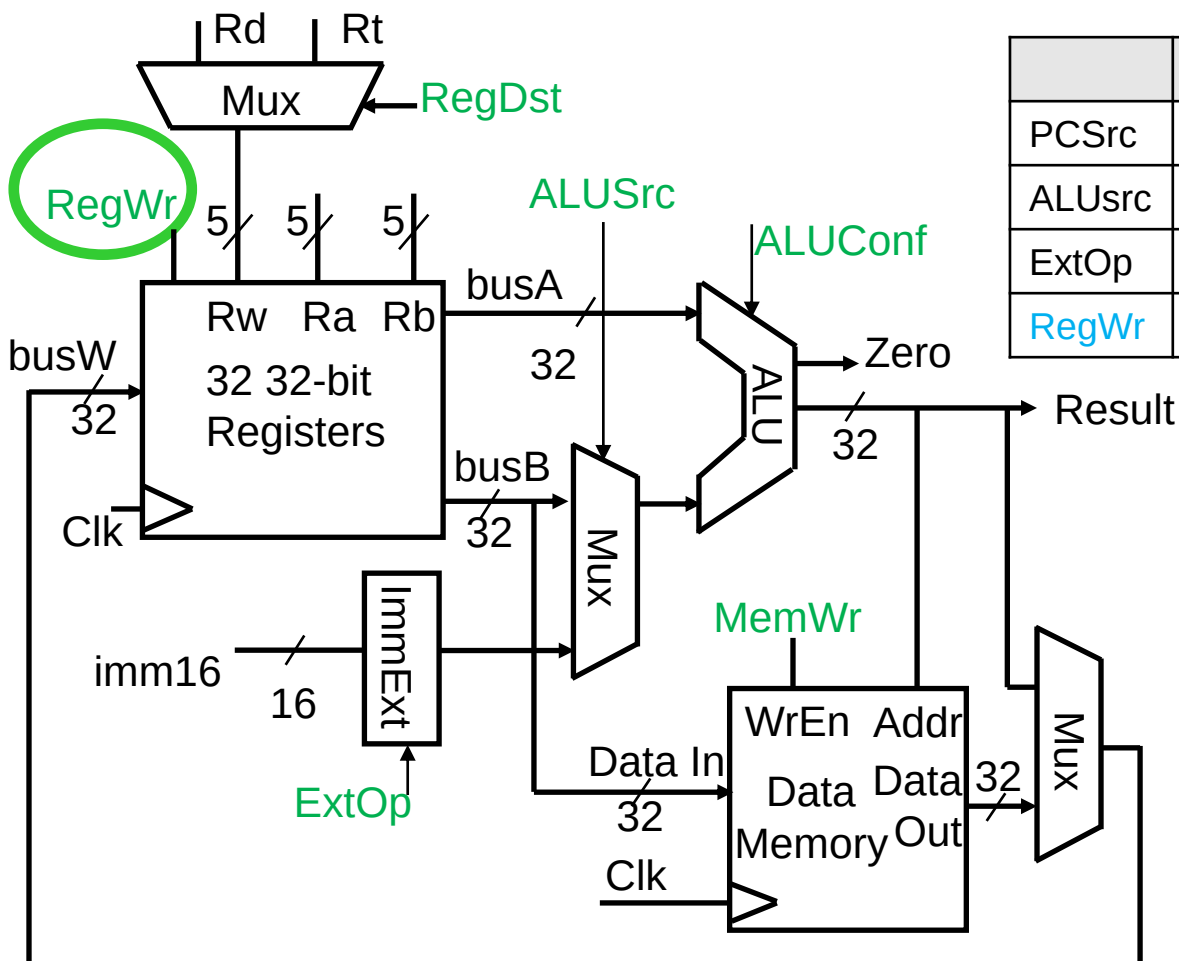
- 0 $\Rightarrow$ “立即数高位填0”
- 1 $\Rightarrow$ “立即数符号扩展”

	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUSrc	0	0	1	1	1	0
ExtOp	X	X	0	1	1	1



RegWr: 寄存器写入控制

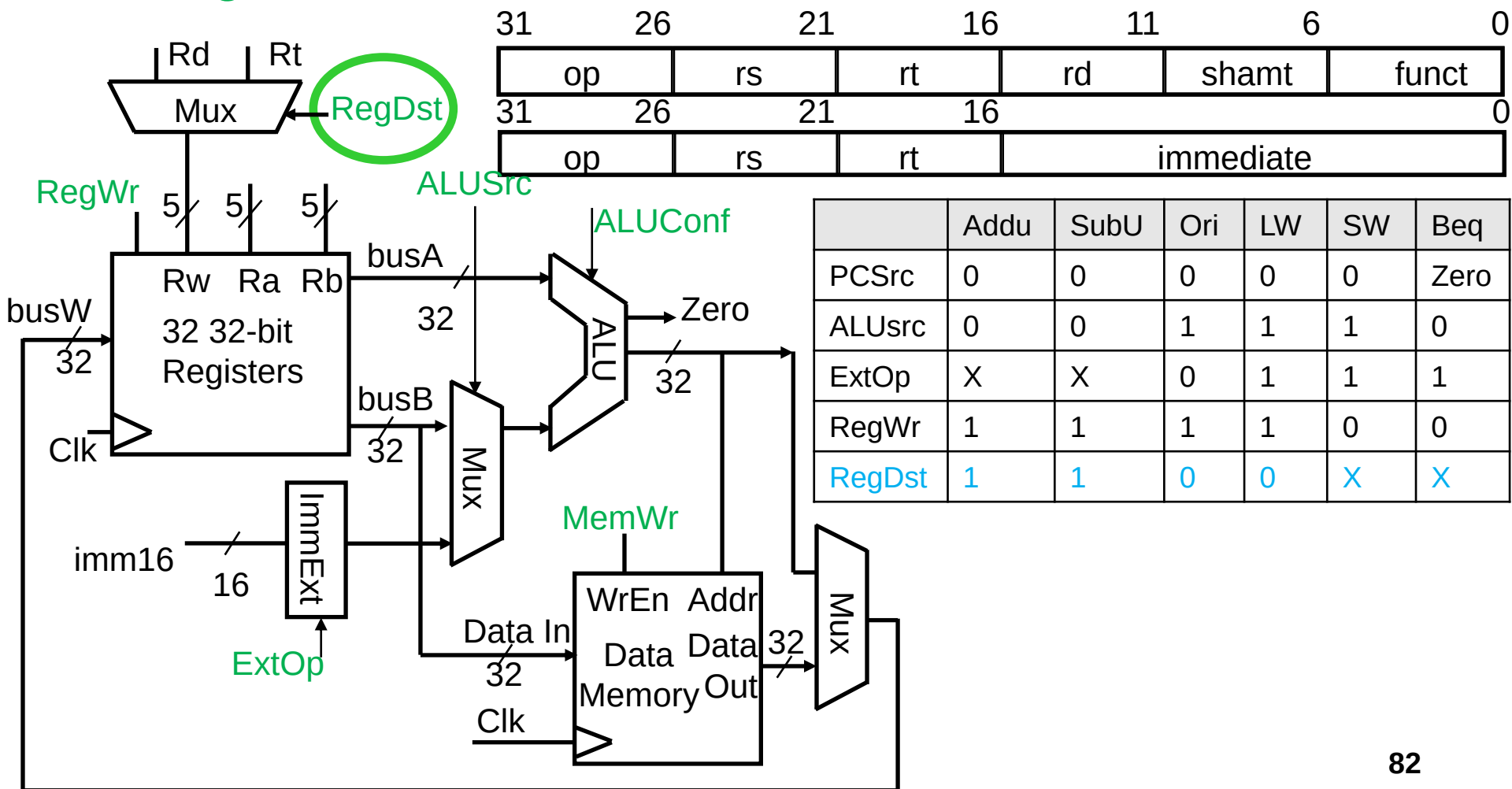
- RegWr: 1 $\Rightarrow$ 写寄存器使能



	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUSrc	0	0	1	1	1	0
ExtOp	X	X	0	1	1	1
RegWr	1	1	1	1	0	0

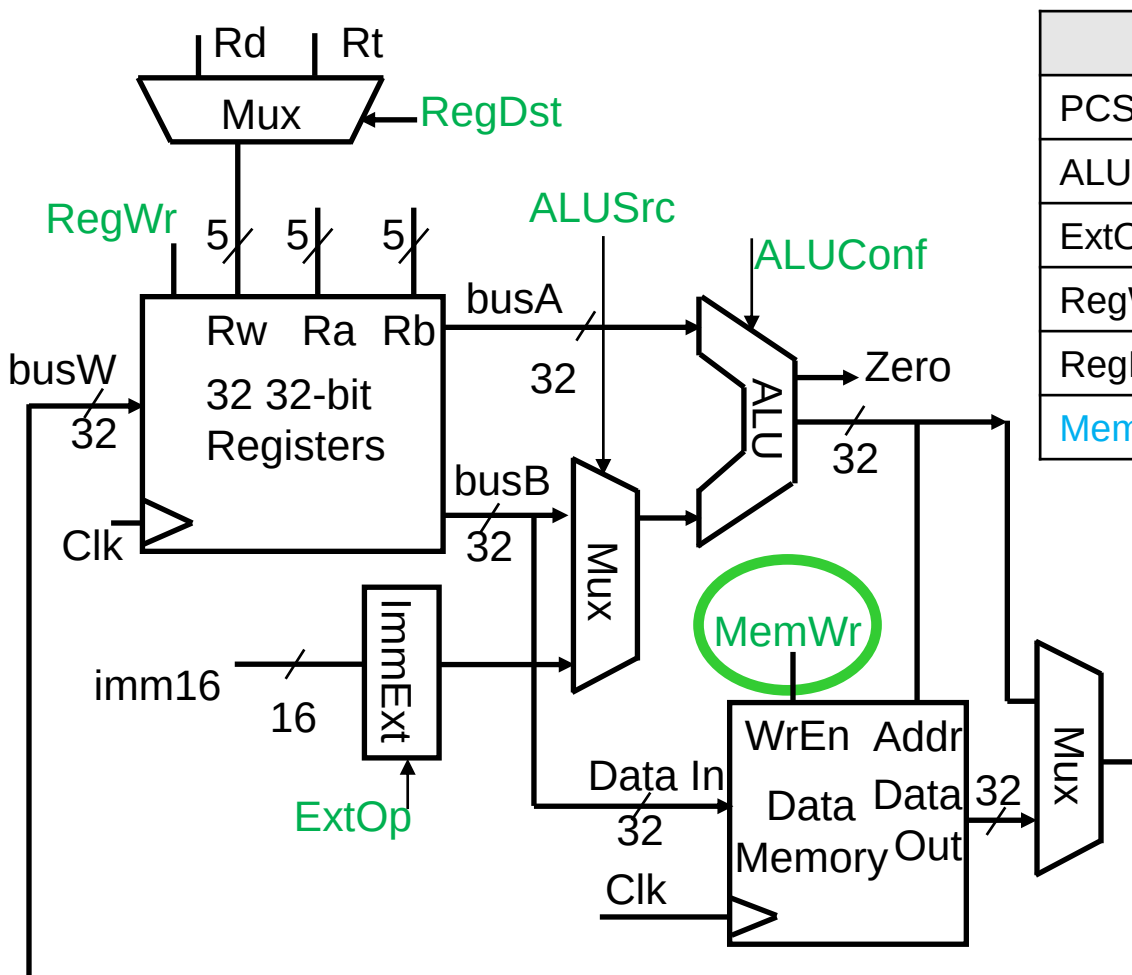
RegDst: 写入寄存器选择

- **RegDst:** 0 $\Rightarrow$ “rt”; 1 $\Rightarrow$ “rd”



MemWr: 存储器写入控制

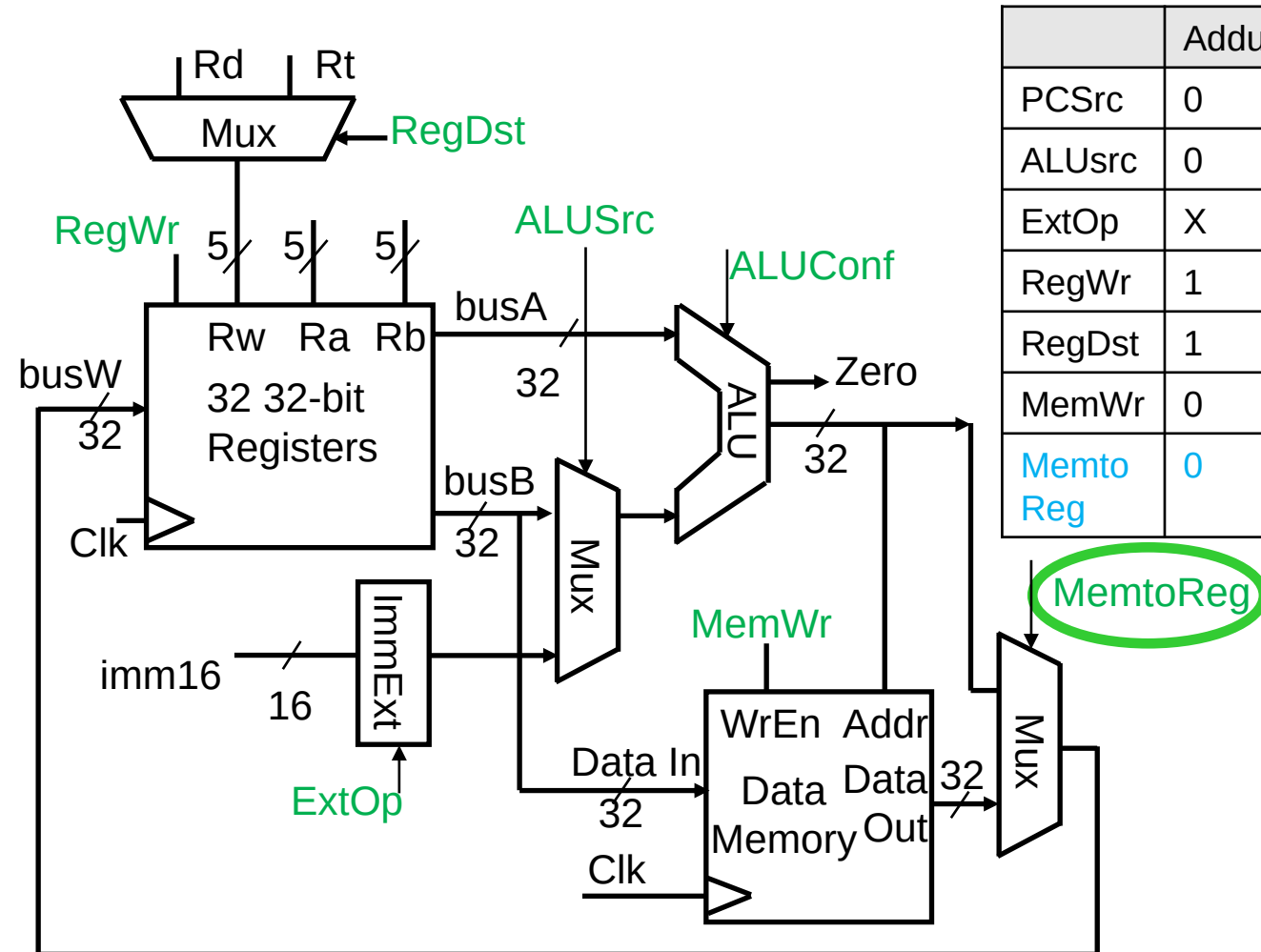
- MemWr: 1 $\Rightarrow$ 写存储器使能



	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUSrc	0	0	1	1	1	0
ExtOp	X	X	0	1	1	1
RegWr	1	1	1	1	0	0
RegDst	1	1	0	0	X	X
MemWr	0	0	0	0	1	0

MemtoReg: 寄存器数据写入端选择信号

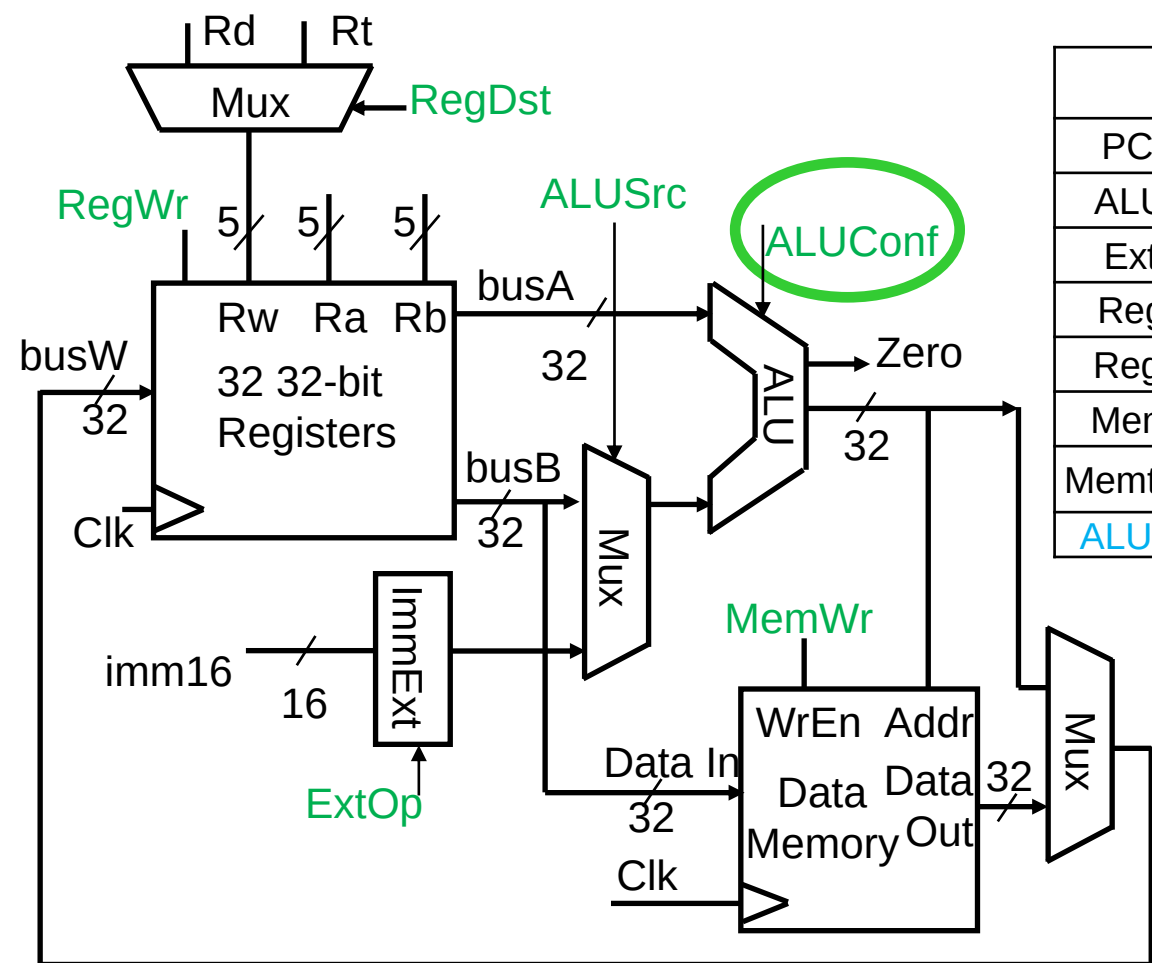
- MemtoReg:** 0 $\Rightarrow$ ALU; 1 $\Rightarrow$ Mem



	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUSrc	0	0	1	1	1	0
ExtOp	X	X	0	1	1	1
RegWr	1	1	1	1	0	0
RegDst	1	1	0	0	X	X
MemWr	0	0	0	0	1	0
MemtoReg	0	0	0	1	X	X

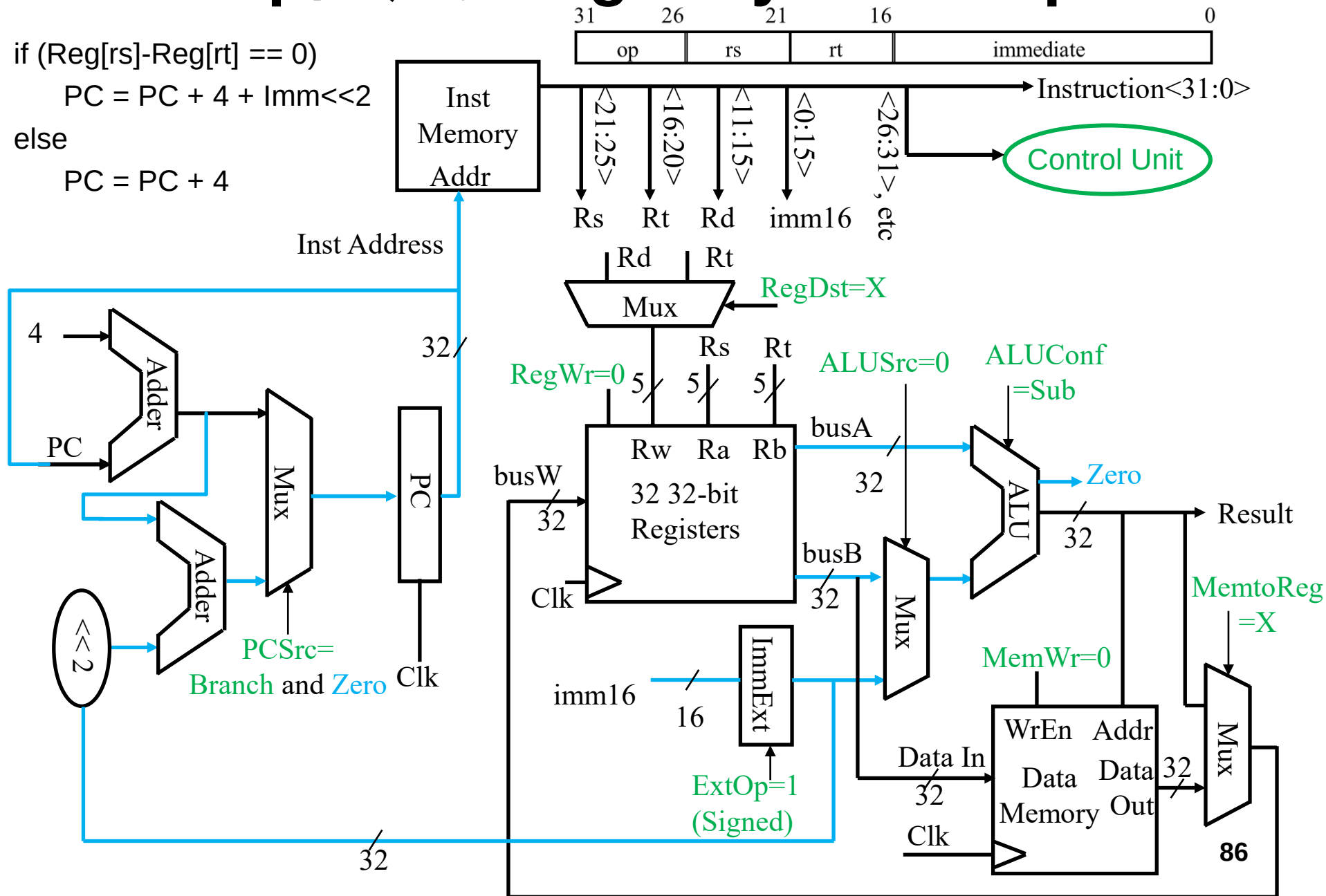
ALUConf: ALU运算选择

- ALUConf:** 0 $\Rightarrow$ “add”, 1 $\Rightarrow$ “sub”, 2 $\Rightarrow$ “or”

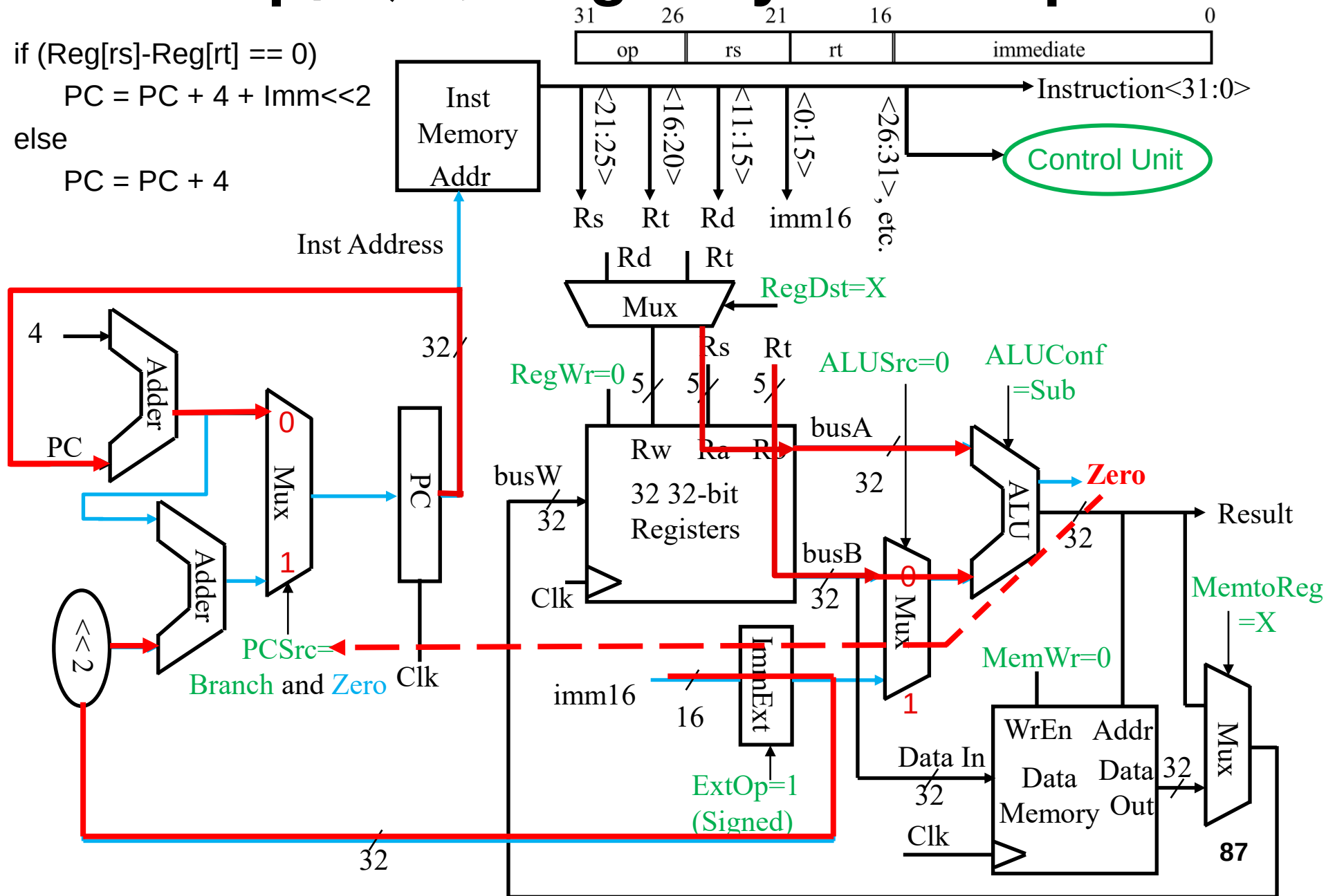


	Addu	SubU	Ori	LW	SW	Beq
PCSrc	0	0	0	0	0	Zero
ALUsrc	0	0	1	1	1	0
ExtOp	X	X	0	1	1	1
RegWr	1	1	1	1	0	0
RegDst	1	1	0	0	X	X
MemWr	0	0	0	0	1	0
MemtoReg	0	0	0	1	X	X
ALUConf	0	1	2	0	0	1

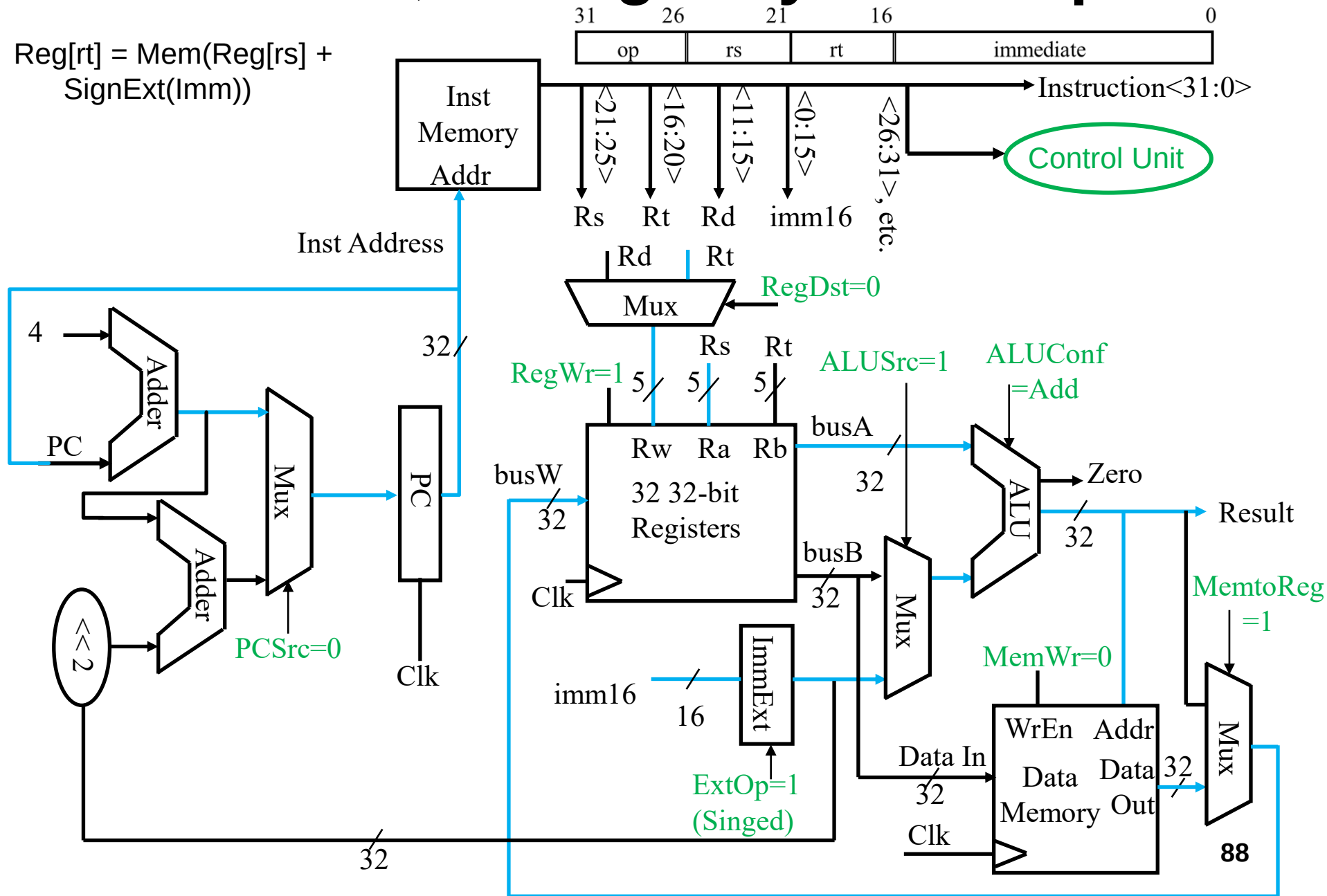
Beq指令的Single Cycle Datapath



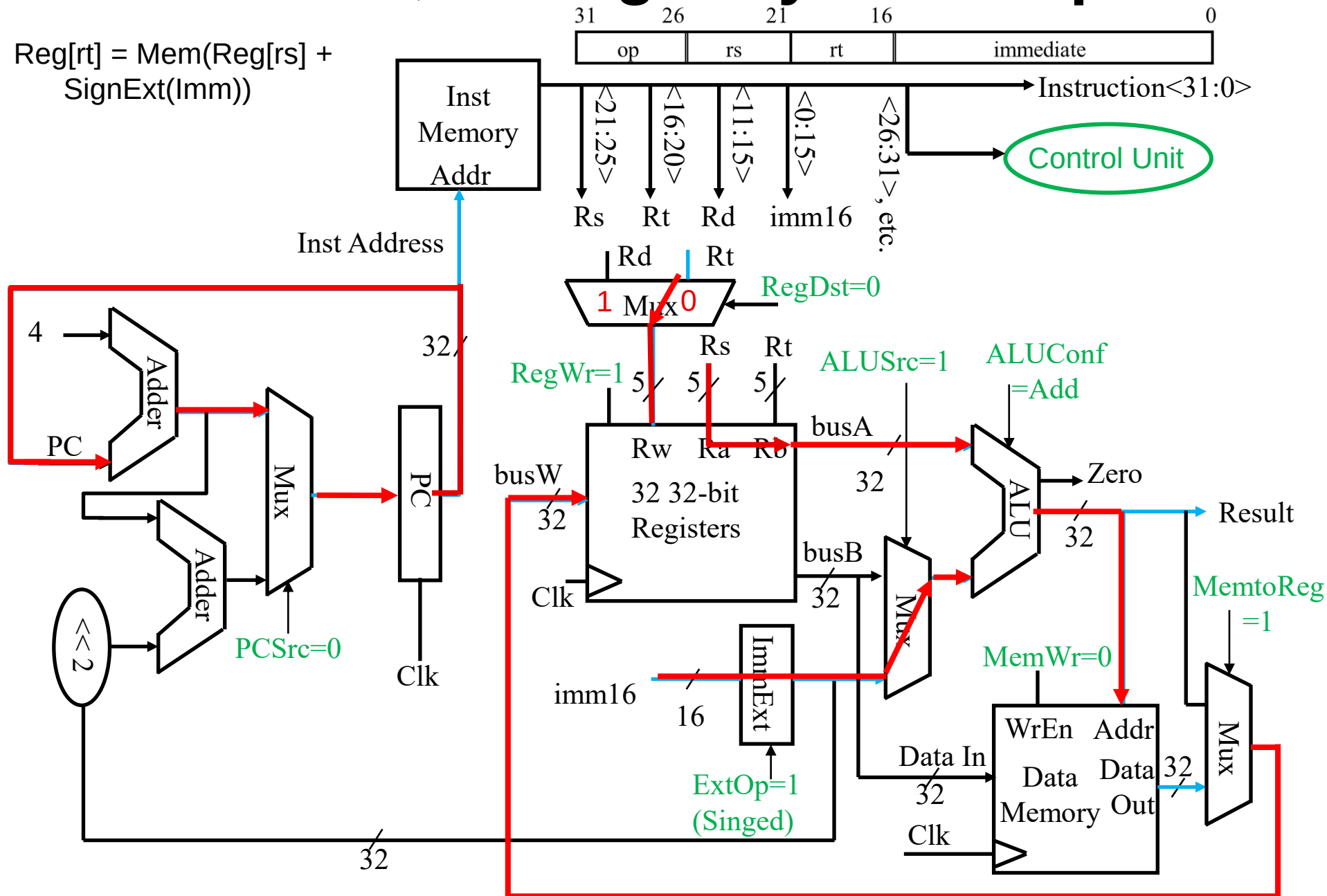
Beq指令的Single Cycle Datapath



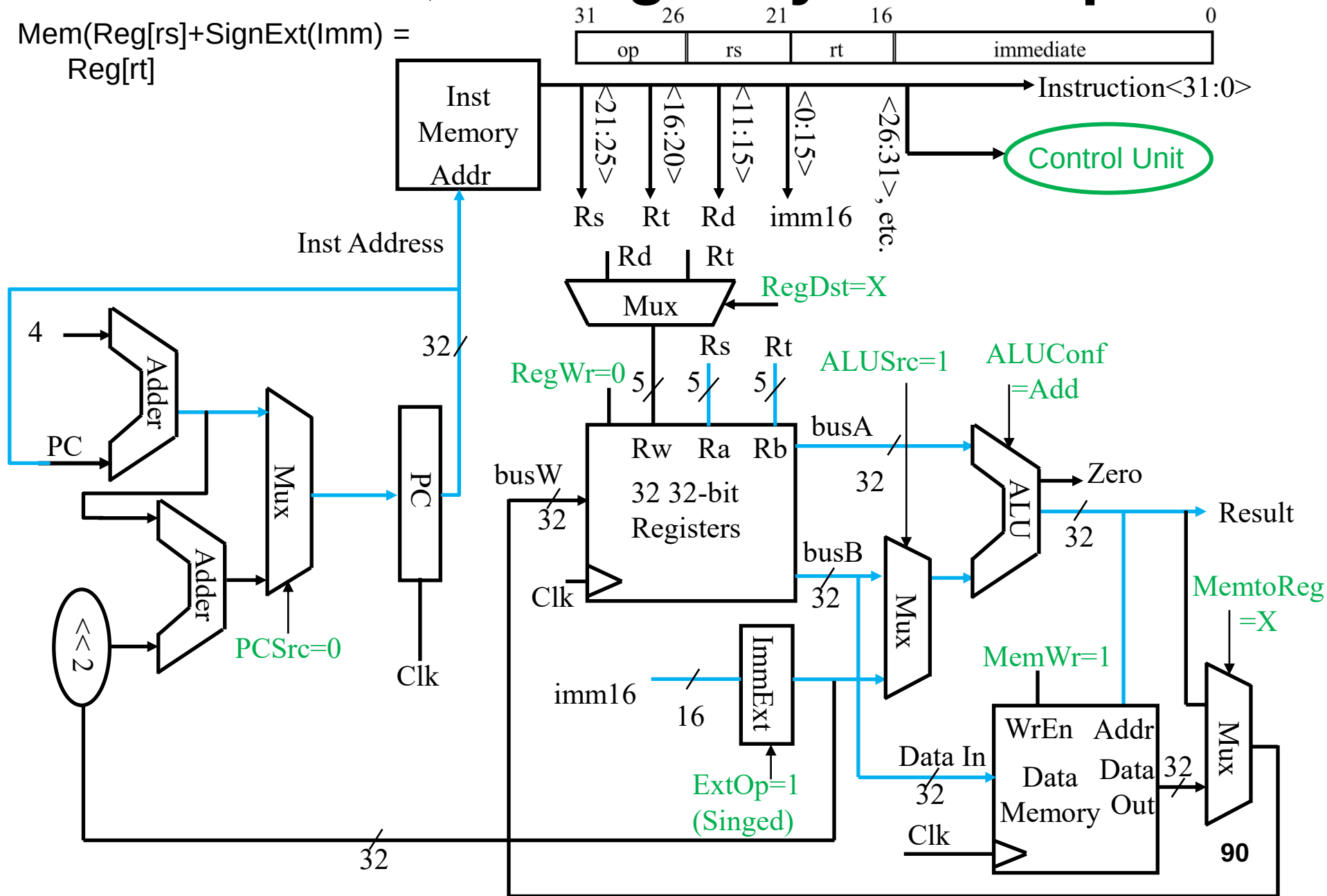
Load指令的Single Cycle Datapath



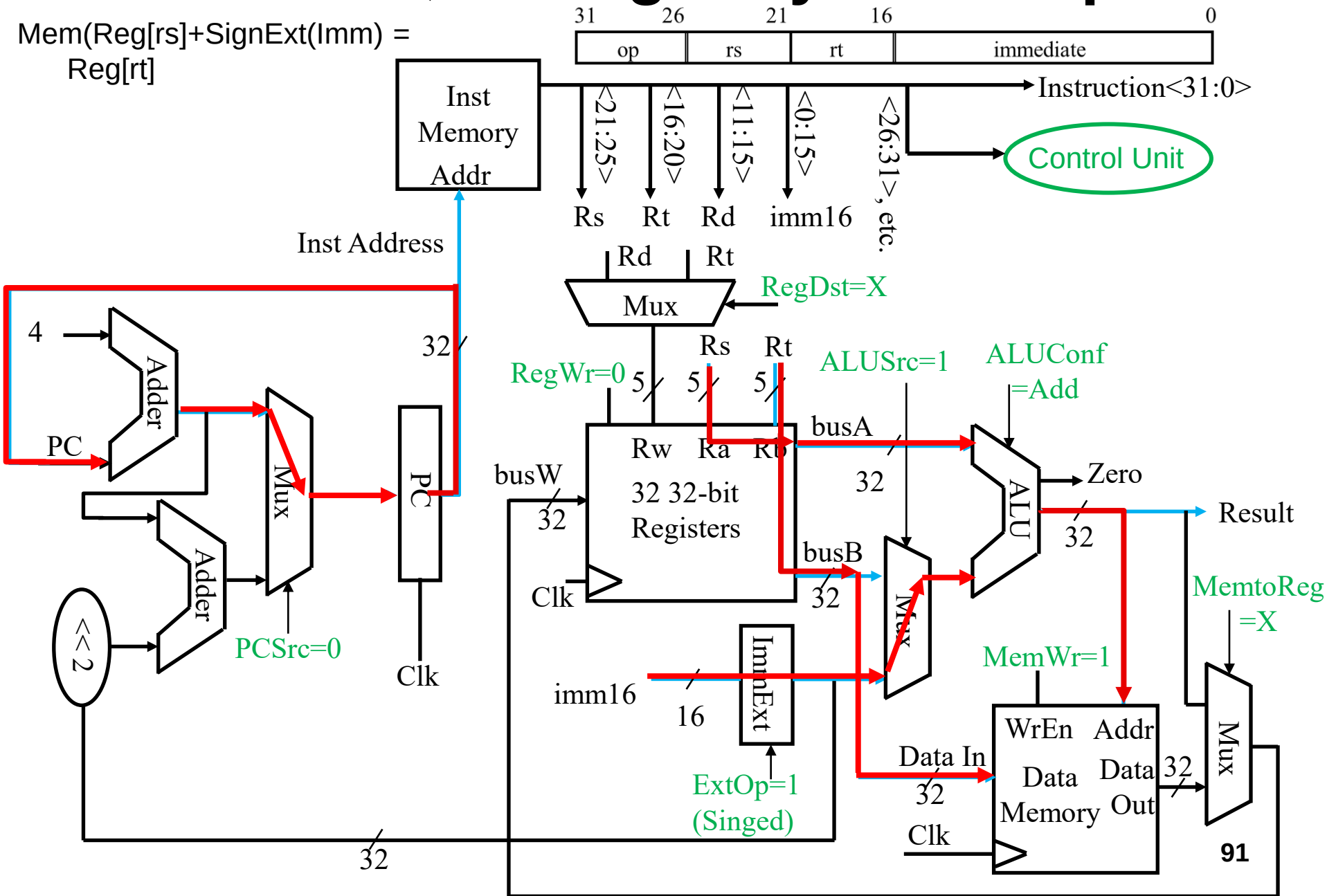
Load指令的Single Cycle Datapath



Store指令的Single Cycle Datapath

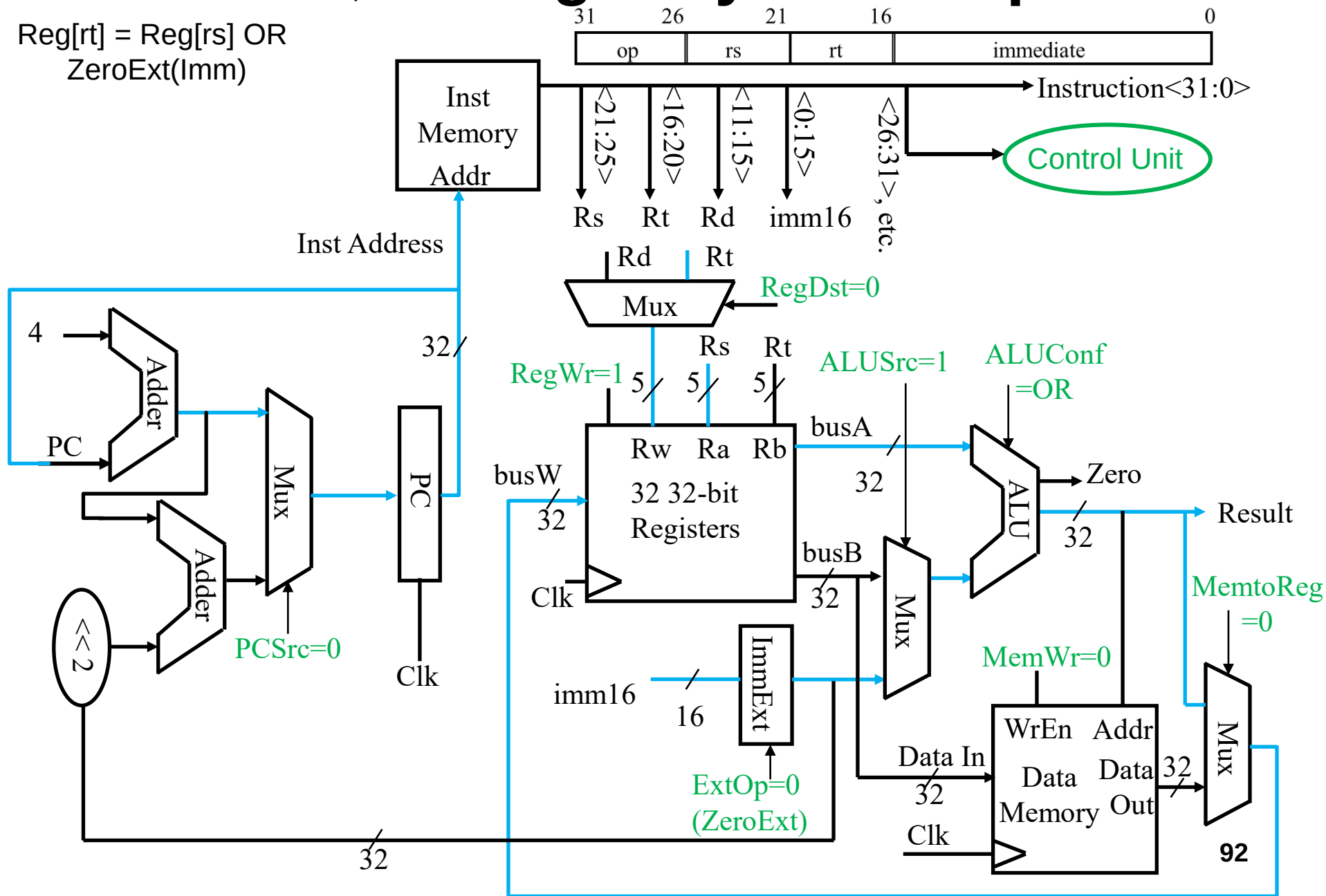


Store指令的Single Cycle Datapath



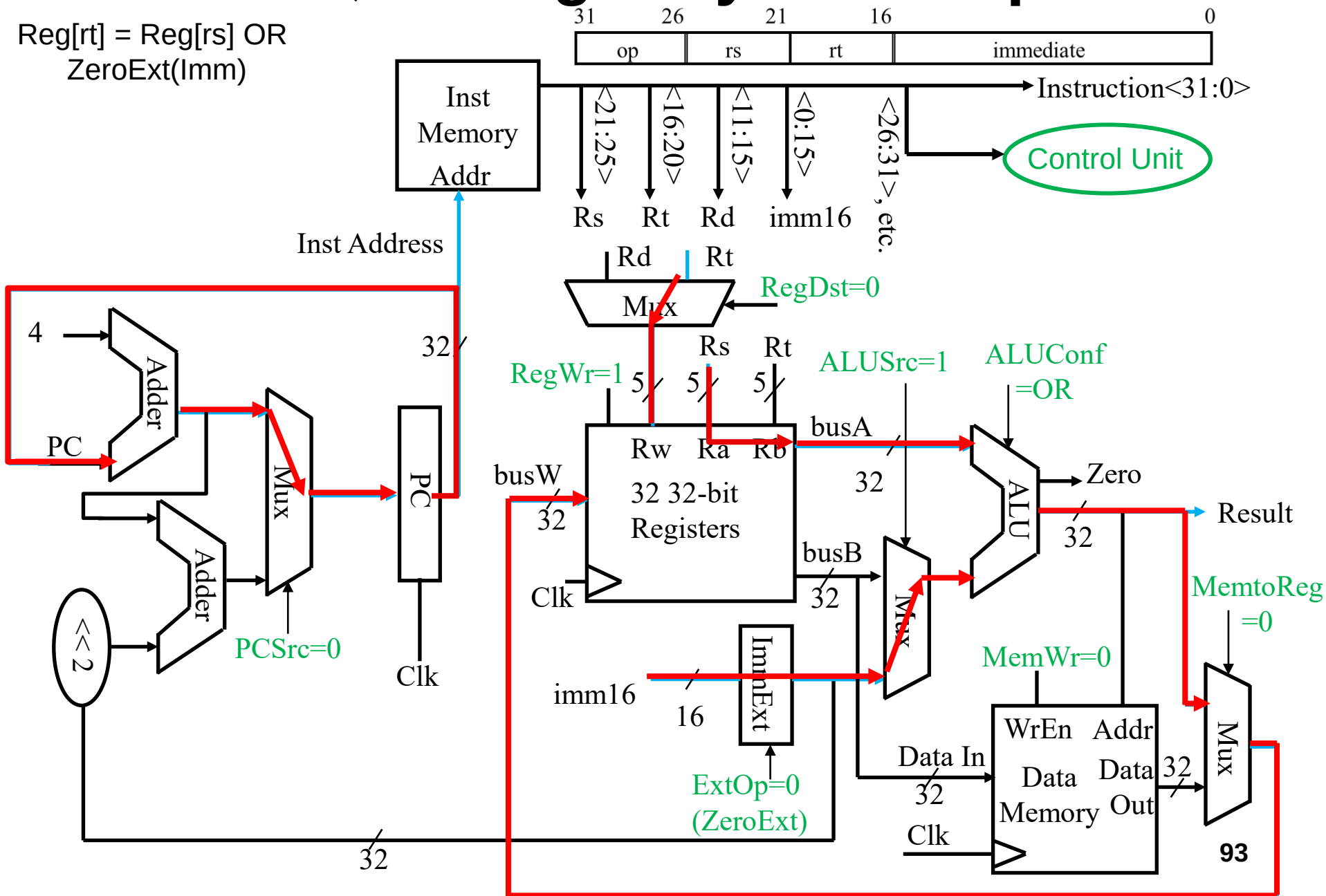
Ori指令的Single Cycle Datapath

$\text{Reg}[rt] = \text{Reg}[rs] \text{ OR } \text{ZeroExt}(\text{Imm})$



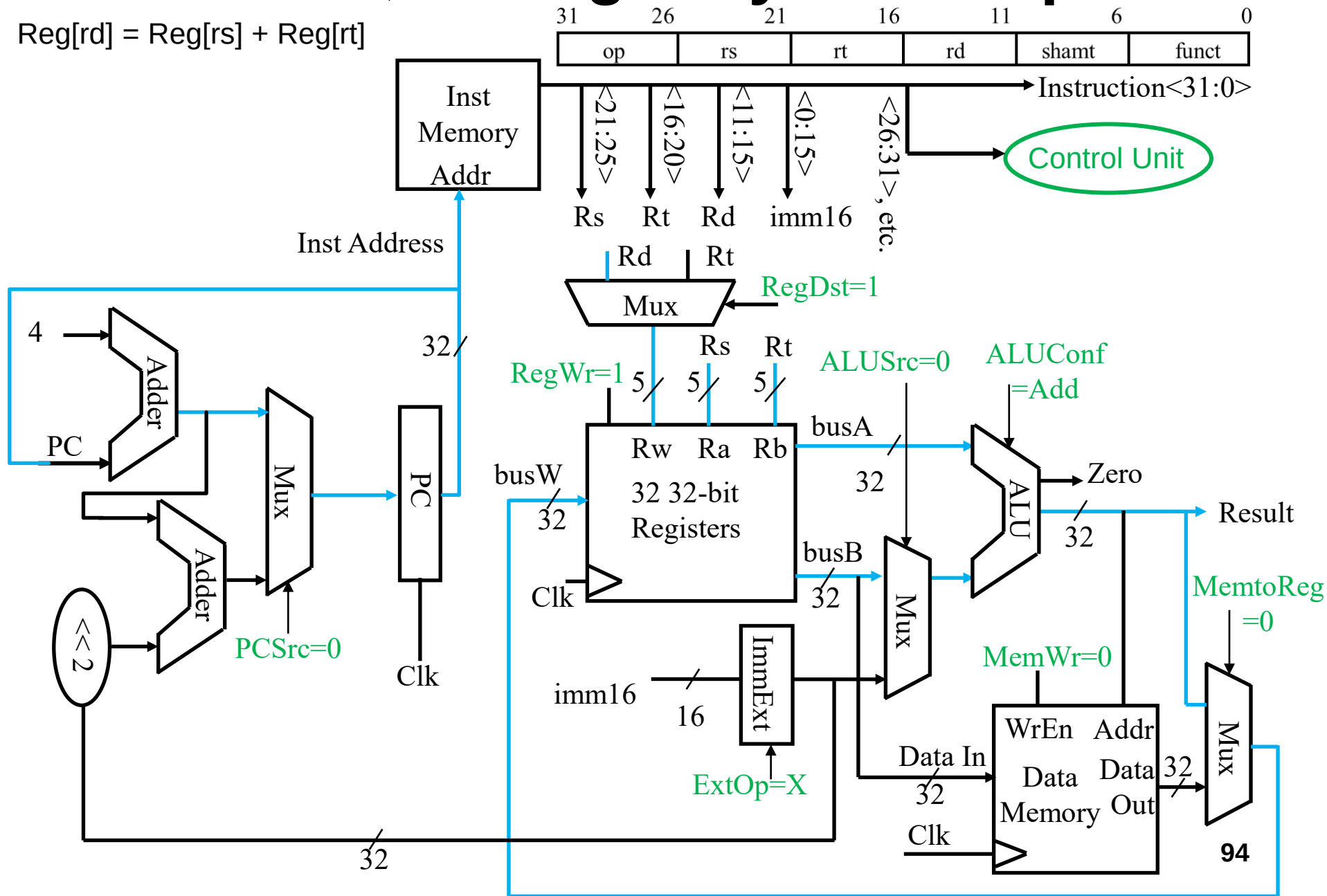
Ori指令的Single Cycle Datapath

$\text{Reg}[rt] = \text{Reg}[rs] \text{ OR } \text{ZeroExt}(\text{Imm})$



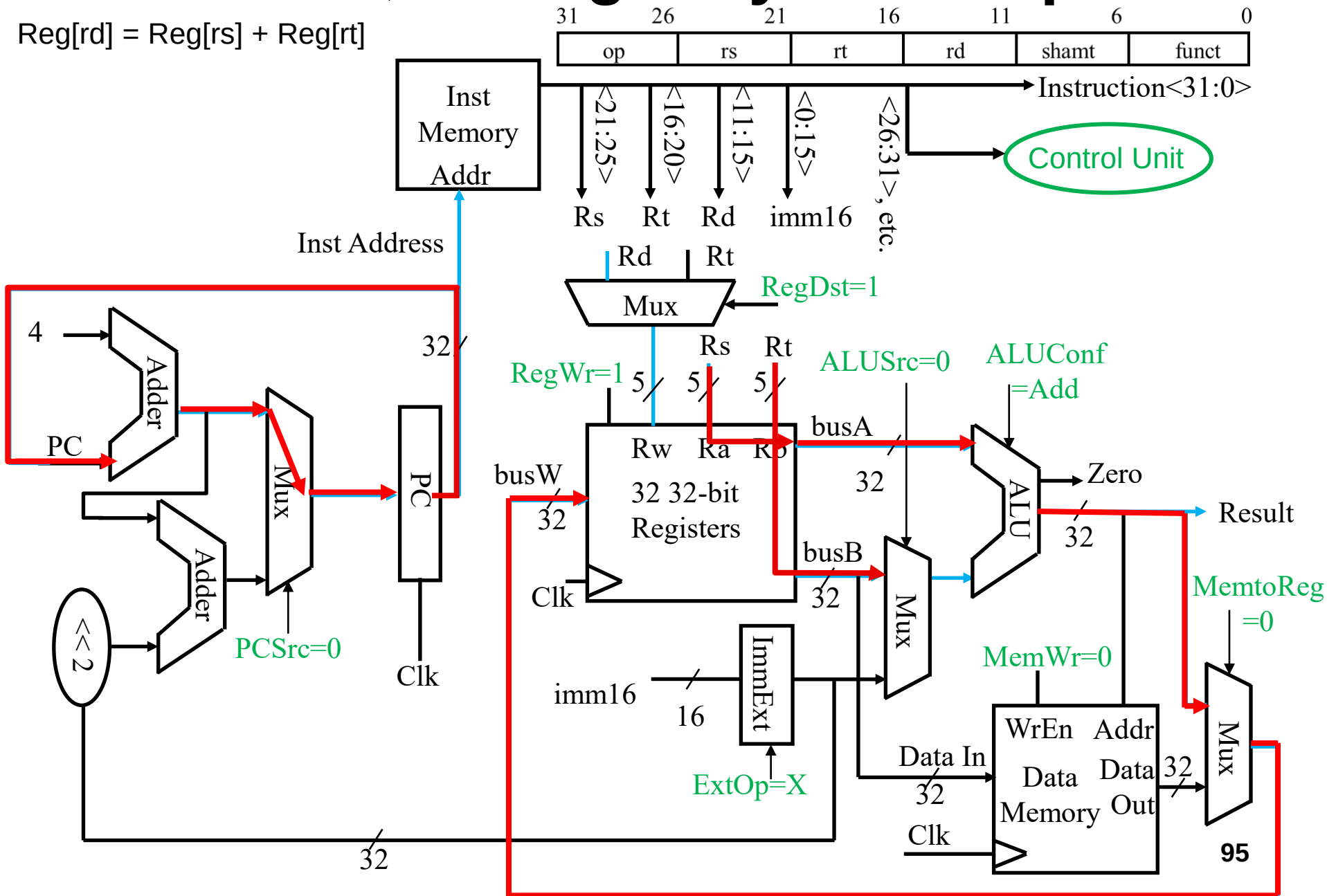
Add指令的Single Cycle Datapath

$$\text{Reg}[rd] = \text{Reg}[rs] + \text{Reg}[rt]$$



Add指令的Single Cycle Datapath

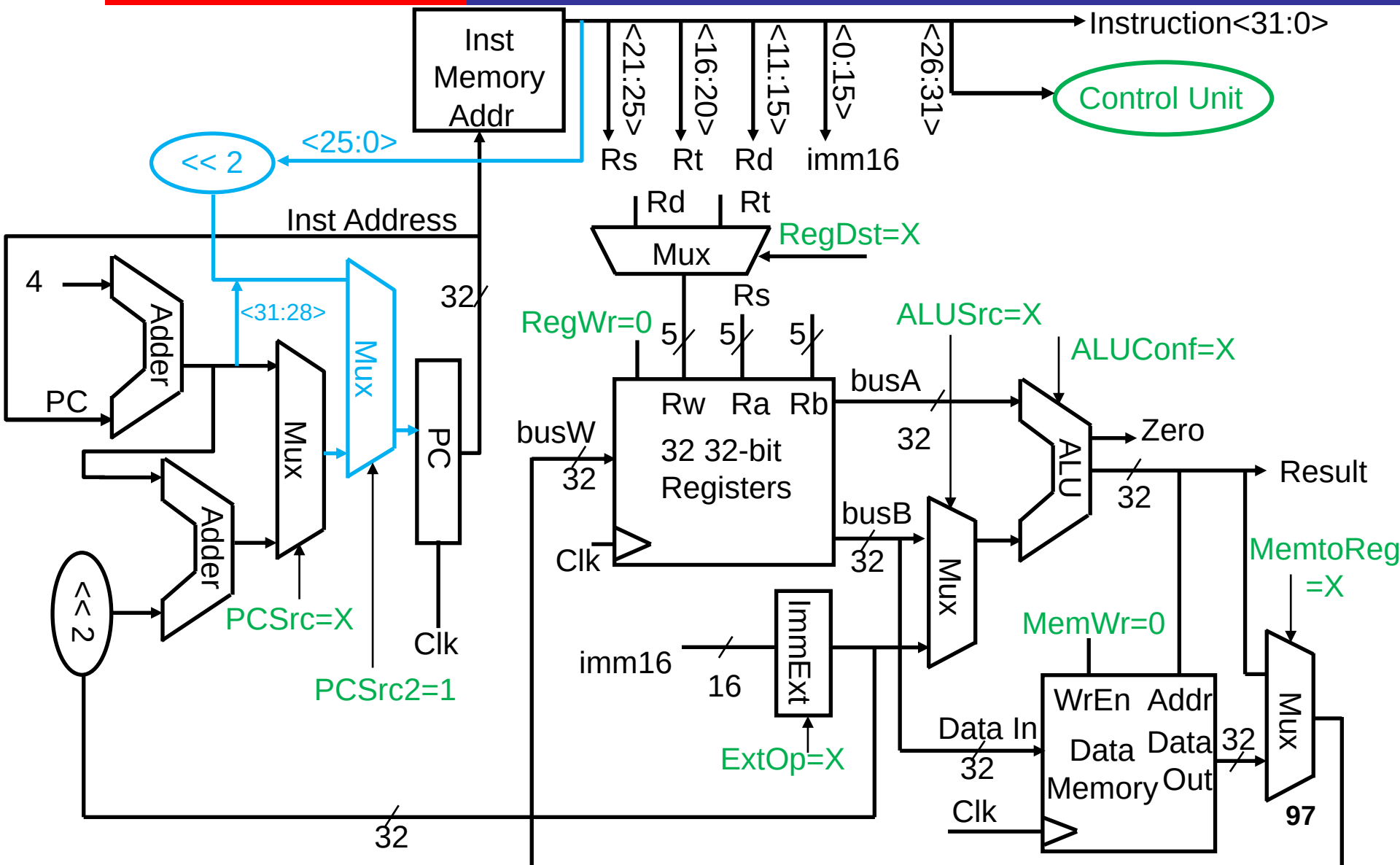
$$\text{Reg}[rd] = \text{Reg}[rs] + \text{Reg}[rt]$$



MIPS子集控制信号小结

inst	Register Transfer	
ADD	$R[rd] \leq R[rs] + R[rt];$	$PC \leq PC + 4$ ALUSrc = RegB, ALUConf = "add", RegDst = rd, RegWr, PCSrc = "+4"
SUB	$R[rd] \leq R[rs] - R[rt];$	$PC \leq PC + 4$ ALUSrc = RegB, ALUConf = "sub", RegDst = rd, RegWr, PCSrc = "+4"
ORI	$R[rt] \leq R[rs] + \text{zero_ext}(\text{Imm16});$	$PC \leq PC + 4$ ALUSrc = Im, Extop = "Z", ALUConf = "or", RegDst = rt, RegWr, PCSrc = "+4"
LOAD	$R[rt] \leq \text{MEM}[R[rs] + \text{sign_ext}(\text{Imm16})];$	$PC \leq PC + 4$ ALUSrc = Im, Extop = "Sn", ALUConf = "add", MemtoReg, RegDst = rt, RegWr, PCSrc = "+4"
STORE	$\text{MEM}[R[rs] + \text{sign_ext}(\text{Imm16})] \leq R[rs];$	$PC \leq PC + 4$ ALUSrc = Im, Extop = "Sn", ALUConf = "add", MemWr, PCSrc = "+4"
BEQ	if ($R[rs] == R[rt]$) then $PC \leq PC + 4 + \{\text{sign_ext}(\text{Imm16})\}, 00'b2$ else $PC \leq PC + 4$ Extop = "Sn", ALUConf = "sub", PCSrc = "Br" and "Zero"	

扩展实现：跳转指令



MIPS子集控制信号小结

See Appendix A

	func	10 0000	10 0010	We Don't Care :-)			
	op	00 0000	00 0000	00 1101	10 0011	10 1011	00 0100 00 0010
		add	sub	ori	lw	sw	beq jump
RegDst		1	1	0	0	x	x x
ALUSrc		0	0	1	1	1	0 x
MemtoReg		0	0	0	1	x	x x
RegWrite		1	1	1	1	0	0 0
MemWrite		0	0	0	0	1	0 0
PCSrc		0	0	0	0	0	ZERO x
PCSrc2		0	0	0	0	0	0 1
ExtOp		x	x	0	1	1	1 x
ALUConf<2:0>		Add	Subtract	Or	Add	Add	Subtract xxx

	31	26	21	16	11	6	0							
R-type	op		rs		rt		rd		shamt		funct		add, sub	
I-type	op		rs		rt		immediate						ori, lw, sw, beq	
J-type	op		target address										jump	98

本节课小结

- **5个步骤设计单周期处理器**
 1. 分析指令集 => datapath 的需求
 2. 确定datapath的组成模块
 3. 组装datapath
 4. 通过分析指令确定影响寄存器转移的控制信号.
 5. 设计控制信号
- **MIPS 实现这样一个功能比较容易**
 - 指令结构简单,等长; 源寄存器, 立即数总是在相同的位置
 - 只对寄存器内容或者立即数进行操作,操作类型少
- **单周期datapath=> CPI=1, 时钟周期长**

本节课小结

- **单周期MIPS处理器执行阶段**

1. 取指令 (Instruction Fetch, IF)
2. 指令译码和控制信号生成 (Instruction Decode, ID)
3. 操作执行 (Execution, EX)
4. 内存访问 (Memory Access, MEM)
5. 写回寄存器堆 (Write Back, WB)

作业

- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区